RamboQuant

Complete Design Guide

2026-10-21

RamboQuant — Complete Design Guide

Generated	Revision	Website
See PDF cover	See `git log`	ramboq.com

Tech Stack — SvelteKit · Litestar · PostgreSQL · async SQLAlchemy 2.x · KiteTicker WebSocket · Kite / Dhan / Groww adapters · MCP · Gemini.

About the author

Ramana R Ambore, FRM — Platform Architect & Quantitative Developer for **RamboQuant LLP** · *AI-augmented · built with Claude Code*.

30+ years across mainframe modernization and cloud-native financial platforms, including **19 years at Fidelity Investments**. Currently **Principal System Analyst** at Fidelity — leading billing-platform modernization on AWS + Snowflake and distributed fee-calculation engines — and concurrently the **platform architect + quantitative developer** building **RamboQuant** end-to-end (live at **ramboq.com**), applying a **forward-deployment-engineer ethos** (end-to-end ownership, tight feedback loops, ship-what-works) and leveraging **Claude Code** as a force multiplier for platform-scale solo output.

FRM (GARP, 2022) and **CFA Level 3** candidate: the derivatives-risk, options-pricing (Black-Scholes / Greeks), and portfolio-analytics theory from those programs materialize directly as RamboQuant's derivatives layer, hedge-proxy β regression, and units-based NAV accounting. The Master's in Computer Science + Fidelity engineering discipline (distributed systems, event-driven architecture, legacy-modernization patterns) keeps a multi-broker, real-time, single-operator platform correct and fast. **NTT Innovation Award** recipient (top-40 global innovator). Based in Merrimack, NH.

Profile: ramanaambore.me

Experience & recognition

Current role	Principal System Analyst · Fidelity Investments — billing-platform modernization on AWS + Snowflake; distributed fee-calculation engines.
Industry depth	30+ years FinTech, mainframe to cloud-native — derivatives risk, options pricing (Black-Scholes, Greeks), multi-leg strategy analytics.
Recognition	NTT Innovation Award — top-40 global innovator, selected for financial-services engineering.

Credentials

FRM (GARP, 2022) · CFA Level 3 · Master's, Computer Science · Six Sigma Green Belt · IBM Certified DB2 DBA · Sun Certified Java Programmer.

Links

- **Website**: ramboq.com
- **Profile**: ramanaambore.me
- **Contact**: through the ramboq.com contact page
- **Repo**: github.com/RamanaAmbore/algo-trader

RamboQuant platform scope

Builds and maintains the RamboQuant platform end-to-end: a production application covering multi-broker order routing, real-time market data pipelines, options analytics, portfolio tracking, and operator + investor-facing tooling.

Full tech stack:

Layer	Technologies
Frontend	SvelteKit · Svelte 5 (runes) · Vite · ag-grid-community · @tanstack/svelte-query · Tailwind · Playwright (e2e)
API	Litestar 2.x · **msgspec.Struct** schemas (10× faster than pydantic) · uvicorn (single-worker) · asyncio
Persistence	PostgreSQL 17 · async SQLAlchemy 2.x · asyncpg · **polars** (routes) · pandas (broker SDK boundary only)
IPC	**mmap** at `/dev/shm/ramboq_ticks` (KiteTicker → API tick pipeline) · UDS at `/tmp/ramboq_conn.sock` (main API ↔ conn_service) · SSE (live LTP push) · WebSocket · in-process BroadcastBus
Queues	In-process asyncio + `EventQueue` (algo/agent/order/mcp_audit) + `write_queue` (disk + db bulk-batched workers) · **ARQ + Redis** background worker as a separate systemd unit (`ramboq_worker.service`)
Brokers	kiteconnect · dhanhq · growwapi · pyotp (Kite 2FA TOTP) · KiteTicker WebSocket · Fernet-encrypted credentials at rest
Intelligence	Gemini (`google-genai`) market summaries · MCP server (17 read + 2 persist + 6 write-gated tools) · fpdf2 (monthly investor statements) · babel (i18n / number formatting)
Security	JWT HS256 (24h TTL) · PBKDF2-SHA256 password hashing · `cryptography.Fernet` (broker creds encryption) · maxminddb (visitor geolocation)
Deploy	systemd (`ramboq_api` , `ramboq_conn` , `ramboq_dev_api` , `ramboq_worker` , `ramboq_hook`) · nginx reverse proxy · Cloudflare DNS · `webhook.ramboq.com` HMAC-authenticated auto-deploy hook
Observability	Perf-snapshot cron + admin dashboard · web-vitals runtime capture (Playwright) · radon (cyclomatic complexity) · pytest-json-report (test-duration tracking) · vulture (dead-code detection)

About this document

The full developer onboarding document. Read top-to-bottom to understand the codebase end-to-end; reference specific sections for ongoing work. Flow diagrams use Mermaid. Tech-stack rationale (why / what / how / where) is interleaved with each subsystem rather than collected separately — easier to learn in context.

Goal: anybody who reads and understands this document should be able to modify and enhance features by making the actual code changes. Each subsystem section names the files; **Part IX** at the end is a cookbook of common change recipes with exact-diff-level guidance.

Table of contents

Part I — Foundation

- §1. Architecture overview
- §2. Tech stack — at a glance
- §3. Core architectural principles
- §4. Concurrency model
- §4.5. Data layer — implementation detail
- §4.6. Database schema overview
- §4.7. Table relationships
- §4.8. Retention policies
- §4.9. Metrics + performance tracking

Part II — Order lifecycle

- §5. Order placement — single ticket (Ticket tab)
  - §5.1. F&O order quantity convention (lots-first API)
- §6. Order placement — basket (Chain tab)
- §7. Chase loop lifecycle
- §8. The order/chase/template tripod

Part III — Templates + exits

- §9. Template attach pipeline
- §10. 4-default template matrix
- §11. Template override merge
- §12. Chase loop invariants
- §13. Trail-stop subsystem
- §13.2. Pair system architecture

Part IV — Brokers

- §14. Broker abstraction

- §14.1. Kite account flipping — market-data broker resolution
- §14.5. Broker abstraction — implementation detail
  - §14.5.9.5. BSE ticker subscription and NSE→BSE equity token fallback
- §15. How to add a new broker
- §16. Broker gotchas

Part V — Frontend

- §17. Frontend modal state
- §18. Frontend state architecture
  - §18.3. Frontend column factory SSOT
  - §18.4. Pure module extractions — Phase 1
  - §18.5. Svelte component extractions — Phase 2 + Phase 3
- §19. The preview pipeline

Part VI — Runtime

- §20. Background task topology
  - §20.1. Sparkline refresh pipeline
- §21. Data refresh — PositionStrip + Dashboard
- §21.5. Frontend → broker API — full round-trip
- §21.5.5. Day P&L backstop — SSOT
- §21.5.6. MCX snapshot multiplier fix
- §21.5.7. Holdings snapshot close_price fix
- §21.6. Persistence three-tier — cache → DB → broker
- §21.7. Stale data semantics — keepStaleOnEmpty and error recovery
- §22. Demo mode
- §22.5. Investor portal — token-as-credential
- §22.6. Investor portal — units-based NAV math
- §22.7. Audit log — forensic trail
- §22.8. Postback fan-out — book_changed bus
- §22.9. History — multi-day orders / trades / funds
- §22.10. Order placement latency — preflight + tick cache + paper-skip
- §22.11. Navbar audit — rename + resequence
- §22.12. #audit workflow + Dhan / Groww postback scaffold
- §22.13. Audit slice D — UX consistency + palette consolidation + 2 defects
- §22.14. Market-status — broker API beats bellwether-quote probe
- §22.15. Chart indicator system — pure module + overlay persistence
- §22.16. Derivatives page — cold-start and payoff improvements
- §22.17. Derivatives page — `_throttledTick` market-close gate
- §22.18. NavStrip — lifetime slot SSOT aligned with MarketPulse TOTAL
- §22.19. CSV export button — all ag-Grid card headers
- §22.20. docs/ folder reorganisation + exhaustive spec files
- §22.21. Spec files expanded — Orders, Activity, Charts (modal + page coverage)
- §22.22. Agent execution — actions.py updates (NCO guard + async close)
- §22.23. Template attach — parent_lot_size NFO/BFO/CDS support
- §22.24. Agent engine — topic suppression doesn't burn lifespan
- §22.25. Demo banner + Showcase page + Nav + Fullscreen + Derivatives Exp Close
- §22.26. Derivatives rows + LogPanel CSS Grid — border-bottom + 2-col grid layout
- §22.27. Modal consolidation — ActivityLogModal sheet pattern + fullscreen inset flush
- §22.28. Agent engine — baseline window initialization + mixed-condition gating
- §22.29. MmapTickReader — virtual-root alias resolution for MCX SSE subscription
- §22.30. MmapTickReader self-healing supervision + tick-staleness indicator

Part VII — Operations

- §23. How to add a new template field
- §24. Testing philosophy
- §25. Logging discipline
- §26. Deployment notes
- §26.5. Recent fixes and operational improvements (Jul 2026)
- §27. Sprint history + audit fixes

Part VIII — Wrap-up

- §28. Reading order for a new developer
- §29. When in doubt
- §30. Operator's mental model

Part IX — Change recipes (cookbook)

- §31. Recipe: add a new route
- §32. Recipe: add a column to an existing table
- §33. Recipe: add a new background task
- §34. Recipe: add a new agent action
- §35. Recipe: add a new template field (worked example)
- §36. Recipe: add a new broker capability flag
- §37. Recipe: add a new page
- §38. Recipe: add a setting
- §39. Recipe: change an existing default template
- §40. Recipe: wire a new notification channel
- §41. Recipe: ship a fix to dev + main
- §42. Cross-cutting checklist before every commit

> Tech-stack rationale boxes appear inline as ⚙ **TECH: WHY · WHAT · HOW · WHERE** callouts throughout. Look for the gear glyph.

Part I — Foundation

1. Architecture overview



Layer	Tech	Key files
Frontend	SvelteKit + Svelte 5 runes + ag-Grid + hand-rolled SVG charts	`frontend/src/`
API	Litestar 2.x + msgspec.Struct schemas	`backend/api/`
DB	PostgreSQL 17 + SQLAlchemy 2.x async + asyncpg	`backend/api/database.py`, `models.py`
Brokers	Vendor SDKs behind a unified `Broker` ABC	`backend/brokers/`
Background	asyncio tasks spawned at app startup	`backend/api/background.py`

2. Tech stack — at a glance

Each choice has a **why/what/how/where** callout inline where relevant (marked by ⚙). Key stacks:

Layer	Tech	Why
API	Litestar 2.x + msgspec	~10× faster JSON encode/decode than pydantic on big payloads
DB	PostgreSQL 17 + SQLAlchemy 2.x async + asyncpg	Fast, reliable, static typing, JSONB for attached_gtts_json blob
Frontend	SvelteKit + Svelte 5 runes + ag-Grid	Smaller bundle, native reactivity, row virtualization for 1000+ ticks/sec
Charts	Hand-rolled SVG (no Chart.js)	150KB saved, tighter control, palette integration
Concurrency	asyncio + single uvicorn worker	Kite token affinity, in-process locks, background task state
Notifications	Telegram (Bot API), SMTP (Hostinger)	Free, reliable, works everywhere
WebSocket	KiteTicker (Twisted reactor → SSE bridge)	Sub-second LTP updates without burning rate limit

Full rationale for each technology appears as ⚙ **TECH** callouts throughout this doc (e.g. §3.1 broker abstraction, §4.2 KiteTicker threading, §4.3 background task lifecycle).

---

3. Core architectural principles

3.1 Single source of truth at the broker boundary

The `Broker` abstract base class (`backend/brokers/base.py`) is the **only** place vendor differences should leak. Every route, agent, and background task talks to a `Broker` instance via `get_broker(account)` from the registry.

⚙ **TECH** — `WHY` Vendor SDKs disagree on EVERYTHING (qty units, status strings, GTT shape). Letting that disagreement propagate past the adapter boundary creates bug surface area in every consumer. `WHAT` The ABC declares ~20 methods (place_order, modify_order, cancel_order, orders, holdings, positions, funds, ltp, quote, historical_data, place_gtt, modify_gtt, cancel_gtt, get_gtts, instruments, profile, holidays, order_status, trades, basket_order_margins). `HOW` New broker? Implement every method, translate to Kite shape in `_normalise_*` helpers, register in `_ADAPTERS`. Capability gaps go in `BrokerCapabilities` — never inline `if broker_id == "groww"`. `WHERE` `backend/brokers/base.py` (ABC); `backend/brokers/adapters/kite.py` / `dhan.py` / `groww.py` (implementations); `backend/brokers/capabilities.py` (matrix).

3.2 Idempotency is the default

Every path that places a broker order or GTT can fire twice — postbacks arrive twice, chase terminals race postbacks, reconcile sweeps re-fire attaches. Four patterns make this safe:

Pattern	Where	What it guards
`attached_gtts_json IS NULL` check	`_fire_template_attach_on_fill`	Double-place TP/SL/Wing at broker
`_TEMPLATE_ATTACH_LOCKS[parent_row_id]`	Same function	Concurrent races within the same row
`_KILLED_ORDER_IDS` dict with 60-min TTL	`chase.py`	Operator kills landing on stale `broker_order_id`
`MAX(prior, cumulative)` clamp	`_record_partial_fill`	Restart causing cumulative to be added again

When adding a new fill-time side-effect, ask: can my handler fire twice for the same parent? If yes, what's the idempotency check?

3.3 Database is authoritative; in-memory is fast-path

The single uvicorn worker (`--workers 1` in prod — see §4.1) means in-process locks are sufficient, but the DB is still the source of truth. After a restart, every chase loop recovers via `recover_from_db` and re-derives state. Don't store anything operationally meaningful in in-process state without a DB write to back it up.

The `attached_gtts_json` column is a deliberate small-state JSON blob rather than a foreign-key normalized table:

- ✅ Atomic write per parent — no half-attached state visible to readers
- ✅ Easy to refactor the GTT spec shape (just version the JSON inside)
- ❌ Harder to JOIN against; we accept this because GTT inspection is rare

3.4 Async by default, sync when forced

Everything API-facing is `async def` over asyncpg. Broker SDK calls are sync — Kite/Dhan/Groww use `requests` under the hood — so we wrap them in `asyncio.to_thread(...)` to keep the event loop unblocked. The threadpool sizing is the default (32 workers); we've never seen it saturate because broker API calls are sub-second.

Anti-pattern to avoid: `broker.method()` directly in an `async def` route handler. Even if it returns "fast," a single 2-second hang stalls every other request on that worker.

3.5 Demo mode = signed-out + prod branch

Demo isn't a separate code path — it's a runtime guard at the API boundary (`backend/api/auth_guard.py`) plus a frontend flag pulled from context. The same routes serve authenticated + demo traffic; the guard masks accounts and blocks writes. This means **a feature works in demo the moment it works for read-only sessions** — there's no separate "demo enablement" step to forget.

3.6 Singleton pattern — one instance, everywhere

Several long-lived, expensive-to-construct components are implemented as **process-wide singletons**. The pattern is used deliberately where all of the following are true: (a) construction is expensive (network handshake, mmap open, DB warm), (b) there is exactly one canonical instance per process, (c) many callers need the *same* instance so state stays consistent. Fits nicely with the single-uvicorn-worker guarantee from §4.1 — no cross-worker cache-coherence problem.

Canonical singletons:

Singleton	Module	Purpose
`Connections`	`backend/brokers/connections.py`	Broker session manager (Kite / Dhan / Groww). Owns credentials (decrypted from `broker_accounts` table via Fernet), 2FA, token refresh, IPv6 binding. `connections.Connections()` returns the same instance every time; `rebuild_from_db()` mutates in place.
`TickerManager`	`backend/brokers/kite_ticker.py`	KiteTicker WebSocket wrapper. **One WebSocket per process** — Kite rejects duplicates. Lives inside conn_service when `RAMBOQ_USE_CONN_SERVICE=1`; owns the mmap tick writer at `/dev/shm/ramboq_ticks`.
`MmapTickReader`	`backend/brokers/mmap_ticker.py`	Main-API tick reader. `get_mmap_reader()` returns the module-level `_singleton`. Polls the shared-memory version-word every 50ms; publishes deltas to `BroadcastBus`.
`BroadcastBus`	`backend/api/broadcast.py`	In-process pub/sub. Every SSE client + WebSocket connection subscribes to the same bus for tick + `book_changed` + `position_filled` events.
`GrammarRegistry`	`backend/api/algo/grammar_registry.py`	Agent condition-tree token catalog. Reloaded on `/api/admin/grammar/reload` (mutates the singleton in place; existing agents pick up the new tokens on their next `run_cycle`).
`MarketLifecycle`	`backend/api/algo/market_lifecycle.py`	Per-exchange open / close / close_settled event bus. Polled every 30s by `_task_market_lifecycle`. Handlers registered via `market_lifecycle.register(event, callback)` at import time.
`MetricRegistry` (perf)	`backend/api/persistence/perf_snapshots.py`	Perf-snapshot writer coordinator — buffers metrics until nightly `_task_perf_snapshot` flushes them.

Convention: singleton modules expose a lowercase accessor (`connections.Connections()`, `get_mmap_reader()`, `broadcast_bus()`) rather than a bare module-level global — the accessor lets us swap the implementation in tests (e.g., replace `_singleton` with a fake) and defers construction until first use.

Anti-patterns to avoid:

- **Don't** create local instances of these classes anywhere in route handlers or background tasks. Always go through the accessor. Multiple `Connections()` instances would each mint a fresh Kite session and the second one would evict the first from Kite's session registry.
- **Don't** cache the singleton reference across `rebuild_from_db()` boundaries — the accessor is cheap; long-lived local references miss config changes.
- **Don't** rely on singleton state during startup — `on_startup` order matters. See `backend/api/app.py::on_startup` for the canonical wire-up sequence.

Testing note: pytest fixtures reset `_singleton = None` in `conftest.py::_reset_singletons` between tests so isolation holds. Tests that need a real singleton use the `real_connections` fixture.

---

4. Concurrency model

4.1 Why one uvicorn worker?

`--workers 1` in prod is **intentional** for three reasons:

- **Kite session affinity:** multiple workers would invalidate each other's Kite tokens because Kite enforces one active session per IP.
- **In-process locking is enough:** all locks are `asyncio.Lock` instances; we never need multiprocess coordination.
- **Background tasks need shared state:** the trail-stop poller's in-memory state (`_TEMPLATE_ATTACH_LOCKS`, the ticker manager's `_tick_map`) is process-scoped. Multi-worker would require Redis or similar.

If we ever scale horizontally we'd need to externalize: tokens → DB, locks → DB advisory locks or Redis, ticker state → a separate fanout service.

4.2 KiteTicker threading

`KiteTicker` runs Twisted internally — **all WebSocket callbacks fire on a Twisted reactor thread**, not the asyncio event loop. The `TickerManager` bridges this:

- Twisted thread writes `_tick_map[token] = ltp` under a `threading.Lock`
- Async handlers read via `get_ltp(token)` — same lock, briefly held

The lock is non-reentrant and the critical section is O(1) so no deadlock risk.

⚙ **TECH** — `WHY` Twisted reactors can't see asyncio's event loop, so we can't `await` from a tick callback. Lock-protected dict is the simplest viable bridge. `WHAT` `TickerManager._tick_map: dict[int, float]` + `_tick_lock: threading.Lock`. `HOW` Tick handler does `with self._tick_lock: self._tick_map[token] = ltp`. Async reader does the same under the lock, briefly. `WHERE` `backend/brokers/kite_ticker.py`.

If you add anything that runs on the Twisted side, never call `asyncio.run_coroutine_threadsafe` without testing both directions of the round-trip. The reactor doesn't know about asyncio's event loop.

4.3 Background task lifecycle

All background tasks are spawned in `app.on_startup` via `asyncio.create_task(...)`. They run forever; cancellation only happens at app shutdown. Each task is responsible for its own error handling — an uncaught exception kills the task silently. Every task body should be:

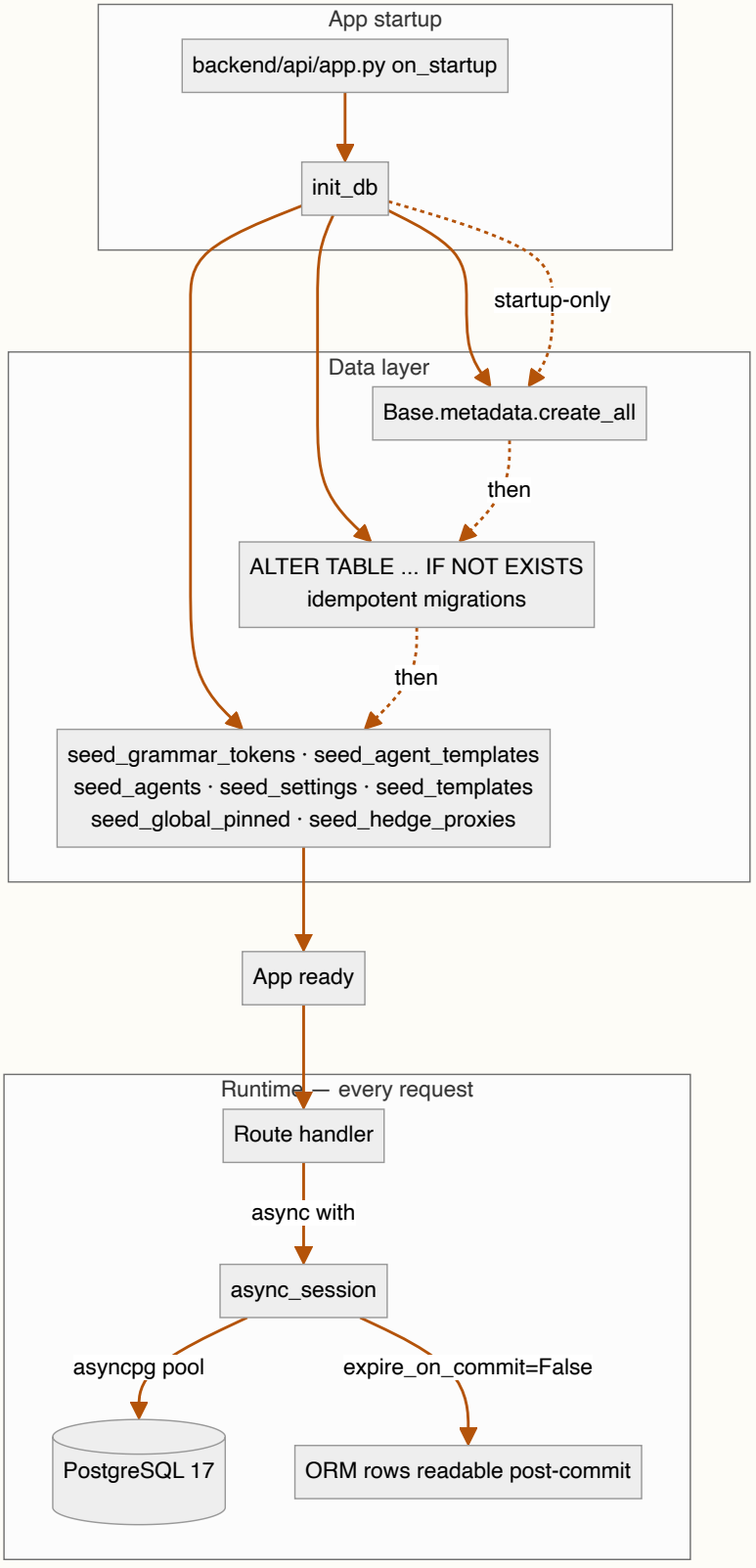
```
async def _task_X():
    while True:
        try:
            ...real work...
        except Exception as e:
            logger.exception(f"_task_X iteration failed: {e}")
        await asyncio.sleep(interval)
```

The `try/except` around the loop body is non-negotiable. We've burned hours debugging "why did the trail stop go silent" only to discover an unhandled `KeyError` ate the task three days earlier.

4.5. Data layer — implementation detail

This section is the "if you're modifying the data layer, here's the actual shape" reference. The data layer = SQLAlchemy 2.x async ORM + PostgreSQL + asyncpg driver. Read this end-to-end before changing any table.

4.5.1 Topology



4.5.2 File map

File	Purpose
`backend/api/database.py`	Engine + session factory + `init_db` (the only place we touch DDL)
`backend/api/models.py`	Every SQLAlchemy declarative model. One file by convention so the data shape is one grep away.
`backend/api/schemas.py`	msgspec.Struct wire types. Mirror of models for HTTP responses.
`backend/shared/helpers/settings.py`	`SEEDS` list + cached settings reader (`get_int / get_float / get_bool / get_string`)
`backend/api/algo/templates_seed.py`	`SYSTEM_TEMPLATES` + the seeder
`backend/config/grammars/agent_grammar.yaml`	System tokens catalog (metrics, operators, scopes, actions, notifications)
`backend/api/algo/grammar.py`	Token resolver functions + grammar registry seeder
`backend/api/cache.py`	In-memory TTL cache with per-key locking (NOT a substitute for the DB; cache invalidates on PATCH)

4.5.3 Engine + session factory

`database.py` is small and worth reading in full. Key shape:

```
# backend/api/database.py
engine = create_async_engine(
    DATABASE_URL,            # postgresql+asyncpg://...
    echo=False,
    pool_size=5,             # max connections kept warm in the pool
    max_overflow=10,         # extra one-off connections when pool exhausted
)

async_session = async_sessionmaker(
    engine,
    expire_on_commit=False,  # load-bearing — see 4.5.5
    class_=AsyncSession,
)

async def init_db() -> None:
    """Idempotent: safe to run on every startup."""
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
        # ALTER TABLE ... IF NOT EXISTS for every column added after
        # the table's initial creation. We don't use Alembic.
        await conn.execute(text(
            "ALTER TABLE algo_orders ADD COLUMN IF NOT EXISTS filled_quantity INTEGER"
        ))
        # ... many more such ALTERs ...
    # Seeders run after the DDL block — see §4.5.7
```

⚙ **TECH — Why `expire_on_commit=False`** — `WHY` After `commit()`, SQLAlchemy's default is to expire all ORM attributes on the committed rows, so the next attribute access triggers a fresh SELECT. That's catastrophic in our codebase because several handler paths commit then immediately read attributes (chase reconcile attach queue, retry_template, postback fallback match). With expire-on-commit we'd issue redundant SELECTs per commit. `WHAT` Setting this to `False` keeps the in-Python row state intact after commit. `HOW` Set globally on the `async_sessionmaker`. Never override per-session — consistency matters. `WHERE` `backend/api/database.py::async_session`.

4.5.4 Models — how to add / modify

`backend/api/models.py` is the canonical schema. Every table is a `Mapped[]`-typed class:

```
# backend/api/models.py
class AlgoOrder(Base):
    __tablename__ = "algo_orders"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    broker_order_id: Mapped[str | None] = mapped_column(String(64), nullable=True, index=True)
    account: Mapped[str] = mapped_column(String(32), index=True)
    symbol: Mapped[str] = mapped_column(String(64), index=True)
    quantity: Mapped[int] = mapped_column(Integer)
    filled_quantity: Mapped[int | None] = mapped_column(Integer, nullable=True)
    status: Mapped[str] = mapped_column(String(32), default="OPEN", index=True)
    mode: Mapped[str] = mapped_column(String(8), default="live", index=True)
    template_id: Mapped[int | None] = mapped_column(
        ForeignKey("order_templates.id", ondelete="SET NULL"), nullable=True
    )
    attached_gtts_json: Mapped[str | None] = mapped_column(Text, nullable=True)
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now(), index=True
    )
    # ... ~30 more fields, see actual file ...

    __table_args__ = (
        Index("ix_algo_orders_mode_status", "mode", "status"),  # composite, Sprint E
    )
```

Rules when adding a column:

1. Add the `Mapped[]` declaration to the model class.
2. Add an idempotent `ALTER TABLE algo_orders ADD COLUMN IF NOT EXISTS ...` to `init_db`. **Never write an Alembic migration** — our pattern is idempotent ALTERs at startup.
3. New column should be `nullable=True` with sensible default unless you're guaranteed to backfill all rows.
4. Composite indexes go in `__table_args__` and need an `ALTER` to ensure existence on rebuild. The convention: `Index("ix_<table>_<cols>", "col1", "col2")`.

Rules when adding a table:

1. Subclass `Base`, set `__tablename__`.
2. Declarative create happens automatically via `Base.metadata.create_all`.
3. Add seeded rows via a new seeder function called from `init_db`.
4. Add the corresponding msgspec.Struct in `schemas.py` if the table is operator-visible.

4.5.5 Session lifecycle in handlers

The canonical handler pattern:

```
@get("/example", guards=[auth_or_demo_guard])
async def example(self, request: Request) -> ExampleResponse:
    async with async_session() as s:
        # Reads + writes happen here
        rows = (await s.execute(
            select(AlgoOrder).where(AlgoOrder.status == "OPEN")
        )).scalars().all()
        # Mutate
        for r in rows:
            r.last_seen = datetime.now(timezone.utc)
        # Single commit at the end of a logical group
        await s.commit()
    return ExampleResponse(rows=[_to_info(r) for r in rows])
```

Anti-patterns to avoid:

- ❌ Holding a session across `await` to a broker SDK call. The broker call could take 5 seconds; the session holds a connection from the pool the whole time. Wrap the broker call in `asyncio.to_thread` OR exit the `async with` block first and re-open after.
- ❌ Committing inside a loop without batching. Each commit is a round-trip — if you have 100 rows to update, batch into one commit at the end.
- ❌ `select(AlgoOrder)` without a WHERE clause and no LIMIT. Full-table scans land in production logs eventually; always paginate operator-visible queries.
- ❌ Mutating an ORM row from one session and reading from another within the same request. Use one session per logical unit of work.

4.5.6 Idempotent migrations pattern

We don't use Alembic. Every schema change is an `ALTER TABLE ... IF NOT EXISTS` in `init_db`:

```
async def init_db():
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
        await conn.execute(text(
            "ALTER TABLE algo_orders ADD COLUMN IF NOT EXISTS filled_quantity INTEGER"
        ))
        await conn.execute(text(
            "ALTER TABLE algo_orders ADD COLUMN IF NOT EXISTS sl_trail_pct NUMERIC(8, 4)"
        ))
        # ... dozens more ...
```

This pattern was chosen because:

- ✅ Zero ops overhead — no migrations folder, no version cursor, no rollback worry.
- ✅ Deploy is just `git pull + restart`. The new column appears the moment the restart finishes.
- ✅ Branch-switching works — if dev is ahead of main, switching back to main doesn't break (the column exists; the code that uses it is gone).
- ❌ No DOWN migrations. We don't drop columns; if we want to remove a field, we stop reading from it but leave the column in place.
- ❌ Cannot rename columns easily. The workaround: add a new column, dual-write for one deploy, switch reads, then stop writing the old.

The pattern is suitable for small-team prod with infrequent destructive changes. If we ever need ten-engineer concurrent migrations, this stops scaling and we'd move to Alembic.

4.5.7 The seeders — bootstrapping defaults

Seven seeders run at startup. Six fire from inside `init_db()` (in `backend/api/database.py`); `seed_hedge_proxies` is registered as its own `on_startup` coroutine in `app.py` so it runs after the DDL block:

Seeder	Where it lives	What it seeds	Operator-overridable?
`seed_grammar_tokens`	`backend/api/algo/grammar.py`	Grammar tokens (metric/scope/op/channel/format/template/action_type)	Toggleable `is_active`; cannot delete system tokens
`seed_agent_templates`	`backend/api/algo/template_registry.py`	Reusable agent templates referenced by built-in agents	Toggleable; refresh on boot
`seed_agents`	`backend/api/algo/agent_engine.py`	10 built-in agents (6 loss-, *3 expiry-*, 1 manual)	Editable; seeder force-resets `schedule` field + force-inactives built-in summary agents on every boot
`seed_settings`	`backend/shared/helpers/settings.py`	Settings rows (`alerts.cooldown_minutes`, `chase.max_consecutive_errors`, etc.)	Yes — operator edits via `/admin/settings`; seeder preserves `value` and only refreshes metadata
`seed_templates`	`backend/api/algo/templates_seed.py`	4-default order templates (`default-bull`, `default-long-option`, `default-bear`, `default-short-vol`)	Partial — system templates are overwritten on every restart, but operator's saved copies (different `user_id`) survive
`seed_global_pinned`	`backend/api/routes/watchlist.py`	Pinned global watchlists (Markets, Default)	Editable per-user; global rows refresh
`seed_hedge_proxies`	`backend/api/routes/hedge_proxies.py` (runs from `app.on_startup`, not `init_db`)	Six default pairs (GOLDBEES → GOLD/GOLDM, etc.)	Editable via `/admin/settings → Hedge proxies`

The recurring tension in seeders: **how much to overwrite on every boot vs preserve operator changes?** The pattern:

- **Description / schema / metadata** — always refreshed (code is the source of truth)
- **`value` / `conditions` / `actions` / `events`** — preserved on existing rows (operator's overrides)
- **`status`** — preserved EXCEPT for built-in summary agents which force-reset to `inactive` (see `seed_agents`)
- **New rows added** with whatever default the SEEDS const ships

If you're adding a new seeder, follow this pattern. The audit-friendly path is `INSERT ... ON CONFLICT DO UPDATE SET <metadata only>` so existing values can't be clobbered.

4.5.8 Settings cache layer

`backend/shared/helpers/settings.py` exposes `get_int`, `get_float`, `get_bool`, `get_string` readers backed by an in-process dict cache:

```
_SETTINGS_CACHE: dict[str, Any] = {}
_CACHE_GENERATION = 0

def get_int(key: str, default: int) -> int:
    val = _SETTINGS_CACHE.get(key)
    if val is None:
        return default
    return int(val)
```

The cache loads on first read + invalidates on every PATCH (the route handler bumps `_CACHE_GENERATION` and clears `_SETTINGS_CACHE`). This means:

- ✅ Hot-path reads are O(1) dict lookup — settings are checked thousands of times per minute on background tasks.
- ✅ Operator edits take effect on the next read, no service restart.
- ❌ Multi-worker would need per-worker invalidation (we're single-worker — see §4.1 — so this isn't an issue).

When adding code that reads a setting, **always go through `get_*` helpers**. Never `SELECT ... FROM settings WHERE key = ...` in a hot path.

4.5.9 `attached_gtts_json` — the small-state JSON blob pattern

`AlgoOrder.attached_gtts_json` deserves its own subsection because it's the load-bearing column for template attach (§9):

```
# Persisted shape (string-encoded JSON)
[
    {
        "kind": "gtt",
        "label": "TP",
        "id": "abc123",                  # broker GTT id
        "current_trigger": 250.0,
        "sl_trail_pct": null,
        "tp_trigger": 250.0,
        "highest_ltp": 200.0,            # set if sl_trail_pct is non-null
        "lowest_ltp": 200.0,
        "sibling_id": null               # set for Groww emulated OCO pairs
    },
    {
        "kind": "wing",
        "label": "Wing BUY",
        "id": "def456",                  # broker order id (not GTT — wings are real positions)
        "qty": 50,
        "symbol": "NIFTY24APR21500PE"
    },
    ...
]
```

Why a blob and not a child table? Three reasons:

- Atomic write per parent — readers never see "half-attached" state.
- Easy to refactor the spec shape (just version the JSON inside, no migration).
- GTT inspection is rare — we don't JOIN against it. When we read it, we read the whole bracket anyway.

Idempotency rule: the `_fire_template_attach_on_fill` function checks `attached_gtts_json IS NULL` before writing. Once populated, it's never re-attached (see §9 for the full guard chain). To force a re-attach, the operator hits `POST /orders/algo/<id>/retry-template`.

4.5.10 Pool sizing + connection accounting

Defaults:

- `pool_size=5` connections kept warm
- `max_overflow=10` extra one-off connections under burst
- `pool_pre_ping=True` checks the connection is alive before checkout

We've never seen pool exhaustion in prod because:

- Single worker (§4.1) caps concurrent request handlers at the asyncio scheduler's natural limit.
- Background tasks use SHORT sessions (`async with` inside the loop body, not around the loop).

Symptoms of pool exhaustion (if you ever see them):

- Slow request handlers despite low DB CPU.
- `TimeoutError: QueuePool limit of size 20 overflow 10 reached` in logs.

Fix path: investigate which handler is holding sessions across `await` to a slow external call. **Don't bump `pool_size` first** — it's almost always a code-shape problem, not a sizing one.

4.5.11 Demo masking — at the data layer boundary

Read paths for demo sessions mask account values via `mask_column` (§22). The masking happens **at the route layer**, not at the data layer:

```
# Wrong — masking inside the ORM
class AlgoOrder(Base):
    account: Mapped[str] = mapped_column(String(16))
    @property
    def account_display(self):
        return mask_column(self.account)

# Right — masking at the route handler
@get("/orders")
async def list_orders(self, request: Request) -> list[OrderInfo]:
    rows = await self._fetch()
    is_demo = request.state.is_demo
    return [
        OrderInfo(account=mask_column(r.account) if is_demo else r.account, ...)
        for r in rows
    ]
```

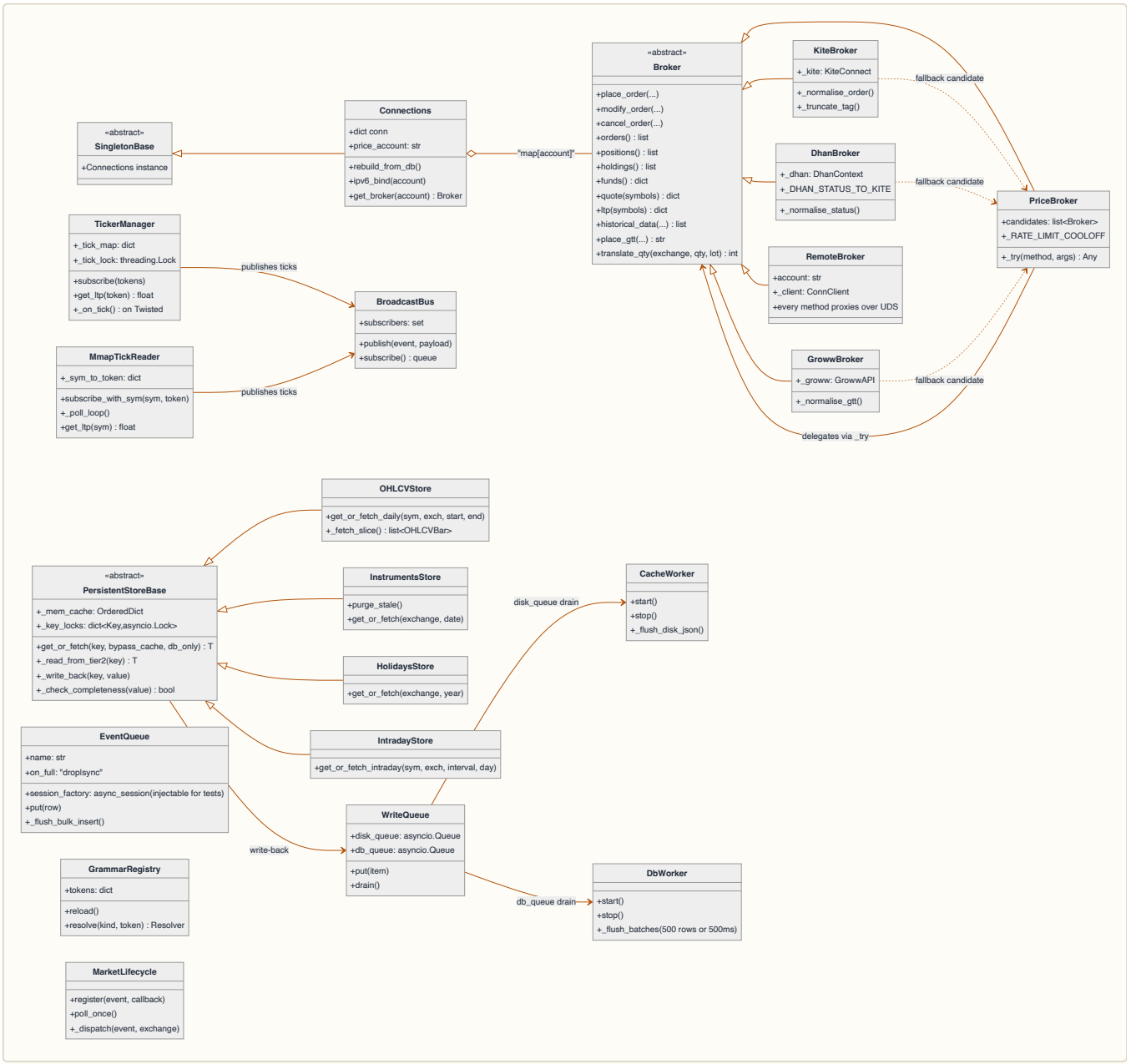
This keeps the data layer free of presentation concerns and avoids subtle bugs (e.g. ORM expressions seeing masked values during a JOIN).

4.5.12 Backend runtime object graph

The class diagram below is the high-level shape of the backend runtime. Every box is a real class you can grep. Method sets are the operationally-important surface (not every method on every class — those live in the source).

Two clusters:

- **Broker layer** (`backend/brokers/`) — Connections singleton owns all broker sessions; adapters implement the Broker ABC; local vs remote proxy split lets the same route code run against in-process brokers OR a UDS conn_service (see §14.5).
- **Persistence layer** (`backend/api/persistence/`) — `PersistentStoreBase` is the three-tier read/write scaffold; four concrete stores share it. Two write-back workers (`cache_worker` + `db_worker`) drain the in-process write queues.



Grep map — every class name above resolves to a single file:

Class	Module
`Connections` (+ `_IPv6SourceAdapter`)	`backend/brokers/connections.py`
`Broker` ABC	`backend/brokers/base.py`
`KiteBroker` / `DhanBroker` / `GrowwBroker`	`backend/brokers/adapters/{kite,dhan,groww}.py`
`PriceBroker` + `get_broker` + `get_market_data_broker`	`backend/brokers/registry.py`
`RemoteBroker` + `ConnClient`	`backend/brokers/client/remote_broker.py` + `client.py`
`TickerManager` + `BroadcastBus`	`backend/brokers/kite_ticker.py`
`MmapTickReader`	`backend/brokers/mmap_ticker.py`
`PersistentStoreBase`	`backend/api/persistence/store_base.py`
`OHLCVStore` / `InstrumentsStore` / `HolidaysStore` / `IntradayStore`	`backend/api/persistence/{ohlcv,instruments,holidays,intraday}_store.py`
`WriteQueue` / `CacheWorker` / `DbWorker`	`backend/api/persistence/{write_queue,cache_worker,db_worker}.py`
`EventQueue`	`backend/api/persistence/event_queue.py`
`GrammarRegistry`	`backend/api/algo/grammar_registry.py`
`MarketLifecycle`	`backend/api/algo/market_lifecycle.py`

4.6 Database schema overview

RamboQuant's data model spans 35+ SQLAlchemy tables, split into logical domains. Two PostgreSQL databases: `ramboq` (prod, main branch) and `ramboq_dev` (dev branches). All tables live in the branch-local DB except `broker_accounts`, which is shared.

Table categories

Auth + User Management — User identity, email verification, roles, compliance:

Table	Purpose	Key columns
`users`	Operator profiles + LP investor records. Unified table for internal staff + external LPs.	id (PK), account_id (unique), username, role (designated/trader/risk/admin/partner), email, pan, kyc_verified, contribution_date, share_pct, assigned_accounts, assigned_strategies, token_version (for force-logout)
`auth_tokens`	One-time email verification + password-reset tokens. Single table with `purpose` discriminator.	id (PK), user_id (FK), purpose (verify|reset), token (unique), expires_at, used_at
`impersonation_events`	Audit of admin/designated impersonating a partner. Tracks session start + end.	id (PK), actor_username, target_username, started_at, ended_at, end_reason
`investor_tokens`	Long-lived URL tokens for LP-facing portal access. Token IS the credential.	id (PK), user_id (FK), token (unique), expires_at, revoked_at, last_visit_at, visit_count
`investor_events`	LP capital ledger — subscriptions, redemptions, bootstrap events. Source of truth for units-based NAV math.	id (PK), user_id (FK), event_type (subscription|redemption|bootstrap), event_date, amount, nav_per_unit, units_delta
`monthly_statements`	Audit of auto-emailed LP statements. One row per (user, period_year, period_month).	id (PK), user_id (FK), period_year, period_month, generated_at, sent_at, recipients_json, pdf_size_bytes, error

Broker Connection — Account credentials + market metadata:

Table	Purpose	Key columns
`broker_accounts`	Shared table (ramboq DB only) storing encrypted broker credentials for all branches.	id (PK), account (unique, e.g. ZG0790), broker_id (kite|dhan|groww), user_id (FK), api_key_enc, access_token_enc, source_ip, priority, poll_priority, circuit_breaker_enabled, display_order
`broker_connection_events`	Shared table — audit log of connection lifecycle (auth_fail, fetch_fail, token_ok, circuit_open/close, ticker_error, etc.). Enables operator forensics on credential/network issues.	id (PK), account, event_type (VARCHAR 32), event_ts (TIMESTAMP TZ, indexed), detail (JSONB)
`market_holidays`	Exchange holiday calendar (NSE/MCX/CDS). Seeded from broker API, cached.	id (PK), exchange, holiday_date (unique per exchange)
`market_special_sessions`	Special trading sessions (e.g. Muhurat trading). Operator-editable overrides. Zero-width rows (start = end) mean closed all day.	id (PK), exchange, date, start_time, end_time, reason, source (NULL/ `manual` = operator, `nse_auto` = MCX evening-closed holiday from the NSE feed; added 2026-10-20)
`exchange_schedule`	Configurable segment opening hours and snapshot timing (equity + commodity exchanges). DB-backed single source of truth for `closed_hours_or_broker` gate logic via `exchange_clock.is_exchange_open(exchange)`. Default rows seeded at startup; operator can override via `/api/admin/exchange-schedule`.	id (PK), gate (NON-MCX|MCX), date (NULL=default, non-NULL=override), session_name, is_open (boolean), exchanges (TEXT[] array), open_time, close_time, snapshot_time, snapshot_reset_time, weekdays (int[] for Mon=0…Sun=6), reason; UNIQUE (gate, date, session_name)

Watchlists — User-defined symbol groups:

Table	Purpose	Key columns
`watchlists`	Named list containers. Global rows (is_global=True, user_id=NULL) shared across all users.	id (PK), user_id (FK nullable), name, is_global, is_default, is_pinned, sort_order
`watchlist_items`	Individual symbols in a watchlist. Includes operator-supplied alias.	id (PK), watchlist_id (FK), tradingsymbol, exchange, alias (optional label), sort_order

Orders + Execution — Core order lifecycle and attribution:

Table	Purpose	Key columns
`algo_orders`	Master order row. Spans manual + agent-fired + template-attached + chase iterations. Mode: sim/paper/live/replay/shadow.	id (PK), account, symbol, exchange, transaction_type (BUY/SELL), quantity, filled_quantity (cumulative across partials), initial_price, current_limit (re-quoted during chase), fill_price, status (OPEN/FILLED/REJECTED/CANCELLED), mode, engine (manual/sim/paper/live/replay/shadow/expiry), agent_id (FK nullable), broker_order_id, template_id (FK), attached_gtts_json (JSON list of {kind, label, id, …}), basket_tag, parent_order_id (for TP/SL children), strategy_id (FK), request_id (for audit drill-through)
`algo_order_events`	Append-only timeline per order. One row per state transition (placed, chase_modify, fill, unfill, reject, cancel).	id (PK), order_id (FK), ts, kind (placed|chase_modify|fill|unfill|reject|cancel|postback|…), message, payload_json
`algo_events`	Legacy event log (pre-AgentEvent era). Deprecated; kept for compatibility.	id (PK), algo_order_id (FK nullable), event_type, detail, timestamp
`strategies`	Named bucket for order attribution. Owns lots + provides per-strategy P&L rollup.	id (PK), slug (unique), name, description, owner_user_id (FK nullable), capacity_cap_inr, target_volatility, is_active
`strategy_lots`	Per-strategy FIFO lot ledger. Opens on fill, closes when counter-direction consumes it. Authoritative for per-strategy P&L.	id (PK), strategy_id (FK), open_order_id (FK nullable), account, symbol, exchange, side (B/S), qty, remaining_qty, open_price, close_price, realized_pnl, opened_at, closed_at

Agents + Conditions — Rule engine and alert infrastructure:

Table	Purpose	Key columns
`agents`	Declarative rules: condition tree → alert → actions. Includes loss alerts, expiry alerts, user-defined custom agents.	id (PK), slug (unique), name, long_name (3-part descriptor), description, conditions (JSONB tree), events (alert channels), actions (order/cancel/close side-effects), scope (per_account|all_accounts), schedule (market_hours|continuous|...), cooldown_minutes, fire_at_time (gate to HH:MM window), trade_mode (paper|live per-agent override), status (active|inactive|cooldown|completed), lifespan_type (persistent|one_shot|n_fires|until_date), tier (critical|high|medium|low for noise reduction), topic (agent grouping tag), digest_window_sec (buffer outgoing alerts), debounce_minutes, condition_first_true_at, tags (JSONB list), blackout_windows (quiet hours)
`agent_events`	Alert events fired by agents. Persisted for operator inspection + MCP queries.	id (PK), agent_id (FK), event_type (fired|suppressed|error|...), trigger_condition (the condition leaf that fired), detail, sim_mode (simulator flag), timestamp
`grammar_tokens`	Token catalog (metrics, scopes, operators, channels, actions). Extensible alphabet for condition/alert/action trees.	id (PK), grammar_kind (condition|notify|action), token_kind, token (name), value_type, resolver (dotted path to function), params_schema, enum_values (JSONB), is_active, is_system

NAV + Performance — Investor slicing and daily metrics:

Table	Purpose	Key columns
`nav_daily`	Daily firm-level NAV snapshot. Written after broker positions settle. Authoritative for all investor slicing.	id (PK), as_of_date (unique), nav, cash_total, positions_mtm, holdings_mtm, accounts_snapshot (JSONB list), note
`strategy_snapshots`	Daily per-strategy P&L rollup. Charts the strategy performance curve.	id (PK), strategy_id (FK), as_of_date, open_lots_count, open_notional, realised_pnl, unrealised_pnl, margin_allocated
`perf_snapshots`	Nightly static + runtime metrics per page. Used for trend analysis + performance budgets.	id (PK), page_or_route, metrics_json (JSONB), static_metrics_json, captured_at

Data Snapshots — Intraday market state + persistent cache:

Table	Purpose	Key columns
`daily_book`	Intraday snapshot of positions, holdings, or funds per account per symbol. Captured at market close, useful when markets closed. Schema includes `previous_close` (frozen yesterday's settlement price) to stabilize day P&L math during closed-hours reads, and `previous_close_backup` (immutable backup of pre-fix value).	id (PK), kind (positions|holdings|funds), account, symbol, exchange, qty, avg_price, ltp, pnl, pnl_pct, date, captured_at, segment (NSE|MCX|...), **previous_close**, **previous_close_backup**
`ohlcv_daily`	5-year OHLCV history. Persistence tier 2 fallback for chart data.	symbol, exchange (PK), date (PK), open, high, low, close, volume
`instruments_snapshot`	Per-exchange symbol→token map. Refreshed daily.	id (PK), exchange, date, payload (JSONB full instruments dict), row_count
`holidays_snapshot`	Exchange holiday sets per year. Immutable once year closes.	id (PK), exchange, year, dates_json (JSONB list)
`intraday_bars`	5/15/30/60-minute bars. 90-day rolling retention.	id (PK), symbol, exchange, date, interval (5min|15min|30min|60min), bar_ts, open, high, low, close, volume
`movers_snapshots`	Nightly snapshot of top movers (NIFTY, NIFTYNXT50, etc). Pre-computed so /pulse doesn't timeout.	id (PK), index_symbol, snapshop_date, movers_json (JSONB list)

Audit + Compliance — Forensic trails:

Table	Purpose	Key columns
`audit_log`	HTTP request + mutation audit trail. Every write captured by middleware + explicit handlers.	id (PK), actor_user_id, username, role, action, category (order.place|order.fill|agent.action|system.nav|...), method, path, target_type, target_id, status_code, summary, request_id (FK to al go_orders for drill-through), client_ip, created_at
`mcp_audit`	Mutations initiated via MCP (Claude Code research mode). Tracks tool calls + results.	id (PK), user_id (FK), tool_name, args_redacted, result_status, result_summary, request_id, created_at
`admin_email_events`	Audit of admin-triggered alert sends (e.g. manual test notifications).	id (PK), admin_user_id, recipient_email, subject, body_preview, sent_at, error
`visitor_log`	Minimal analytics — timestamps of /auth/login + visitor count.	id (PK), visitor_ip, last_seen_at, visitor_count

Market Lifecycle + Background Jobs — System state:

Table	Purpose	Key columns
`market_lifecycle_events`	Audit log of market open/close transitions per exchange. Indexed for operator drill-down.	id (PK), exchange, event_type (nse:open|nse:close|nse:close_settled|...), fired_at, captured_at
`code_metrics_snapshots`	Captured per release (commit SHA). Query count, test counts, response times.	id (PK), release_version, static_metrics_json, runtime_metrics_json, captured_at

Configuration + Extensibility:

Table	Purpose	Key columns
`settings`	DB-backed tunables: alert cooldown, retry counts, market hours, etc. Operator-editable via `/admin/settings`.	id (PK), category, key (unique), value_type (int|float|bool|string), value (operator-editable), default_value, description, schema (JSON validation), units
`order_templates`	Reusable bracket recipes. System templates (default-bull, default-bear, etc) seeded at boot; operator saves custom copies.	id (PK), user_id (FK nullable), name, slug, owner_user_id (FK nullable), tp_pct, sl_pct, wing_premium_pct, wing_strike_offset, wing_qty, tp_scales_json, tp_order_type, sl_trail_pct, is_system
`hedge_proxies`	Cross-reference between holdings (GOLDBEES) and option roots they hedge (GOLD). Includes β regression.	id (PK), proxy_symbol, target_root, is_active, note, beta, correlation, regression_at
`research_threads`	Persistent Chat threads in `/admin/research` (MCP Lab). One row per thread; messages stored as JSON.	id (PK), created_by_user_id (FK), title, slug, messages_json (JSONB), summary, created_at, updated_at
`sim_recordings`	Deterministic event logs for replay. Captures every state mutation so operator can re-run identical scenarios.	id (PK), label, scenario, seed_mode, started_at, ended_at, duration_sec, tick_count, event_count, payload (JSONB event stream), owner_user_id (FK)
`sim_iterations`	Multi-run coordinator. First iteration references itself; others reference iteration 1 via `parent_run_id`.	id (PK), slug (unique), parent_run_id (FK self-ref nullable), iteration_index, iterations_total, regime, seed, started_at, ended_at, end_reason, params_json, summary_json

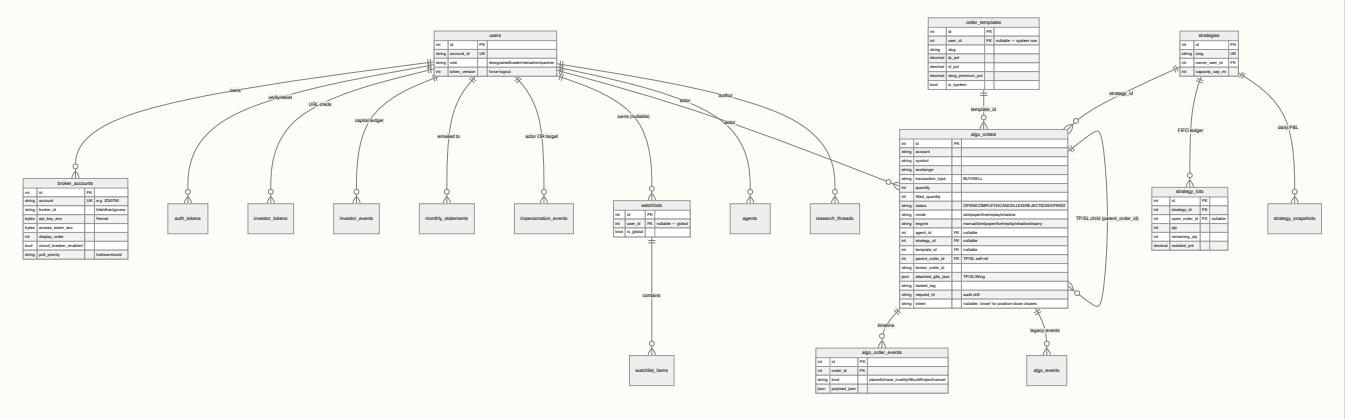
News + Market Reports:

Table	Purpose	Key columns
`market_report`	Single-row cache (id=1) for daily market summary from Gemini API.	id (PK, always 1), content, cycle_date, refreshed_at, generated_at
`news_headlines`	RSS headlines cache. Pre-filtered on /market page.	id (PK), link (unique), title, summary, published_at, source, category

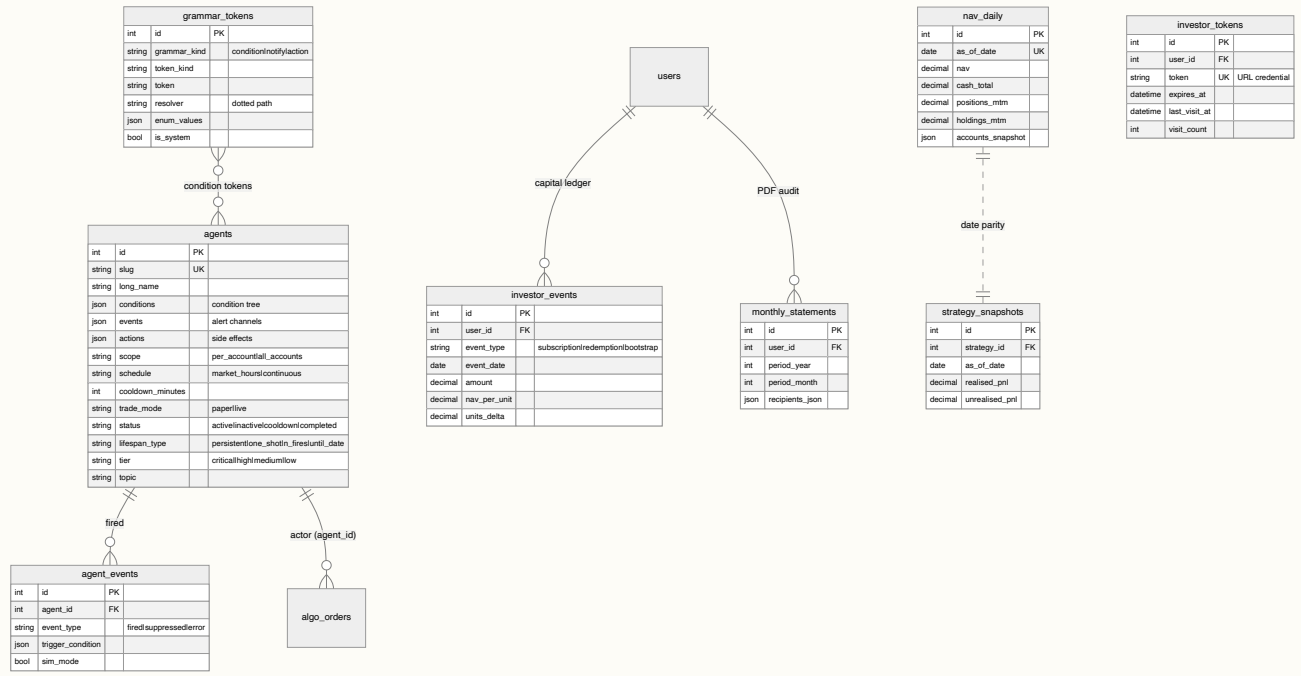
4.7 Table relationships

The full schema spans 35+ tables. To keep the ERD readable we render it in four domain-scoped panels — auth/user + orders/execution, agents + NAV + investor slice, persistence + market data, and audit + configuration. Every table in §4.6 appears in exactly one panel; foreign keys that cross panels are annotated in prose after each diagram.

4.7.1 Panel A — Auth · users · orders · execution



4.7.2 Panel B — Agents · alerts · NAV · investor slice

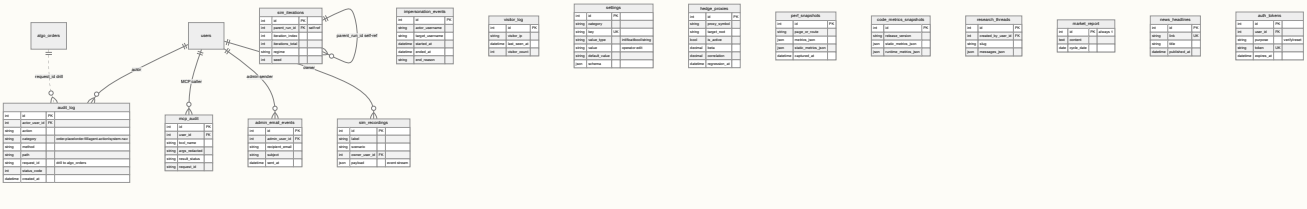


4.7.3 Panel C — Persistence · market data cache



Panel C has NO foreign keys (persistence stores are self-contained market-data cache). `daily_book` is the closed-hours snapshot anchor for every operator-visible surface (positions, holdings, NAV, sparklines); the latest-batch CTE in `snapshot_gate.latest_snapshot_ltp_map` reads from here. `market_lifecycle_events` audits every open / close / close_settled transition dispatched by the singleton `MarketLifecycle` bus.

4.7.4 Panel D — Audit · configuration · extensibility



Cross-panel foreign keys:

- `algo_orders.agent_id` (Panel A) → `agents.id` (Panel B) — `ON DELETE SET NULL`
- `algo_orders.request_id` (Panel A) drill-through from `audit_log.request_id` (Panel D)
- `agent_events.agent_id` (Panel B) → `agents.id` (Panel B) — `ON DELETE CASCADE`

Key foreign-key patterns:

- `algo_orders.agent_id` → agents.id: `ON DELETE SET NULL` (preserve order history if agent deleted)
- `agent_events.agent_id` → agents.id: `ON DELETE CASCADE` (retire agent history when agent deleted)
- `algo_orders.parent_order_id` → algo_orders.id: `ON DELETE SET NULL` (self-ref for TP/SL children)
- `algo_order_events.order_id` → algo_orders.id: `ON DELETE CASCADE` (timeline only meaningful with parent)
- `investor_events.user_id` → users.id: `ON DELETE RESTRICT` (capital ledger is audit trail; explicit operator action required to delete LP)
- User.id: multiple targets use `ON DELETE CASCADE` (auth_tokens, investor_tokens, monthly_statements) or `SET NULL` (impersonation, strategy owner)

4.8 Retention policies

Data retention is staggered — critical audit trails kept longer than ephemeral cache. Configured via `settings` table; operator can adjust retention via `/admin/settings` → Retention.

Table	Retention	Config key	Rationale
`ohlcv_daily`	5 years	(hardcoded)	SEBI Cat-III backtest horizon
`instruments_snapshot`	7 days	(hardcoded)	Symbol token changes rarely; old snapshots are cheap to drop
`holidays_snapshot`	Forever	(hardcoded)	Reference data — rarely used but occasionally queried for audits
`intraday_bars`	90 days	(hardcoded)	Intraday charts old after 3 months; 90-day window covers quarterly strategy review
`algo_events`	30 days	`retention.algo_events_days`	Operational log; older entries rarely queried
`algo_order_events`	90 days	`retention.algo_order_events_days`	Order timeline — longer window for compliance disputes
`auth_tokens`	7 days after expiry	`retention.auth_tokens_days`	One-time verify/reset tokens; ephemeral by design
`mcp_audit`	90 days	`mcp.audit_retention_days`	MCP tool calls (research mode) — compliance audit trail
`audit_log`	365 days	`retention.audit_log_days`	User action audit trail — 1-year window for disputes
`nav_daily`	Forever	(hardcoded)	Investor reporting — Cat-III requires 8-year retention
`daily_book`	Forever	(hardcoded)	Intraday snapshots become the P&L source after market close
`investor_events`	Forever	(hardcoded)	Capital ledger — units-based NAV depends on full history
`monthly_statements`	Forever	(hardcoded)	Investor statements are permanent records
`strategy_snapshots`	Forever	(hardcoded)	Strategy performance is historical record
All others	Forever	—	Configuration, metadata, non-critical operational logs kept for full history

Cleanup mechanism: nightly cron (`03:10 IST` and staggered) runs `DELETE FROM <table> WHERE created_at < now() - interval`. Per-table cleanup is idempotent — multiple runs don't corrupt state.

Operator override: Set retention to `0` in `/admin/settings` to disable deletion for that table (useful during active investigation).

4.9 Metrics + Performance tracking

RamboQuant tracks two orthogonal signals: **static code health** (per-file metrics) and **runtime Web Vitals** (per-page response times, JavaScript heap pressure). Both are persisted nightly to `perf_snapshots` and visualized in `/admin/perf`.

⚙ **TECH — Why centralized metrics** — `WHY` Operator feedback (Jun 2026): "dropdown lag", "refresh button stuck", "frontend heap keeps growing". A single dashboard surfacing cyclomatic complexity + LCP + JS heap reveals systemic issues (subscription leaks, long tasks, state bloat) before they degrade UX. `WHAT` Nightly snapshot of static + runtime metrics, persisted with ISO8601 timestamp + page_or_route tag. `HOW` `scripts/perf_baseline.py` computes static via `radon cc` + line-count; `scripts/perf_capture_run.sh` runs Playwright against dev to harvest LCP/TBT/heap via `performance.timing` + Chrome DevTools Protocol. Cron orchestrator (`_task_perf_snapshot` at 04:00 IST) executes both; inserts row via `POST /api/admin/perf/snapshot` (internal, no auth required from cron). `WHERE` `backend/api/models.py::PerfSnapshot`, `backend/api/routes/admin.py`, `scripts/perf_*.py`.

Static metrics (per file)

Metric	Ideal	How	Notes
LOC	Components < 1500, Pages < 3000	`wc -l` (excludes blank + comment-only)	Baseline for cognitive load
cc_max	< 20	Cyclomatic complexity of most complex function in file via `radon cc`	Red flag at > 50: hard to maintain, high defect risk
cc_avg	< 8	Mean complexity across all functions	Safe zone; > 15 indicates refactor candidate
$effect count	< 20 per Svelte file	Regex boundary match `$effect(` declarations	Too many effects = hard to trace reactivity
$state count	< 50 per Svelte file	Regex boundary match `$state(` declarations	Large state trees risk stale-cache bugs
$derived count	Monitor trend	Regex boundary match `$derived` declarations	No hard limit; watch for explosion as sign of over-reactivity

Cyclomatic complexity thresholds (colour coding):

- Green: `cc < 10` — safe, easy to read
- Yellow: `cc 10–20` — moderately complex, watch carefully
- Red: `cc > 20` — hard to maintain, high defect risk
- Critical: `cc > 50` — refactor mandatory before merge

Runtime metrics (Web Vitals)

Metric	Ideal	How measured	Notes
LCP	< 2500 ms	Largest Contentful Paint — largest above-fold element render time	Chrome DevTools via Playwright
TBT	< 200 ms	Total Blocking Time — sum of long-task durations during page load	Chrome DevTools via Playwright
JS Heap	< 50 MB	V8 heap size after page settles (Chrome only)	Leak detector; baseline should not grow session-over-session
Route p50	< 200 ms	Median backend request latency (per endpoint)	Sampled from access logs
Route p95	< 500 ms	95th-percentile tail latency	Catches outlier slow requests
QPS	< 20 typical	Requests per second per route	Expected load during market hours

Data flow

Static compute — `scripts/perf_baseline.py --with-cyclomatic --no-build`:

- Walks `backend/` + `frontend/src` recursively
- For each `.py` file: LOC count + `radon cc` function list
- For each `.svelte` file: LOC count + regex boundary match for `$effect(`, `$state(`, `$derived` declarations (regex patterns in the script handle `$derived.by(...)` and `$props({...})`)
- Outputs JSON with per-file metrics

Runtime capture — `scripts/perf_capture_run.sh`:

- Spins up Playwright against `dev.ramboq.com`
- For each route in a curated list (positions, holdings, orders, dashboard, etc):
  - Navigate → wait for settle (3s idle) → capture `window.performance.timing.navigationStart`, `performance.memory.usedJSHeapSize` (Chrome only)
  - Extract LCP via `PerformanceObserver` + `'largest-contentful-paint'` entrypoint
  - Sum `performance.getEntriesByType('longtask')` duration for TBT
- Outputs JSON per route

Nightly persist — `_task_perf_snapshot` (04:00 IST):

1. Calls `perf_baseline.py --with-cyclomatic --no-build`
2. Calls `perf_capture_run.sh`
3. Merges static + runtime JSONs
4. `POST /api/admin/perf/snapshot` with `{page_or_route, static_metrics, runtime_metrics}`
5. Inserts `PerfSnapshot(captured_at=now(), page_or_route=..., metrics_json=..., static_metrics_json=...)` row

DB schema

```
class PerfSnapshot(Base):
    __tablename__ = "perf_snapshots"

    id                : int (PK)
    page_or_route     : str (e.g. "GET /api/positions", "/orders", "/dashboard")
    metrics_json      : dict (JSONB — runtime metrics: lcp_ms, tbt_ms, heap_mb, qps, ...)
    static_metrics_json : dict (JSONB — static metrics: loc, cc_max, cc_avg, ...)
    captured_at       : datetime (UTC, unique per (page_or_route, day))

    __table_args__ = (
        Index("ix_perf_snapshots_route_ts", page_or_route, captured_at.desc()),
    )
```

Retention: 365 days (config: `retention.perf_snapshots_days`, default 365).

Admin endpoints

- `GET /api/admin/perf/latest` — latest snapshot per page/route (no history).
- `GET /api/admin/perf/history?page=<route>&days=30` — time series for a single page_or_route over N days. Returns `[{captured_at, metrics_json, static_metrics_json}]`.
- `GET /api/admin/perf/regressions?days=7&threshold_pct=10` — detect pages where any metric exceeded 7-day median by > 10%. Returns list with regression alert.
- `POST /api/admin/perf/snapshot` — internal cron endpoint. No auth; HMAC signature (`X-PERF-SIGNATURE` header, signed with `api_secret`) required.

All endpoints are `admin`-guarded except `/snapshot`.

Frontend surface

`/admin/perf` dashboard — card grid layout:

- **Code Health Card** — time-series chart of cc_max + LOC trend (30-day window)
- **Runtime Card** — time-series chart of LCP + TBT + heap (30-day window)
- **Regression Alert** — sticky callout at top if regressions detected in last 7 days
- **Metrics Glossary** — info tooltips (hover on metric label → show WHAT/IDEAL/IMPACT/FIX)

Metric metadata SSOT — `frontend/src/lib/data/metricMetadata.js` exports `METRIC_META`:

```
export const METRIC_META = {
  cc_max: {
    WHAT: "Highest cyclomatic complexity of any function in the file",
    IDEAL: "< 20 for maintainability",
    IMPACT: "High cc means more defects, longer review cycles",
    FIX: "Extract complex functions into helpers, simplify boolean logic"
  },
  lcp_ms: {
    WHAT: "Largest Contentful Paint — largest above-fold element render time",
    IDEAL: "< 2500 ms",
    IMPACT: "Poor LCP = operator sees blank page for seconds",
    FIX: "Lazy-load off-viewport components, defer non-critical JS"
  },
  // ... per metric
};
```

InfoHint component (reused everywhere) displays this tooltip on hover.

`#major` tag workflow

When operator adds `#major` prefix to any ask, the concurrent refactor flow is triggered:

1. Audit agent fetches top-3 cyclomatic/LOC hotspots from last 30 days of `perf_snapshots`
2. Identify regressions flagged in last 7 days
3. Parallel refactor tasks per hotspot
4. Recompute metrics after changes + assert no regression vs baseline

Example:

```
#major add a new route

→ audit sees cc_max creeping from 18→22 in chase.py over 2w
→ refactor: extract _chase_place_order, _chase_poll, _chase_status into focused
  functions (break one 60-line function into 3 functions × ~20 lines each)
→ recompute: cc_max now 14, tests pass
→ merge
```

Key files

- `backend/api/models.py::PerfSnapshot` — schema
- `backend/api/routes/admin.py::perf_snapshot` — cron endpoint
- `scripts/perf_baseline.py` — static metrics compute
- `scripts/perf_capture_run.sh` — runtime capture via Playwright
- `backend/api/background.py::_task_perf_snapshot` — orchestrator (04:00 IST)
- `frontend/src/lib/data/metricMetadata.js` — METRIC_META glossary
- `frontend/src/routes/(algo)/admin/perf/+page.svelte` — dashboard

4.10 Market data architecture — refresh cadences

The platform drives all live market data through three independent, asynchronous refresh cadences that serve different purposes:

Tier 1: Tick cadence (~1 second, WebSocket)

What it is: Real-time LTP updates from KiteTicker WebSocket → `/dev/shm/ramboq_ticks` mmap → SSE push to frontend.

What it updates:

- PositionStrip `liveSpot` overlay during market hours
- MarketPulse per-row LTP sparklines + `_positionsDelta` SSE flash
- Payoff diagram `liveSpot` during strategy analysis

Gate: Freezes when `isMarketOpen() = false` (no ticks after session close).

Performance: One broker WebSocket subscription (per primary account), distributed mmap-to-RAM bandwidth < 1 MB/min.

Files:

- `backend/brokers/kite_ticker.py` — WebSocket subscribe + write mmap
- `frontend/src/lib/PositionStrip.svelte` — SSE reader + `_positionsDelta` computed

Tier 2: Book cadence (5 second frontend / 30 second backend cache)

What it is: Polling refresh of positions, holdings, margins, and funds snapshots.

Frontend: `marketDataStores.svelte.js` `_tickBookPollers` fires every 5 seconds.

Backend: `positions`, `holdings`, `funds` routes maintain `_RAW_CACHE` (30s TTL). One broker round-trip per TTL window shared by all routes + background tasks.

Net effect:

- Broker is queried **at most once per 30 seconds**
- Frontend re-renders **every 5 seconds** with the freshest cached data
- Order fill → cache invalidate → page re-render **within ~200ms**

Gate: No market-hours gate. Polls continuously (off-market shows last snapshot).

Invalidation: `book_changed` event from terminal postback fires `book_changed` bus and triggers immediate reload on listening pages (Pulse, Dashboard, Derivatives, Orders, Performance).

Performance: ~1–2 broker RPC calls per 30 seconds per account; frontend renders every 5 seconds with no broker touch on cache hits.

Files:

- `frontend/src/lib/data/marketDataStores.svelte.js` — `_tickBookPollers` interval
- `backend/brokers/broker_apis.py` — `_RAW_CACHE` (30s TTL), `@for_all_accounts`
- `backend/api/routes/orders.py` — `book_changed` broadcast + cache invalidation

Tier 3: Performance cadence (5 minute background tasks)

What it is: Async background tasks that compute NAV, portfolio Greeks, equity curves, and persist to deques + database.

When it runs: `_task_performance` background loop (5 min intervals during market hours).

What it computes:

- Firm NAV + per-account NAV breakdown (read from `positionsDayPnlStore`)
- Agent P&L attribution per strategy
- Equity curve snapshots → `_intraday_equity` deque

Note: Modern UI reads `positionsDayPnlStore.total` (book-cadence store) instead of this task's output. The task is maintained for diagnostic/audit purposes; real-time values come from Tier 2.

Files:

- `backend/api/background.py::_task_performance` — 5-min loop
- `frontend/src/lib/data/positionsDayPnlStore.svelte.js` — canonical frontend source

Coordination: the `book_changed` event

Terminal order postbacks trigger a coordinated refresh across all three tiers:

```
# backend/api/routes/orders.py::order_postback
invalidate("orders")                      # Tier 1 + 2
if _terminal_status:
    invalidate("positions")               # Tier 2
    invalidate("holdings")                # Tier 2
    broadcast({"event": "book_changed"})  # Tier 3 listener
```

Frontend subscription (every algo page via `(algo)/+layout.svelte::onMount`):

```
import { bookChanged } from '$lib/data/bookChanged';

let _bookCounter = 0;
$effect(() => {
  const n = $bookChanged;
  if (n <= _bookCounter) return;
  _bookCounter = n;
  loadPositions();  // Reload page's primary data
});
```

A 200ms debounce coalesces basket-order bursts (4 fills produce one reload, not four).

Net effect: Order fill → Kite postback → cache invalidate → book_changed broadcast → frontend reloads → all three tiers see consistent data within ~1 second.

4.11 Market close / off-market behaviour

After NSE settlement (~~15:30 IST) or MCX close (~~23:30 IST), the platform transitions from live streaming to snapshot mode. Every surface must behave consistently:

The frozen state

Component	Value	Update rule
LTP	Settlement price from broker	Frozen until next session open
close_price	Prior session's settlement LTP	Frozen until next session open
Positions grid	Last 30-second cached snapshot	No broker calls during closed hours
Holdings grid	Last 30-second cached snapshot	No broker calls during closed hours
Option chain quotes	Last cached depth (stale)	Broker returns no real data
Payoff diagram	Last computed values with settlement spot	No live Greeks re-computation
Day P&L	Computed as `(settlement_ltp - prev_close) × qty`	Frozen (no intraday moves)

The invariant: close_price

CRITICAL: The `close_price` field must **NEVER** be modified after session close.

Lifecycle:

```
Session 1 open:   close_price = prev_session's settlement LTP (ONLY moment it changes)
Session 1 running: close_price = constant (never updated)
Session 1 close:  close_price = constant (never updated)
Session 2 open:   close_price = session 1's settlement LTP (ONLY moment it changes again)
```

Guardian logic:

- `backend/api/routes/positions.py::_override_stale_close_from_snapshot()` — replaces any drifted `close_price` from broker with `daily_book.previous_close` (frozen at first intraday snapshot)
- Equivalent for holdings via `backend/api/routes/holdings_helpers.py::_build_holding_row_from_snapshot()`

Why it matters:

- Day P&L formula `(ltp - close) × qty` is **always correct** under this invariant
- Off-market, both `ltp` and `close_price` are frozen, so day P&L is stable (correct — no market moved)

- Works identically across 1-day gaps, weekends, multi-day holidays

The gate: `closed_hours_or_broker`

Every live-data route uses this canonical gate:

```
# backend/api/helpers/snapshot_gate.py
async def closed_hours_or_broker(
    exchange: str,
    snapshot_fn: Callable[[], Awaitable[Response]],
    broker_fn: Callable[[], Awaitable[Response]],
) -> tuple[Response, str]:
    """
    Return (response, source) where source ∈ {"live", "snapshot", "snapshot-fallback"}
    broker_fn is NEVER called if any segment is CLOSED
    """
    if _any_segment_open(exchange):
        response = await broker_fn()
        return response, "live"
    else:
        response = await snapshot_fn()
        return response, "snapshot"
```

Invariant: `broker_fn` NEVER executes when markets are closed. Zero broker load after 15:30 IST (NSE) and 23:30 IST (MCX).

Delegation to exchange_clock — `_any_segment_open()` and `is_exchange_closed_now()` now delegate to `backend/api/helpers/exchange_clock` instead of reading hardcoded time constants. The exchange schedule is DB-backed and configurable via the `ExchangeSchedule` table (§4.6). See §4.13 for the exchange_clock architecture.

Files:

- `backend/api/helpers/snapshot_gate.py` — gate implementation
- `backend/api/helpers/exchange_clock.py` — DB-backed schedule cache (§4.13)
- `backend/api/routes/positions.py::get_positions()` — wired
- `backend/api/routes/holdings.py::get_holdings()` — wired
- All other data routes follow the same pattern

4.12 Day P&L computation — four formulas

The platform uses one of four distinct day P&L formulas depending on position type. Every value in the NavStrip P pill, Positions grid Day P&L column, and Performance page TOTAL row is computed using the SSOT logic below:

Frontend SSOT: `baseDayPnlForPosition(r)` in `nav.js`

```
/**
 * Primary path (preferred when prev_settlement_pnl available):
 *   day_delta = pnl − prev_settlement_pnl
 *
 * Fallback (for positions opened today):
 *   day_delta = pnl − overnight_qty × (close_price − average_price)
 *
 * Stale-snapshot guard:
 *   if close_price === ltp: return 0 (off-market snapshot, both values frozen)
 *
 * Short position fix (oq < 0):
 *   applied to both paths (not just oq > 0)
 */
export function baseDayPnlForPosition(r: PositionRow): number {
  if (!r) return 0;
  const oq = r.overnight_quantity ?? 0;

  // Primary path: use settled P&L delta
  if (typeof r.prev_settlement_pnl === 'number') {
    return r.pnl − r.prev_settlement_pnl;
  }

  // Stale-snapshot guard
  if (r.close_price <= 0 || (r.close_price === r.ltp)) {
    return 0;
  }

  // Fallback: compute yesterday's unrealized synthetically
  const yesterday_unrealised = oq * (r.close_price − r.average_price);
  return r.pnl − yesterday_unrealised;
}
```

The four formula cases

Case	Position state	Formula	Example
A: Overnight open	oq>0, qty>0	`(ltp − close) × qty`	Hold NIFTY from yesterday, still holding today
B: Closed overnight	oq>0, qty=0	`(exit_price − close) × qty`	Held overnight, exited during this session
C: New today	oq=0, qty>0	`(ltp − entry_price) × qty`	Opened & exited only today
D: Holdings	any qty	`(ltp − prev_close) × qty`	Equity holding (qty = remaining, not opening_qty)

Key invariant: `close` always means the **previous session's settlement LTP**, frozen at the moment the prior session ended. Never changes during this session.

Holdings sold: P&L splits

When a holding is partially or fully sold:

1. **Sold quantity** moves to a CNC positions row
2. **Holdings grid** shows **remaining quantity only** (not opening_quantity)
3. **Holdings day P&L** = `(ltp − prev_close) × remaining_qty`
4. **Sold-portion day P&L** lives **exclusively in the CNC positions row** (Case B formula)
5. **No sharing or double-counting** between holdings and positions surfaces

Files:

- `frontend/src/lib/data/nav.js::baseDayPnlForPosition` — sole SSOT (poll-only, §1 redesign — `livePositionDayPnl` /its SSE-tick delta term were removed entirely; positions/holdings Day P&L follows the book-poll cadence only, 5s foreground/30s background, no live-tick input)
- `backend/api/algo/pnl_math.py::apply_day_change_backstop` — backend fallback for case A only
- `backend/api/routes/positions.py::_override_stale_close_from_snapshot` — frozen-close guardian
- All callers: PerformancePage TOTAL row, Derivatives Greeks aggregate, Dashboard hero, MarketPulse position rows, Snapshot grid, Legs grid

---

4.13 Exchange schedule — DB-backed market timing configuration

Problem it solves: Hardcoded opening hours (`_NSE_OPEN_H` , `_MCX_CLOSE_M` , etc.) scattered across `background.py` made market-schedule changes (holidays, special sessions, new gates) require code edits + deployments. After Oct 2024 timezone drift fixes, all timing became configurable via the `ExchangeSchedule` table, eliminating hardcoded constants.

Design: Exchange schedule is a **configurable ORM model** with DB-backed in-process cache (60s TTL). Default rows are seeded at startup (date=NULL, filtered by weekday for regular schedules). Operator can create date-specific overrides for holidays + special sessions via the admin controller (`/api/admin/exchange-schedule`). The cache is refreshed on every mutation and consulted by `snapshot_gate.py` + background tasks.

API: exchange_clock module

File: `backend/api/helpers/exchange_clock.py`

Module-level state:

- `_TODAY_NSE_OPEN: time | None` — cached NSE session open time for today, set at startup and refreshed at 04:00 IST from `exchange_schedule` table
- `_load_today_open_time()` — async loader; reads effective rows for NON-MCX gate and extracts the open time (may be NULL on holidays)
- `get_nse_open_time() -> time | None` — sync accessor; returns the cached NSE open time, or None if unknown/holiday

Public API:

Function	Signature	Purpose
`is_exchange_open(exchange)`	`str → bool` (sync)	Check if a given exchange (e.g. "NSE") is open now. Resolves exchange to gate, then checks effective rows; skips rows where exchange not in `row.exchanges` . Returns False only if all effective rows are closed (`open_time IS NULL`).
`is_exchange_closed(exchange)`	`str → bool` (sync)	Inverse of `is_exchange_open`
`is_any_segment_open(exchanges)`	`List[str] | None → bool` (sync)	Check if ANY segment is open; None checks all
`sessions_with_snapshot_time_now()`	`() → List[ExchangeSchedule]` (sync)	Return rows whose `snapshot_time` matches within ±1 min (skips holiday rows with open_time=None)
`settlement_cutoff_for(gate)`	`str → datetime` (async)	Returns `default_row.open_time` (08:00) as the reset boundary for daily snapshots (used in positions/holdings close-price queries)
`refresh()`	`() → None` (async)	Reload cache from DB (skips if cache is fresh < 60s)
`seed_and_warm()`	`() → None` (async)	On_startup callable; seeds defaults, loads cache
`load_today_open_time()`	`() → None` (async)	Async loader for dynamic NSE open time; called at startup (before holiday refresh) and at 04:00 IST daily; sets `_TODAY_NSE_OPEN` from DB schedule
`get_nse_open_time()`	`() → time | None` (sync)	Returns cached NSE session open time for today, or None if unknown or market closed

Cache behavior:

- **Module-level:** `_CACHE: List[ExchangeSchedule]` (in-process, not shared across processes)
- **TTL:** 60 seconds; skip DB call if cache is fresh
- **Thread-safe:** `asyncio.Lock` around refresh so concurrent calls share one DB fetch
- **Fail-open:** If cache is empty or an exchange is unknown, assume market is open (returns True) so the live broker path keeps the data fresh

Override-first priority rule

Date-override-first rule — `_effective_gate_rows(gate: str) → list[ExchangeSchedule]` returns rows governing today's gate:

- If ANY row with `date == today` exists for the gate → absolute precedence; default row suppressed entirely
- Otherwise apply default row (date=NULL) with weekday filter
- Helper ensures only one flow path is active per gate per date

Open/closed flag — `open_time IS NOT NULL` means open; `open_time IS NULL` means closed (holiday). The `is_open` boolean column is no longer used in session logic.

Exchange-membership per-row — `is_exchange_open(exchange)` iterates `_effective_gate_rows(gate)` and skips rows where the exchange is NOT in `row.exchanges` :

- Resolves exchange (e.g., "NFO") to its gate (e.g., "NON-MCX")
- For each effective row, checks `exchange in row.exchanges` before checking session time
- Allows override rows covering a subset of exchanges (e.g. Muhurat on NSE+BSE but not NFO)
- If all applicable rows are skipped (exchange not in any), returns closed (False)

Default seed rows (simplified)

Two rows are created at startup (idempotent — duplicate key skips):

gate	exchanges	open_time	close_time	snapshot_time	weekdays	date
NON-MCX	NSE, BSE, NFO, BFO, CDS	08:00	15:30	15:45	[0,1,2,3,4] (Mon–Fri)	NULL (default)
MCX	MCX	08:00	23:30	23:45	[0,1,2,3,4] (Mon–Fri)	NULL (default)

Weekdays filter: Array of 0–6 (Mon=0 … Sun=6). Values [0,1,2,3,4] = Mon–Fri only; empty array or NULL means applies every day. Pre-open (PRE) and post-close (POST) sessions are absorbed into the NON-MCX main gate.

Consumer integration

snapshot_gate.py delegation:

```
def _any_segment_open(exchanges: list[str] | None = None) -> bool:
    try:
        from backend.api.helpers import exchange_clock
        return exchange_clock.is_any_segment_open(exchanges)
    except Exception:
        return True  # fail-open
```

background.py snapshot dispatch:

The old `_snapshot_probe_nse_mcx()` task was replaced. Now:

```
async def _snapshot_probe_nse_mcx():
    sessions = exchange_clock.sessions_with_snapshot_time_now()
    for session in sessions:
        if session.gate == "NSE":
            await trigger_close_snapshot("NSE")
        elif session.gate == "MCX":
            await trigger_settlement_capture("MCX")
        # ... per-gate dispatch
```

Positions/holdings cutoff:

```
# OLD (hardcoded): today_ist_8am = ...
# NEW (dynamic):
cutoff = await exchange_clock.settlement_cutoff_for("NSE")
rows = await session.execute(
    select(DailyBook).where(
        DailyBook.captured_at < cutoff
    )
)
```

Admin route: ExchangeScheduleController

Path: `GET|PUT|DELETE /api/admin/exchange-schedule`

- **GET** — list all rows (default + overrides)
- **PUT** — upsert a row (date=specific date creates an override; date=NULL updates a default)
- **DELETE** — remove a row by id

On mutation: Cache is invalidated + refreshed, so the next `closed_hours_or_broker` call picks up the change within 1 request latency.

Operator workflow: To close markets for a holiday:

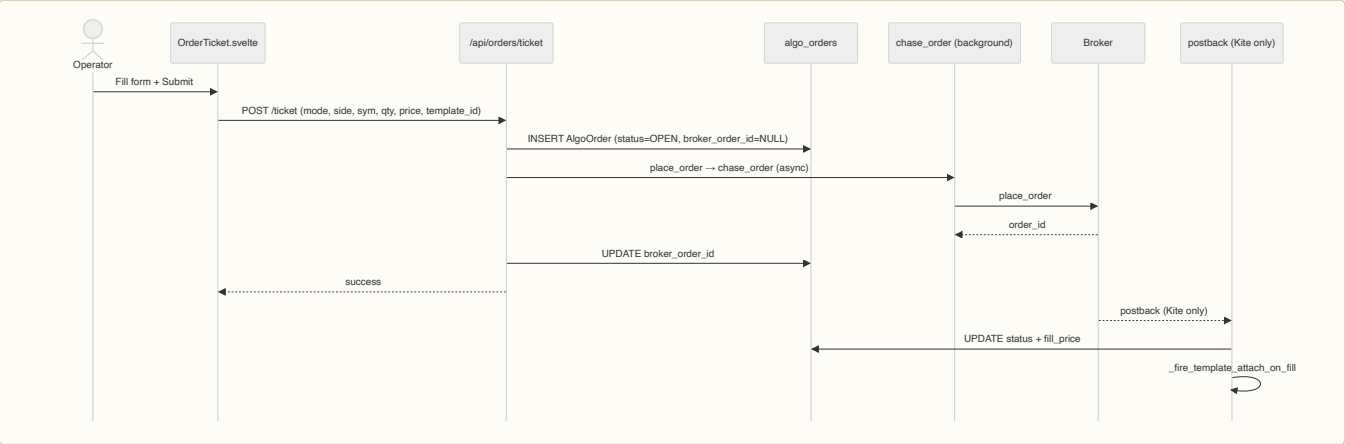
1. `/admin/exchange-schedule` → `PUT` row with `date=holiday_date, open_time=NULL, close_time=NULL, snapshot_time=NULL` for the desired gates (e.g., "NON-MCX" and/or "MCX")
2. Immediate effect: next refresh-rate call sees markets closed (`is_exchange_open` returns False); background tasks skip market-hours gates
3. To cancel the override, `DELETE` the row or `PUT` with the proper open_time restored

Files

- `backend/api/helpers/exchange_clock.py` — SSOT cache + public API
- `backend/api/routes/exchange_schedule.py` — admin CRUD controller
- `backend/api/models.py::ExchangeSchedule` — ORM model
- `backend/api/background.py` — updated `_snapshot_probe_nse_mcx()` (deprecated; integrated into on_startup + segment-specific background tasks)
- `backend/api/app.py::on_startup` — calls `exchange_clock.seed_and_warm` after `init_db`
- `backend/api/helpers/snapshot_gate.py` — delegates to `exchange_clock.is_exchange_closed` , `is_any_segment_open`

Part II — Order lifecycle

5. Order placement — single ticket (Ticket tab)



Files: `frontend/src/lib/order/OrderTicket.svelte` (submit) → `backend/api/routes/orders.py::ticket_order` → `backend/api/algo/chase.py::chase_order` → postback (Kite) or polling.

Race-window guard: AlgoOrder commits with `broker_order_id=NULL` first. Fast IOC fill in this window is caught by postback-fallback matching `(account, symbol, side, qty, status=OPEN)` within 60s.

5.1 F&O order quantity convention (lots-first API)

P0 commitment: all F&O order APIs (`/ticket` , `/basket` , `/preview-ticket-template`) now accept **LOTS** as input for instruments with `lot_size > 1` (futures + options). The API converts lots → contracts at the request boundary using `contracts = lots × lot_size` .

Request shape:

- **F&O** (`lot_size > 1`): send `quantity: <lots>` (frontend builds `_lots` key)
- **Equity** (`lot_size = 1`): send `quantity: <raw_qty>` (no change)

Guard chain (G1, G2 — critical):

Guard	Check	Level	Applies to
G1 `LOT_MULTIPLE`	`qty % lot_size == 0`	Removed from `_ticket_enforce_lot_and_fat_finger`	Dead (correct by construction)
		Kept in `_arm_take_profit` (live path, inline before broker call)	Live F&O close orders
		Kept in `apply_plan_live` GTT layer (top of function, sync check)	Template attach paths
G2 `FAT_FINGER_5_LOT_CAP`	`qty ≤ 5 lots` (NFO/CDS/BFO) or `≤ 20 lots` (MCX)	Main path, checked at `_ticket_validate_input`	All F&O places
	`CLOSE intent bypass`	G2 skipped if `intent='close'` (operator kill)	Closes only
Adapter ceiling (50-lot)	Hard block at `kite.py:place_order`	Applies to all quantities	Last-line defense (no bypass)

Files:

- `backend/api/routes/orders_place.py::_ticket_validate_input` — lots → contracts conversion
- `backend/api/routes/orders_place.py::_ticket_enforce_lot_and_fat_finger` — G2 check (G1 removed)
- `backend/api/routes/orders_basket.py` — same conversion + guards for batch legs
- `backend/api/algo/template_attach.py::apply_plan_live` — GTT layer G1 + G2 sync checks before `broker.translate_qty`
- `backend/brokers/adapters/kite.py::translate_qty` , `place_order` — adapter-level qty handling
- `frontend/src/lib/order/orderTicketSubmit.js::buildPlacePayload` — sends `_lots` for F&O

Cold-cache 503 guard — extended from MCX-only to ALL F&O exchanges (NFO, CDS, BFO, BCD). When instruments cache is empty at startup or post-refresh, `/api/orders/ticket` returns 503 with a "cold start" message if it can't resolve the lot_size. The operator retries after 5-10s once warm.

Schema update — `BasketLeg` msgspec.Struct adds optional `strategy_id: Optional[int] = None` field for future per-leg strategy attribution.

5.2 Order entry data displays — LTP in tab bar, OrderDepth shows prior close only

Tab bar LTP display — The SymbolPanel tab bar (`oes-tab-ltp` class in SymbolPanel.svelte) is the **canonical LTP display** on the order-entry modal. It is **always visible**, updated via SSE WebSocket push at 4Hz cadence, and serves as the operator's primary real-time price reference while filling an order.

OrderDepth panel constraint — The OrderDepth component (bid/ask depth ladder below the Ticket form) displays the **prior session's close price only** (not live LTP) to avoid duplication and reduce visual clutter. The depth ladder is used for volume profile reference (how many orders at each level); the price reference should come from the always-visible tab bar. This prevents the operator from having two competing price signals on the same modal.

Rationale:

- Single LTP source of truth (tab bar) ensures consistent price reference across all order tickets and basket legs
- OrderDepth avoids information redundancy; depth data and price level reference are independent concerns
- Prior close in OrderDepth provides context for overnight gap without distracting from live action

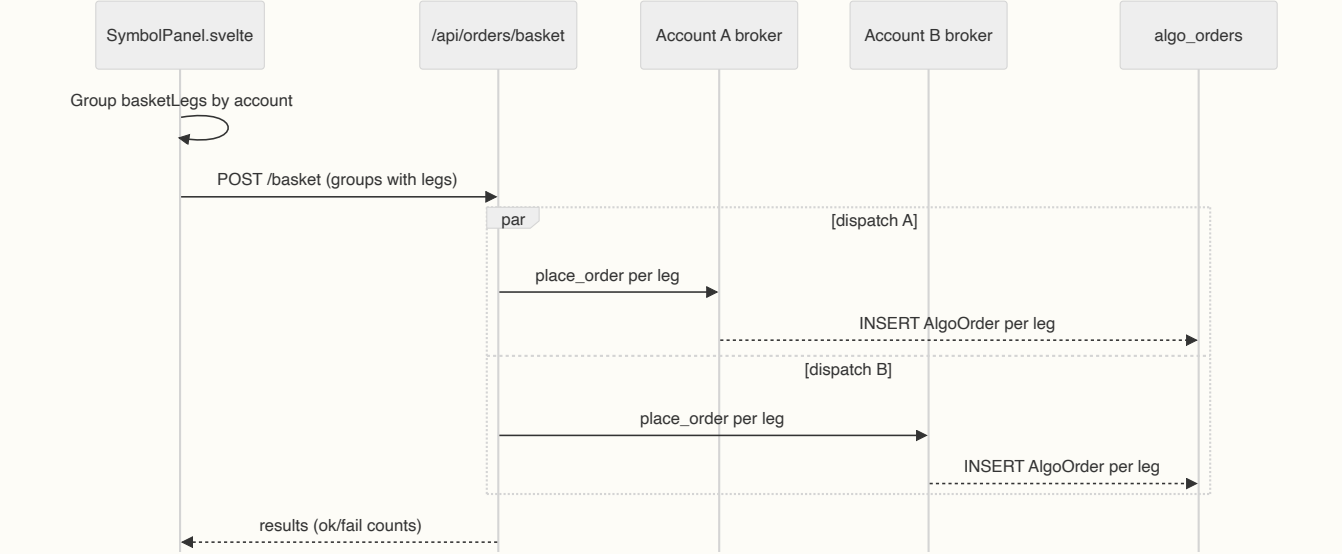
Files:

- `frontend/src/lib/SymbolPanel.svelte` — tab bar LTP display, `oes-tab-ltp` class
- `frontend/src/lib/order/OrderDepth.svelte` — prior close display logic; no live LTP rendering

---

6. Order placement — basket (Chain tab)

Multi-leg submission grouped per-account, dispatched in parallel via `asyncio.gather` :



Files: `frontend/src/lib/SymbolPanel.svelte::submitBasket` (grouping) → `backend/api/routes/orders.py::place_basket` (parallel dispatch).

Template isolation: legs with explicit `template_id` ignore shell overrides. Legs with no `template_id` inherit shell defaults.

6.1 Option chain API and expiry picker

Endpoint: `GET /api/options/chain-quotes?symbol=<sym>&exchange=<exch>&expiry=<YYYY-MM-DD>` (optional expiry)

The chain-quotes endpoint powers the **Chain tab** option-chain strike picker. Two modes of operation:

Mode 1 — Expiry list only (no expiry param)

```
GET /api/options/chain-quotes?symbol=NIFTY&exchange=NFO
→ {expiries: ["2026-08-14", "2026-08-21", "2026-08-28"], rows: []}
```

Response time: ~1s (API scan of instruments table, no broker call). Used when the tab first loads — populates the expiry dropdown without waiting for the full instruments cache.

Mode 2 — Full chain (with expiry param)

```
GET /api/options/chain-quotes?symbol=NIFTY&exchange=NFO&expiry=2026-08-14
→ {
    expiries: [...],
    rows: [
      {strike, ce_sym, ce_ls, pe_sym, pe_ls, exchange, ce_bid, ce_ask, pe_bid, pe_ask, ...},
      ...
    ]
  }
```

Response time: ~1–2s (backend calls broker for quotes on resolved symbols). Results cached for 3 seconds per symbol/expiry pair.

Response shape (`ChainQuoteRow`):

- `strike` — strike price (e.g., 24500)
- `ce_sym` , `pe_sym` — resolved call/put tradingsymbols (e.g., `NIFTY26AUG24500CE`)
- `ce_ls` , `pe_ls` — call/put lot sizes (used for order quantity validation)
- `exchange` — which exchange these contracts trade on
- `ce_bid` , `ce_ask` , `pe_bid` , `pe_ask` — live quotes for each leg
- Plus standard fields: `ce_oi` , `pe_oi` , `ce_iv` , `pe_iv` , etc.

Frontend flow (OptionChainTab.svelte):

1. Tab opens → calls `chain-quotes` without `expiry` → expiry dropdown populates
2. User clicks expiry → calls `chain-quotes` with `expiry` → strikes populate
3. User clicks strike → prepopulates Ticket form with resolved symbols + qty from row

Backend implementation (`backend/api/routes/options.py::chain_quotes`):

- When `expiry` is None: scans instruments table for unique expiries on symbol/exchange, returns sorted list
- When `expiry` is set: resolves CE/PE symbols via instruments map, fetches quotes via broker.quote(), applies OI/bid-ask filters
- All broker calls wrapped in `asyncio.to_thread` with 3s TTL cache via `@get_or_fetch`

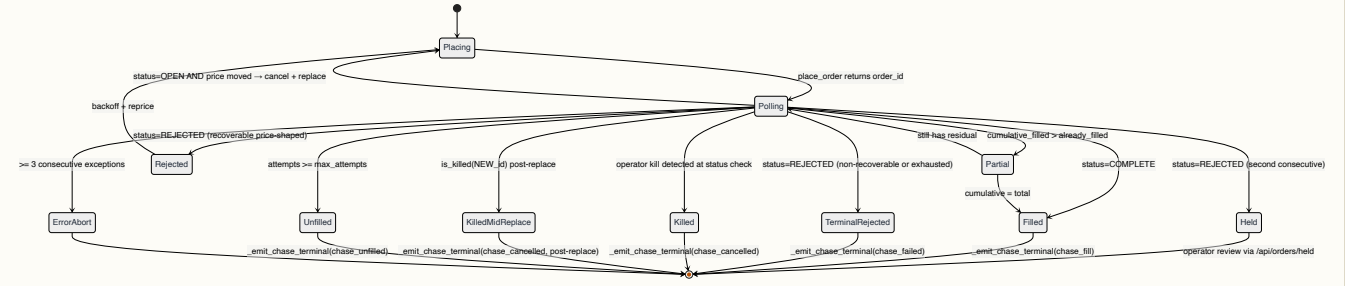
Invariant: Symbol resolution (instrument lookup) happens at the backend. Frontend never calls broker APIs directly; it builds forms from backend-resolved data.

Files:

- `backend/api/routes/options.py::chain_quotes` — API handler
- `frontend/src/lib/order/OptionChainTab.svelte` — UI + expiry/strike selection
- `backend/api/models.py::ChainQuoteRow` — response schema

---

7. Chase loop lifecycle



Key files:

- `backend/api/algo/chase.py::chase_order` — main loop
- `backend/api/algo/chase.py` — partial-fill branch (search `_record_partial_fill`)
- `backend/api/algo/chase.py` — kill-race post-replace check (search `is_killed(current_order_id)` after `_sync_algo_order_id`)
- `backend/api/algo/chase.py::_emit_chase_terminal` — snapshot + downstream attach
- `backend/api/algo/chase.py::_sync_algo_order_id` — writes `broker_order_id` , `current_limit` , `last_attempt_at` , `next_attempt_at` , and `interval_seconds`

Timing columns in AlgoOrderInfo — `next_attempt_at` , `last_attempt_at` , and `interval_seconds` are now exposed in the API response. Used by `ChaseCard.svelte` to display countdown timer, last-attempt age (e.g., "Next attempt in 12s" · "Last attempt 3m ago"), and the current chase interval.

⚙ **TECH — sync polling vs WebSocket order updates** — `WHY` Postback delivery is unreliable for non-Kite brokers (Dhan + Groww are poll-only). Sync polling is the lowest-common-denominator that works everywhere. `WHAT` `chase_order` calls `_order_status` every 20s (configurable per chase). `HOW` Each iteration: depth quote → adjusted limit → cancel old + place new → sync ID → wait → poll status. `WHERE` `backend/api/algo/chase.py`.

Partial-fill math (post C-1 fix):

```
cumulative_filled tracks fills across all cancel-and-replace attempts (counter, not recomputed)
new_delta = cumulative_filled − already_filled (from prior fill check)
fire partial branch when: cumulative_filled > 0 AND new_delta > 0 AND cumulative_filled < quantity
```

Market-hours pre-flight (added 2026-07): Before the first `_place_order` call, `chase_order` checks `is_any_segment_open()`. If the exchange is closed, the function returns `ChaseResult(FAILED)` immediately and fires an operator alert. No broker call is made.

7.1 Chase recovery on startup

`backend/api/background.py::recover_live_chases()` fires during app startup to restart chase loops that were interrupted by a service restart. Queries for rows where `status='OPEN'`, `engine='live'`, `next_attempt_at IS NOT NULL`, and `created_at < 120s ago` (grace period to avoid restarting chases still initializing). For each recovered row, restores the original `intent` field from the DB — when `intent='close'` was set by a position-close chase, recovery re-applies it so the 50-lot ceiling bypass applies to the resumed chase. This prevents close orders from being silently truncated to 50 lots after a restart.

Key files:

- `backend/api/background.py::recover_live_chases()` — recovery handler
- `backend/api/background.py::on_startup()` — calls recovery on app boot (line ~4953)

7.2 Order lifecycle events — audit trail per order

`backend/api/algo/order_events.py` exports `write_event(order_id, kind, detail)` — a fire-and-forget journal entry for every significant order event. Four event kinds fire automatically on the live path:

Kind	When	Caller	Detail example
`placed`	After live broker.place_order succeeds	`_opp_live_handle_success` in `orders_place.py`	`"LIVE placed as order_id=12345"`
`agent_trigger`	After live agent action fires (cancel, modify, etc.)	`_write_live_order` in `actions.py`	`"loss-alert: cancel at −₹5K"`
`reconcile`	During open-order watchdog reconcile (if status changed)	`_rco_reconcile_account` in `orders.py`	`"reconciled: broker returned FILLED"`
(implicit via audit log)	On postback / chase terminal / manual cancel	Postback/chase/UI	Captured by audit_log middleware

Events live in the `algo_order_events` table (per-order timeline, 90-day retention) and surface in the Order Activity panel and `/admin/history` drill-through. Kind `"agent_trigger"` is valued for post-mortem — "why did this agent fire when it did?" — and links `agent.slug` to the trigger event for compliance.

Implementation: All writers use `backend/api/audit.py::write_event` (async fire-and-forget); failures swallow to logger.warning, never block the request.

Files:

- `backend/api/algo/order_events.py::write_event` — journal writer
- `backend/api/models.py::AlgoOrderEvent` — schema
- `backend/api/routes/orders.py` — postback / reconcile event callers
- `backend/api/routes/orders_place.py::_opp_live_handle_success` — placed event for live orders

7.3 Cancel alert on postback

`backend/api/routes/orders.py::_opp_send_cancel_alert` fires fire-and-forget on every postback with `status='CANCELLED'`. Sends ntfy alert (Telegram / ntfy.sh) with masked account, side (BUY/SELL), qty, symbol, and status reason.

Pre-fix: operator could see an order silently cancel at the broker (e.g., Kite GTT rejection, Dhan order kill) with no notification. Now every cancel surfaces immediately.

Alert payload masks the account number (e.g., `ZG***`) and includes readable fields:

```
Order cancelled — symbol=NIFTY50 side=BUY qty=1 account=ZG*** reason=GTT_REJECTED
```

Called from `_postback_broadcast_fanout` (inside `order_postback` route) with no latency added to the broker ack path (async dispatch via `asyncio.create_task`).

Files:

- `backend/api/routes/orders.py::_opp_send_cancel_alert` — alert sender
- `backend/api/routes/orders.py::_postback_broadcast_fanout` — invocation point

---

8. The order/chase/template tripod

This is the most complex part of the codebase. Three subsystems with overlapping responsibilities:

8.1 What each one owns

	Owns	Reads
Order routing (`orders.py`)	The broker-facing entry path. Single ticket + basket.	Settings, templates
Chase loop (`chase.py`)	The per-order placement lifecycle. Cancel + replace + status polling + partial fill accounting.	Broker, AlgoOrder, kill signal
Template attach (`template_attach.py`)	Post-fill exit-rule wiring. TP/SL/Wing/Scale/Trail GTTs at the broker.	OrderTemplate, AlgoOrder (read-only at attach time)

8.2 Why three? Why not one big "manage this order" function?

History: the chase loop existed first (single ticket → place + chase to fill). Templates were bolted on later (Phase 0–3 + Sprints A–E). The current shape is intentional — each subsystem can be tested in isolation:

- Chase tests use a mock `_order_status` that returns a scripted sequence.
- Template tests build a `TemplatePlan` directly and assert the GTT spec shape.
- Routes are integration-tested with real broker mocks (`backend/tests/`).

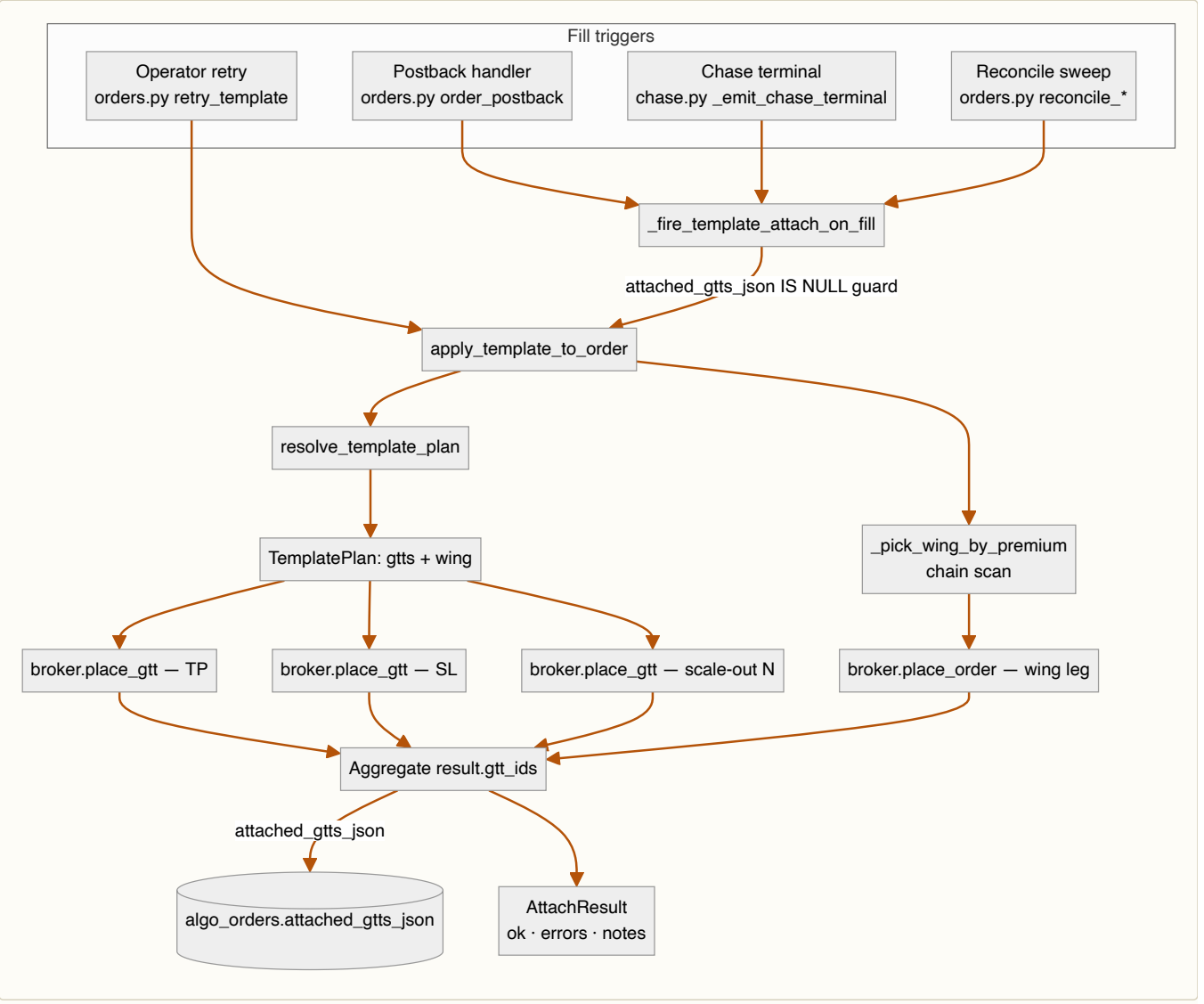
8.3 The mode pivot

`mode ∈ {sim, paper, live, shadow}` decides which adapter the order actually hits. The pivot happens at submit time (`_resolve_mode` in `backend/api/algo/actions.py`) and is **persisted on the AlgoOrder row** — every downstream branch (chase terminal, postback, reconcile, template attach) reads `row.mode` to decide whether to call a real broker or the paper engine.

Gotcha: the chase loop runs the same code regardless of mode. Paper mode is achieved by injecting the paper engine's `place_order` adapter at the broker registry boundary. Don't add `if mode == 'live'` branches inside chase — the abstraction is the broker registry, not the chase.

Part III — Templates + exits

9. Template attach pipeline



Key files:

- `backend/api/algo/template_attach.py::resolve_template_plan` — override merge + scope resolution
- `backend/api/algo/template_attach.py::_pick_wing_by_premium` — OI + spread filters
- `backend/api/algo/template_attach.py::AttachResult` — return type (NOT `TemplateAttachResult` — the docs previously had this wrong)
- `backend/api/routes/orders.py::_fire_template_attach_on_fill` — idempotency guard + persistence
- `backend/api/routes/orders.py::retry_template` — manual re-fire path. Persists `attached_gtts_json` per H-7 + trail-stop scaffolding per Sc.5

Market-hours guard (added 2026-07): After lot_size resolution and before plan computation, `_symbol_exchange_open()` is called. If the exchange is closed:

- Templates with a wing leg (MARKET order): attach is refused, operator alert fires, `AttachResult.errors` returned.
- GTT-only templates: proceed normally (Kite accepts GTTs 24×7).

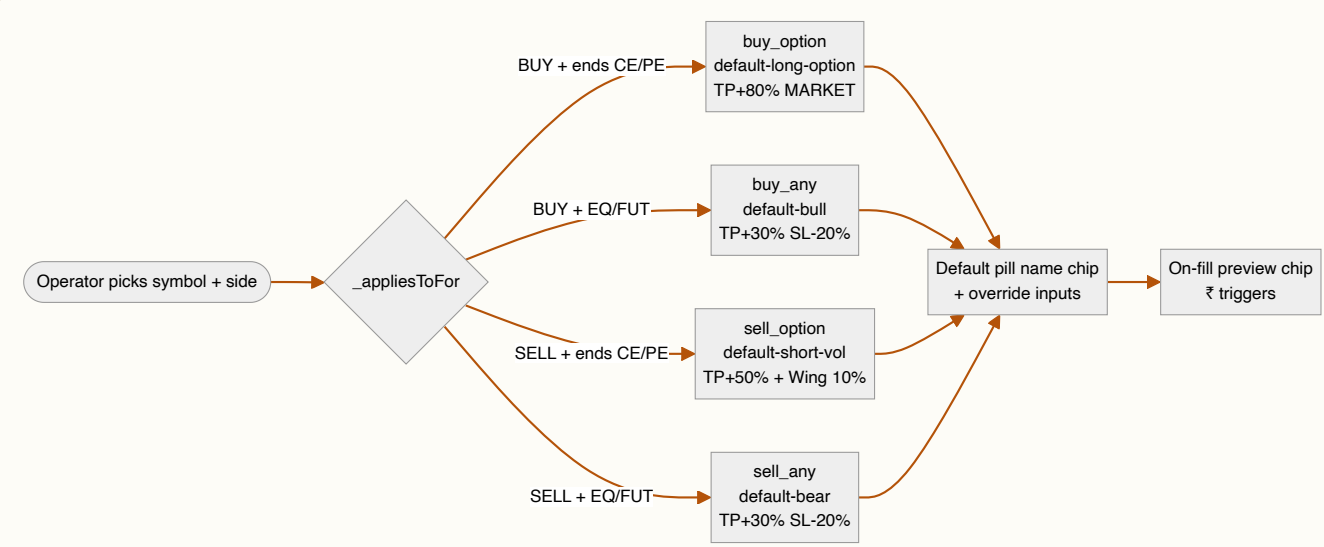
`apply_plan_live` also checks the exchange-open state before any broker call and returns `AttachResult.errors` if closed.

Idempotency: `_get_template_attach_lock(parent_row_id)` + `attached_gtts_json IS NULL` check. Strong dict with 1h TTL after M-5 fix replaces the prior WeakValueDictionary.

⚙ **TECH — JSON blob vs normalized table for `attached_gtts_json`** — `WHY` Each parent has 1-5 GTTs + maybe a wing. A child table would mean a JOIN on every order grid render. The blob lets us read the whole bracket in one column-fetch. `WHAT` Stored as a JSON array of entries (`{kind: "gtt", label: "TP", id: "...", current_trigger: ..., sl_trail_pct: ...}`). `HOW` Always write atomically (single column update);

read+parse on every access (cheap because rows are small). `WHERE backend/api/models.py::AlgoOrder.attached_gtts_json + _fire_template_attach_on_fill .`

10. 4-default template matrix



Key files:

- `backend/api/algo/templates_seed.py::SYSTEM_TEMPLATES` — seeded defaults + rebalance logic
- `frontend/src/lib/order/OrderTicket.svelte::_appliesToFor`
- `frontend/src/lib/SymbolPanel.svelte::_appliesToFor` — same helper at shell level
- `frontend/src/lib/SymbolPanel.svelte::_sideAwareDefault` — with fallback to focused-leg symbol
- `frontend/src/lib/automation/BracketsTab.svelte` — Order Template browse/manage (ported from deleted `/automation/templates` route)

11. Template override merge

Operator overrides flow through multiple layers; understanding the merge order saves hours of debugging.

```
Request:
  template_id=T1, tp_pct_override=20, sl_pct_override=None

Backend persist (orders.py::_build_overrides_json ~line 541):
  AlgoOrder.template_id          = T1
  AlgoOrder.template_overrides_json = '{"tp_pct": 20}'   # only NON-None overrides

At fill (template_attach.py::_pick):
  tp_pct = _ov.get("tp_pct") or template.get("tp_pct")
       → 20 (override wins)
  sl_pct = _ov.get("sl_pct") or template.get("sl_pct")
       → template's saved sl_pct (no override)
```

Per-leg vs shell: when a basket leg has `template_id` set explicitly (not inherited from `_sharedTemplateId`), the SHELL overrides DO NOT flow through. This is the audit-Sc.12 fix — pre-fix the shell's `tp_pct_override` silently contaminated a leg that the operator had retargeted to a different template.

```
submitBasket logic:
  effective_template = leg.template_id ?? shell_template
  if leg has its own template_id:
      tp_override = leg.tp_pct_override  (do NOT fall through to shell)
  else:
      tp_override = leg.tp_pct_override ?? shell.tp_pct_override
```

12. Chase loop invariants

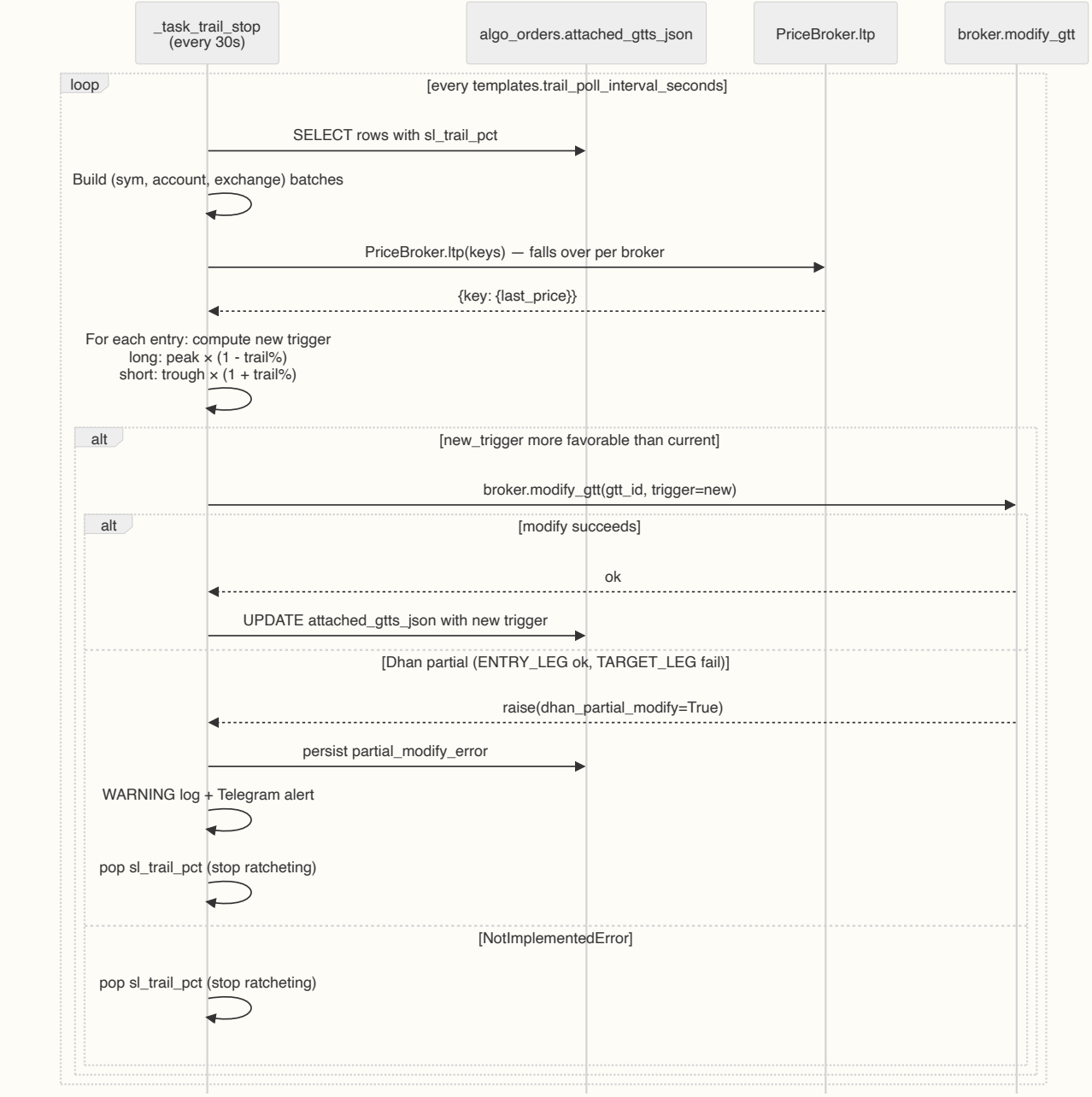
Six things the chase loop MUST guarantee:

1. **AlgoOrder.broker_order_id always matches the LATEST broker order.** Cancel-and-replace updates this via `_sync_algo_order_id`. Without it the postback handler can't resolve a row by `broker_order_id`.
2. **AlgoOrder.current_limit reflects the latest re-quoted limit.** Added in M-6. ChaseCard renders this when present so the operator sees the live limit, not entry.
3. **AlgoOrder.filled_quantity is monotonic and never exceeds AlgoOrder.quantity.** The `MAX(prior, cumulative)` clamp in `_record_partial_fill` enforces this post C-1 fix. Template attach reads `filled_quantity` to size exit GTTs.
4. **Operator kills take effect on the very next loop iteration.** `mark_killed(broker_order_id)` is synchronous + the loop checks `is_killed(current_order_id)` (a) at status-check time AND (b) immediately after replace (C-2 fix). The dict has a 60-min TTL so a stale kill flag can't survive across days.
5. **Partial fills get persisted on every NEW delta, not just the first.** The branch fires when `cumulative > already_filled` post-C-1 fix.
6. **A chase that hits >= 3 consecutive exceptions aborts** (`_MAX_CHASE_ERRORS = 3`). Prevents infinite re-trying against a broker that's down.

Break any of these and template attach sizes wrong, kills get ignored, or zombie chases burn rate limit.

---

13. Trail-stop subsystem



Key files:

- `backend/api/background.py::_task_trail_stop`
- `backend/api/background.py` — Dhan partial-modify detect + alert (M-2 fix, search `dhan_partial_modify`)
- `backend/brokers/adapters/dhan.py::modify_gtt` — two-leg dispatch (Sprint C)
- `backend/brokers/adapters/groww.py` — emulated OCO trail (currently `NotImplementedError`-skip)
- `backend/brokers/adapters/dhan.py::ltp` — wired via instruments cache (B-2 fix)

Asymmetric SELL guard note: the poller's SELL ratchet check is `current_trigger > 0 AND proposed < current_trigger`. If `current_trigger=0` (entry never persisted), the guard short-circuits → trail silently dead. This is why **every persistence path that writes a trail entry MUST seed `current_trigger`** (see Sc.5a fix in `retry_template`).

13.1 Order ticket enhancements (added 2026-07)

The OrderTicket modal now displays rich market context for every order:

- **Contract LTP in header** — live last-traded price next to the symbol name, updates every 2s
- **Underlying spot price** — header middle shows the index or commodity root level (for options/futures)
- **Days-to-expiry chip** — F&O contracts show remaining DTE in symbol meta row; amber when ≤3 days
- **Market depth section** — OpenInterest, Volume, and bid-ask spread displayed in-modal
- **Position context for CLOSE orders** — entry price + unrealized P&L shown before submission
- **ATM/ITM/OTM classification** — option orders display moneyness relative to current spot
- **Chase aggressiveness row** — LIMIT order type displays a three-mode toggle (Low / Med / High); relocated from header to price-entry row for better UX
- **Exchange-closed badge** — header indicates when market is closed; template with wing leg shows warning
- **Wing-leg warning chip** — amber chip appears when a wing-enabled template is selected during closed hours

Files:

- `frontend/src/routes/(algo)/orders/+page.svelte` — OrderTicket modal
- `frontend/src/lib/order/OrderTicket.svelte` — component logic
- `frontend/src/lib/order/orderTicketSubmit.js` — submission handler
- `backend/api/routes/orders_place.py` — backend validation + market-hours checks

13.2 Pair system architecture

RamboQuant has one pairing concept that operates on positions: **auto-pairing**, a lot-waterfall that groups opposing legs for hedge visibility. It is computed by the system and the operator cannot change it. A separate **parent-child link** (`parent_order_id`) ties a system-armed take-profit or template child order to the entry order that spawned it. The two are unrelated and never share a field.

> **REMOVED 2026-10-17** (commits `d760f352`, `2467151c`): the manual ⟷ **Pair** button, `OrderPairModal`, the `POST /api/orders/pair` route, and the manual position-pin feature (`PositionPinPicker`, `POST /api/positions/pin`, `DELETE /api/positions/pin/{id}`, the `pin_group_key` field, and the `position_pairs` table) were deleted by operator decision. No operator-driven pairing exists. The `position_pairs` table is dropped by the startup migration `_migrate_drop_position_pairs_table()` in backend/api/database.py. Everything below describes live code only.

Position pair groups (auto-pairing)

`pair_group_key` ("P1", "P2"...) is computed server-side by the lot-waterfall algorithm `_auto_pair_positions()` at backend/api/routes/positions.py:108. These are ephemeral: recomputed on every positions fetch, never persisted. The algorithm matches legs automatically; the operator has no control over which legs pair.

Grouping key: `(account, root_symbol, option_type)`. `option_type` is `CE`, `PE`, or `None` (futures and equities). It comes from a digit-anchored suffix check (`_option_type()` at positions.py:75), so equities whose tickers end in "CE" or "PE" (RELIANCE, BAJFINANCE) are never read as options. A CE leg can never waterfall-match a PE leg, even at equal and opposite quantity.

Waterfall: within each group, longs and shorts are matched largest-first by quantity. Pair labels are numbered globally across all groups, visited in a deterministic sort order (account, root, option_type), so the same book yields the same P-numbers regardless of the order the broker returned rows.

Waterfall fields:

- `pair_group_key`: "P1", "P2"... (sequential) or `None` for unpaired legs
- `paired_qty`: quantity matched with the counterpart leg (lots)

- `orphan_qty` : unmatched remainder (`abs(quantity) − paired_qty`)
- `is_orphan` : `True` when `orphan_qty > 0`

UI display (St column): Pulse → Positions grid and the derivatives Legs grid share one badge chain in frontend/src/lib/data/positionStateBadge.js. Highest priority wins:

- GTT (green): the leg has a GTT stop-loss or target
- P1/P2 (cyan): auto-paired group label
- ◇ (amber): orphan, the leg has unmatched quantity

Account constraint (hard invariant): pairing is always intra-account. The grouping key includes `account` , so cross-account legs never pair. Rationale: broker position offsets are settled per account, and Kite/Dhan/Groww do not recognise cross-account hedges.

Execution isolation: chase, bracket armature, and template attach do not read `pair_group_key` or `is_orphan` for any execution decision. Reconciling the waterfall with the MCX CE/PE-net classifier used by expiry auto-close is a separate, open decision.

Parent-child order links (`parent_order_id`)

`AlgoOrder.parent_order_id` (backend/api/models.py:803) points a child order at the entry order that spawned it. Only system paths write it: take-profit arming after a fill, chase, and template wing child orders. No operator action sets it. Two behaviours depend on it:

- **Take-profit double-arm guard:** a parent that already has a child take-profit is not armed again. The check runs inside the parent row's `with_for_update()` lock.
- **Template eligibility:** rows that have a parent are not eligible for template attach or retry (see the gates in `orders_postback.py` and `orders_place.py`).

Deferred: dual-leg template placement on paired positions

No decision to build this. The original sketch placed a hedged GTT on both legs of a paired position. It depended on operator-persisted pairs, which were removed on 2026-10-17, so it is not on the roadmap. It would need a new product decision first, and CLAUDE.md's execution-isolation rule would have to change. The sketch, for reference:

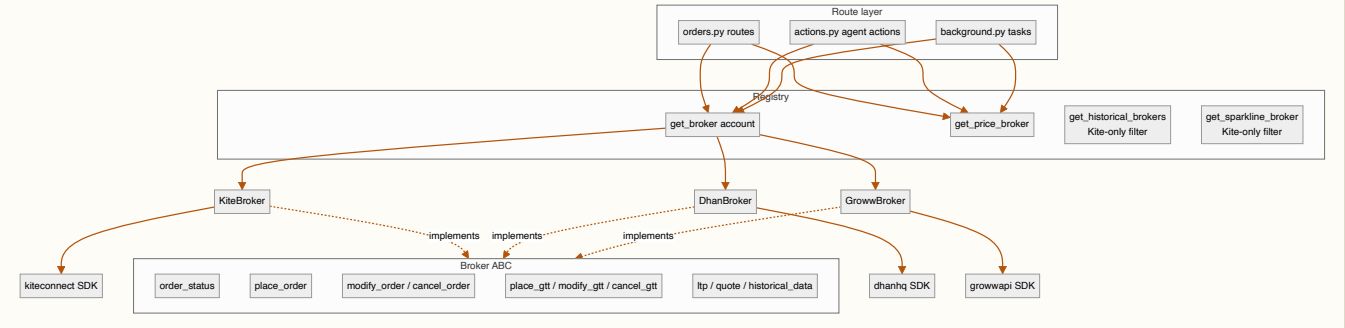
1. `pair_group_id: Optional[str]` on `AlgoOrder` , separate from `parent_order_id` .
2. `TemplatePlan` gains `sibling_account` , `sibling_symbol` , `sibling_qty` (backend/api/algo/template_attach.py:112; currently single-leg only).
3. `apply_plan_live()` coordinates GTT placement on both legs atomically when a counterpart fill arrives.

Key files:

- backend/api/routes/positions.py:75-230 (`_option_type` , `_auto_pair_positions`)
- frontend/src/lib/data/positionStateBadge.js (St-column badge priority, SSOT)
- frontend/src/lib/data/pairGroupSort.js (pair-group adjacency sort)
- backend/api/models.py:803 (`AlgoOrder.parent_order_id`)
- backend/api/algo/template_attach.py:112 (`TemplatePlan` definition)

Part IV — Brokers

14. Broker abstraction



Capability matrix surface:

- `backend/brokers/capabilities.py::BrokerCapabilities` — dataclass with every capability flag
- `backend/brokers/registry.py::get_historical_brokers` — Kite-only filter
- `frontend/src/lib/data/brokerCapWarnings.js` — single source of truth for warning strings (H-5)

- `frontend/src/lib/order/OrderTicket.svelte::capWarningFor` — single-account
- `frontend/src/lib/SymbolPanel.svelte::aggregateCapWarnings` — cross-account (H-5)

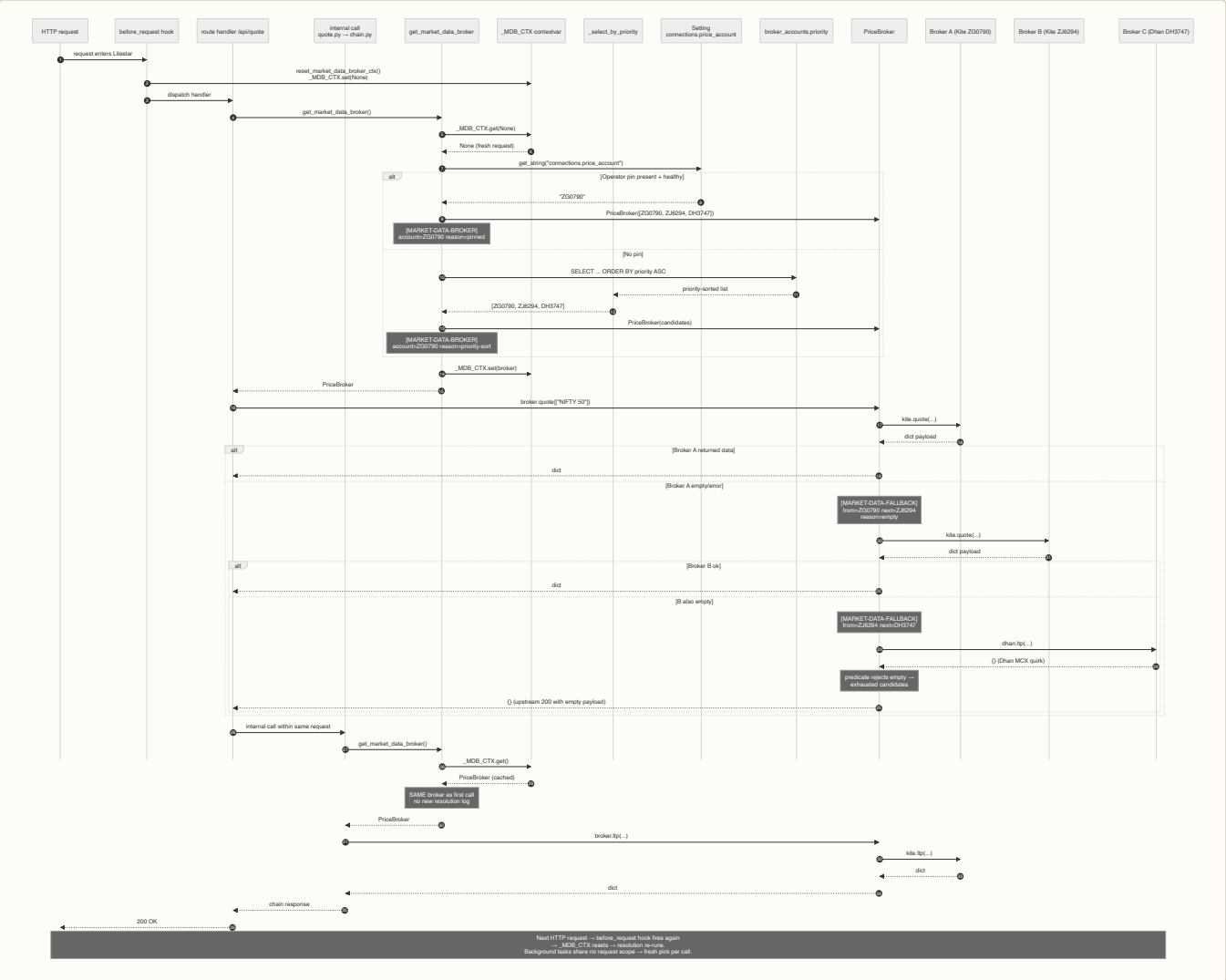
⚙ **TECH — PriceBroker fallback chain** — `WHY` Some brokers can answer quote/ltp/historical (Kite), some can't (Dhan returns `{}` by design for `quote`). Walking the chain lets the operator's chart still render even when their primary account is throttled. `WHAT` `PriceBroker._try(method_name, *args)` iterates eligible brokers, calls method, checks predicates (`_quote_has_data` / `_ltp_has_data` / `_historical_has_data`), returns first successful response. `HOW` Add a new method by name in the predicate map. Rate-limit cool-off (`_RATE_LIMIT_COOLOFF`) excludes throttled accounts for 30s. `WHERE` `backend/brokers/registry.py::PriceBroker`.

14.1 Kite account flipping — market-data broker resolution

`get_market_data_broker()` is the **single** resolution path for every quote / ltp / instruments / historical_data call in route handlers and background tasks. A `contextvars.ContextVar` (`_MDB_CTX`) caches the `PriceBroker` instance for the lifetime of one HTTP request so every callsite within the same handler picks the **same** broker session. On `PriceBroker._try()` failover: try broker A, on transient error / empty predicate, log `[MARKET-DATA-FALLBACK]`, try broker B, etc.

Selection order:

1. `connections.price_account` operator pin (setting)
2. `broker_accounts.priority` ASC
3. Insertion order



Intentionally NOT wired to `_MDB_CTX`:

- `get_sparkline_broker()` — must spread the 3 req/sec Kite historical_data budget across accounts.
- `get_historical_brokers()` — same reason; returns the full Kite-only list for round-robin OHLCV pulls.
- `@for_all_accounts` fan-out (positions / holdings / margins) — fans out per-account by design; the contextvar is irrelevant.

Files:

- `backend/brokers/registry.py` — `_MDB_CTX`, `reset_market_data_broker_ctx`, `get_market_data_broker`, `get_price_broker`, `PriceBroker._try`
- `backend/api/app.py` — `before_request` hook wiring
- Wired callsites (15 files): `quote.py`, `watchlist.py`, `options.py`, `strategies.py`, `positions.py`, `orders_place.py`, `hedge_proxies.py`, `admin.py`, `instruments.py`, `background.py`, `lot_ledger.py`, `paper.py`, `template_attach.py`, `replay/driver.py`, `ohlcv_store.py`, `broker_apis.py`

14.2 Data normalization — quantity/lots/lot_size

File: `backend/brokers/broker_apis.py::_annotate_lot_size()`

After every broker fetch, `_annotate_lot_size()` normalizes all positions to a uniform quantity unit: **CONTRACTS**.

Why: Kite ships MCX positions in LOTS (e.g., lot_size=100, Kite qty=5 → 500 contracts). NFO ships in CONTRACTS. Equity as-is. Without normalization, P&L math and order-entry validation branches inconsistently.

Implementation:

1. Look up the instrument's `lot_size` from the broker's `_LOT_INDEX` (pre-loaded at startup).
2. **For MCX/NCO rows** — multiply `quantity`, `overnight_quantity`, `day_buy_quantity`, `day_sell_quantity` by `lot_size` (convert Kite lots → contracts).
3. **For NFO/CDS/BFO rows** — already in contracts; derive `lots = quantity / lot_size`.
4. **For equity** — no-op; `lots = quantity`, `lot_size = 1`.
5. Add informational fields `lots: int` and `lot_size: int` to every row (audit + UI).

Persistent in daily_book: Both `lots` and `lot_size` columns are written to the `daily_book` table (via SQL migration). They are informational; all P&L and day P&L formulas operate on `quantity` (contracts).

Key invariant: After `_annotate_lot_size()` returns, **`daily_book.quantity` is ALWAYS in CONTRACTS**, regardless of exchange or lot structure.

Bug fixes bundled (Aug 2026):

1. **Day P&L stale-preservation guard** — `day_pnl = CASE WHEN EXCLUDED.ltp IS NOT NULL THEN ...formula... END` ensures that mid-session passes with NULL LTP don't incorrectly preserve a prior session's day_pnl when the formula returns 0 (flat close).
2. **Previous-close advancement gate** — `previous_close` only advances when `ltp` actually changes (not on frozen weekend snapshots). Prevents the next session's day P&L formula `(ltp - previous_close)` from computing wrong values.

Files:

- `backend/brokers/broker_apis.py::_annotate_lot_size` — normalization SSOT
- `backend/api/algo/daily_snapshot.py::snapshot_daily_book` — UPSERT logic with guards
- `backend/api/models.py::DailyBook` — `lots` and `lot_size` model fields
- `backend/brokers/adapters/kite.py::_LOT_INDEX` — lot-size lookup table

14.5. Broker abstraction — implementation detail

Full broker layer architecture — file map, singleton lifecycle, token caching, source-IP binding, and capability matrix — **lives in CLAUDE.md §14.5 for brevity**. This section is a quick read list only.

Files — `backend/brokers/{base.py, kite.py, dhan.py, groww.py, capabilities.py, registry.py}` + `backend/shared/helpers/{connections.py, broker_creds.py, kite_ticker.py}` + `backend/brokers/service/conn_events.py`.

Connection audit log — every auth attempt, token rotation, fetch failure, and circuit-breaker transition is recorded in `broker_connection_events` table via `_emit_conn_event()` in `conn_events.py`. Operator diagnostic endpoint: `GET /api/admin/health/broker-connection-events` — filters by account/event_type/since/limit. See BROKER_SPEC.md §14 for event types.

Key rules:

1. **Kite-shape contract** — every return value must match Kite Connect shape. Dhan/Groww adapters have `_normalise_*` helpers. The `_DHAN_STATUS_TO_KITE` status map is critical (audit B-1).
2. **Singleton per process** — adapters live via `Connections()` singleton. Each Kite login takes 10-15s; re-doing per-request is unworkable.
3. **IPv6 source-binding** — Kite + Dhan enforce one-session-per-IP rules. Each account binds to a unique IPv6 via `_IPv6SourceAdapter` (Kite/Dhan) or ContextVar proxy (Groww).
4. **Token cache** — each broker persists tokens to `.log/<broker>_tokens.json`. On startup, skips login if fresh token cached; fires full login only on miss/expiry/manual delete. **TOCTOU fix (Jul 2026)**: `get_kite_conn()` and `get_dhan_conn()` fast-path returns inside `self._login_lock` to prevent stale handle races during concurrent token refresh.
5. **Registry factories** — use `get_broker(account)` for operator actions, `get_price_broker()` for shared market data, `get_historical_brokers()` for OHLCV + regression, `get_sparkline_broker()` for KiteTicker.

6. **Capabilities immutable** — frozen dataclass with every field explicit per broker (no defaults). Used to render warning chips on OrderTicket when template asks for unsupported GTT shape.

PriceBroker fallback chain — when a broker returns empty data (Dhan returns `{}` for MCX quotes by design), walk to the next broker. Rate-limit cool-off excludes throttled accounts for 30s.

Capability matrix — verbatim from `backend/brokers/capabilities.py`:

```
KITE_CAPS = BrokerCapabilities(
    broker_id="kite", display_name="Kite (Zerodha)",
    gtt_single=True, gtt_oco=True, gtt_modify=True,
    gtt_cap_per_account=50, gtt_validity_days=365, gtt_supports_mcx=True,
    bracket_order=False, cover_order=True, atomic_basket=True,
    order_tag=True, margin_preview=True,
    postback_gtt="reliable", rate_limit_orders_sec=10,
)
DHAN_CAPS = BrokerCapabilities(
    broker_id="dhan", display_name="Dhan",
    gtt_single=True, gtt_oco=True, gtt_modify=True,
    gtt_cap_per_account=50, gtt_validity_days=365, gtt_supports_mcx=False,
    bracket_order=True, cover_order=True, atomic_basket=True,
    order_tag=True, margin_preview=True,
    # Audit fix — no Dhan WebSocket / GTT-fire postback handler is wired
    # in the codebase today; detection is the poll-based _task_oco_pair_watcher
    postback_gtt="poll_only",
    rate_limit_orders_sec=20,
)
GROWW_CAPS = BrokerCapabilities(
    broker_id="groww", display_name="Groww",
    gtt_single=True, gtt_oco=False, gtt_modify=True,
    gtt_cap_per_account=25, gtt_validity_days=90, gtt_supports_mcx=False,
    bracket_order=False, cover_order=False, atomic_basket=False,
    order_tag=False,        # No native tag; broker_order_link sidecar covers it
    margin_preview=False,
    postback_gtt="poll_only",
    rate_limit_orders_sec=5,
)
```

OCO emulation for Groww (no `gtt_oco`) is implemented in `groww.py::place_gtt` via two single-trigger GTTs + the `_task_oco_pair_watcher` background task that cancels the surviving sibling when one fires. There's no `gtt_emulated_oco` flag; emulation is an implementation detail behind the adapter.

`CAPS_BY_BROKER_ID` maps both the canonical `"zerodha_kite"` and the legacy `"kite"` alias to `KITE_CAPS`, so older YAML-seeded rows keep resolving without a column rewrite.

Frontend reads via `GET /api/admin/brokers/{account}/capabilities` (in-memory, no broker call). UI helper `brokerCapWarnings.js` consults the matrix to warn the operator at submit time when a template requests a feature the broker can't provide natively.

14.5.9 Per-broker quirks worth knowing

These are documented inline in code, but listed here for orientation:

Broker	Quirk	Where it's handled
Kite	`tag` is 20-char max	`_truncate_tag` in `backend/brokers/adapters/kite.py`
Kite	Postback HMAC validation	`order_postback` route checksum check
Kite	Rate-limited historical_data quota (low per-second budget)	`_RATE_LIMIT_COOLOFF` 30s window in registry — `_RATE_LIMIT_COOLOFF_SECONDS = 30`
Kite	One-IP-per-app rule	`_IPv6SourceAdapter` mount
Dhan	Token dashboard validity defaults to 5min	Operator must extend to 24h in Dhan dashboard
Dhan	`ltp()` returns `{}` for MCX commodity by design	PriceBroker fallback + B-2 fix logs the empty response
Dhan	`modify_gtt` needs TWO calls (ENTRY_LEG + TARGET_LEG)	Sprint C dispatch in `dhan.py::modify_gtt`
Dhan	One-active-token-per-app-per-IP rule	`_IPv6SourceAdapter` + multi-account stabilizer in `Connections`
Groww	Module-level `requests` calls with no session hook	ContextVar monkey-patch (see §14.5.5)
Groww	No native OCO	Emulated via two single GTTs + `_task_oco_pair_watcher` cancellation of survivor
Groww	`cancel_gtt` needs exchange (numeric id collision risk)	M-4 fix raises if exchange missing
Groww	No `historical_data` support	`historical_data=False` cap; sparkline + chart endpoints fall over to Kite

14.5.9.5 BSE ticker subscription and NSE→BSE equity token fallback

50-cap fix — `_perf_subscribe_book_symbols` used a hard `[:50]` slice that truncated the unresolved-symbols list. BSE holdings appearing after the 50th NSE/NFO entry were never subscribed. Fix: replaced with chunked loop (`CHUNK=50`) that covers ALL unresolved symbols.

NSE→BSE equity companion — for equity exchanges (NSE/BSE), `quote.py:_resolve_token_for_sym` now pairs companions immediately instead of walking derivatives first. Walk order:

Exchange	Token walk order
NSE	`[NSE, BSE, MCX, CDS, NFO, BFO]`
BSE	`[BSE, NSE, MCX, CDS, NFO, BFO]`
MCX/CDS/NFO/BFO	Check that exchange first, no companion pair

Same fix applied at three sites:

1. `quote.py:_resolve_token_for_sym` (main ticker lookup)
2. `batch_sparkline` token resolution
3. `_task_sparkline_warm` token lookup

BFO (BSE F&O) verification:

- ✅ In `_SPARKLINE_EXCHANGES` — subscribed
- ✅ Maps to "NSE" gate (09:15-15:30 IST)
- ✅ Route lookup via `_symbol_exchange_open` → `ctx['nse_open']`
- ✅ Kite segment mapping: BFO → "derivatives"
- ✅ Token found at index 0 in walk order

Files:

- `backend/api/background.py::_perf_subscribe_book_symbols` — chunked loop
- `backend/api/routes/quote.py::_resolve_token_for_sym` — companion pairing
- `backend/api/routes/sparkline.py` — batch + warm token lookup

14.5.10 Modifying the broker layer — guard rails

If you're touching anything in `backend/brokers/` :

- **Never branch in callers by `broker_id` .** The whole point of the abstraction is that callers don't know which vendor they're talking to. If you find yourself writing `if isinstance(broker, KiteBroker)` in a route, the right fix is a new capability flag.
- **Always re-shape vendor responses to Kite shape.** Frontend + chase + template all expect Kite shape. Skipping the normalize step silently breaks things downstream.
- **Status maps are non-negotiable.** Every vendor status must map to a Kite-canonical status (`COMPLETE` , `OPEN` , `CANCELLED` , `REJECTED` , `EXPIRED` , `TRIGGER PENDING`). A missing entry breaks chase fill detection.
- **Wrap sync SDK calls in `asyncio.to_thread` .** Adapters are sync internally; callers MUST go through `asyncio.to_thread(broker.method, ...)` in async handlers to avoid blocking the event loop.
- **Log silent failures.** B-4 audit fix: when a broker SDK returns empty data instead of raising, log `WARNING` with method + symbol + account so the failure surfaces in `api_log_file` .
- **Update `capabilities.py` first.** If you discover a vendor supports something we'd marked `False` , update the matrix BEFORE writing code that uses the feature. Operator-visible warnings flow from the matrix.

---

15. How to add a new broker

If you're integrating a new vendor (e.g. "Upstox"):

Backend

1. **Implement the adapter** in `backend/brokers/upstox.py` . Subclass `Broker` (ABC at `base.py`). Implement EVERY method — there's no "partial" mode. If a method genuinely doesn't apply, raise `NotImplementedError` with a clear message rather than returning empty.
2. **Translate to Kite shape.** Every method that returns operator-facing data (orders, positions, ltp, GTTs) must shape its return to match Kite's structure. Frontend renders are Kite-shape; downstream chase + template code expects Kite-shape. Build a `_normalise_*` helper per category. Mirror the patterns in `dhan.py` and `groww.py` .
3. **Status-string normalization.** Add `_UPSTOX_STATUS_TO_KITE = {...}` . Every Kite-canonical status (`COMPLETE` , `OPEN` , `CANCELLED` , `REJECTED` , `EXPIRED`) must map from one Upstox string. The B-1 audit lesson: a single missing entry silently breaks chase fill detection for an entire broker.

4. **Capabilities.** Add `UPSTOX_CAPS` in `capabilities.py` with EVERY field set explicitly. Don't rely on dataclass defaults — being explicit makes capability gaps visible at code review.
5. **Register in `registry.py`.** Add to `_ADAPTERS` map + `CAPS_BY_BROKER_ID`.
6. **Token caching.** Each broker has its own `.log/<broker>_tokens.json`. Follow the connection wrapper pattern in `connections.py`.
7. **Tests.** Add `backend/tests/test_upstox_broker.py`. Mock the vendor SDK at the boundary; assert your `_normalise_*` outputs.

Frontend

8. **No frontend code change needed.** The `BrokerCapabilities` dataclass is the contract; the cap warning helper at `brokerCapWarnings.js` reads it generically. Operator-visible capabilities surface automatically.

---

16. Broker gotchas

Documented so you don't relearn them the hard way:

Gotcha	Bit us in
Postback arrives before broker_order_id is committed	Race window between AlgoOrder pre-persist + seed-broker_id second commit. Fix: fallback recent-NULL-id match (C-3)
Cumulative vs delta in status polls	Every broker reports `filled_quantity` cumulatively. Pre-fix we added the cumulative value each call → inflation across restarts (C-1)
Kill recorded against old broker_order_id	Cancel-and-replace creates a new id; kill was only checked against old. Operator's kill silently ignored (C-2)
WeakValueDictionary GC during await	`_TEMPLATE_ATTACH_LOCKS` could be GC'd between mint and acquire. Fix: strong dict with TTL (M-5). 1h chosen because longest realistic live-chase window is ~30 min; 1h is 2× headroom
Reconcile attach BEFORE commit	Attach pipeline opened its own session and read pre-commit state. Fix: defer to after commit (C-4 single + bulk)
Empty `_normalise_orders` status map	Groww's "EXECUTED" passed through verbatim; chase loop never saw "COMPLETE" → no fill detection (B-1)
Dhan `ltp()` returned `{}` by design until B-2. Trail stop silently dead — no log, no Telegram, just zero ratchet	
Groww `cancel_gtt` blind segment fallback	Could cancel wrong GTT on numeric id collision. Now raises if exchange missing (M-4)
Naive `datetime.now()` in DB writes	Mix with tz-aware columns → "AT TIME ZONE" errors. Always `datetime.now(timezone.utc)`
Kite's `tag` is 20-char max	We truncate via `_truncate_tag` in `chase.py`
Trail-stop persistence missing `current_trigger`	Asymmetric SELL guard short-circuits with `0`. Every trail-write path must seed (Sc.5a)
OCO double-fire 15s window	`oco_pair_poll_seconds` default. Matches the `poll_only` GTT detection lag operator sees in the cap warning chip. Telegram alert fires on detection (H-8)
60-second postback fallback window	Long enough to cover the slowest IOC fill + DB commit race; short enough to avoid cross-pollination with new orders (C-3)
Adapter `[MARGIN-RAW-*]` logs are conn_service-only	`dhan.py` / `groww.py` run inside `ramboq_conn`, which dev pushes never restart (`webhook/deploy.sh`, `CONN_TOUCHED` is prod-only). Those lines appear on dev only after a prod deploy touching `backend/brokers/`. Use `[MARGIN-RAW-API]` (logged from `run_preflight`, API process) for dev. See §22.31
Dhan/Groww margin gate was inert	Flat `{"total": ...}` response matched none of the keys the parser read, so `required` was always 0. Fixed in §22.31

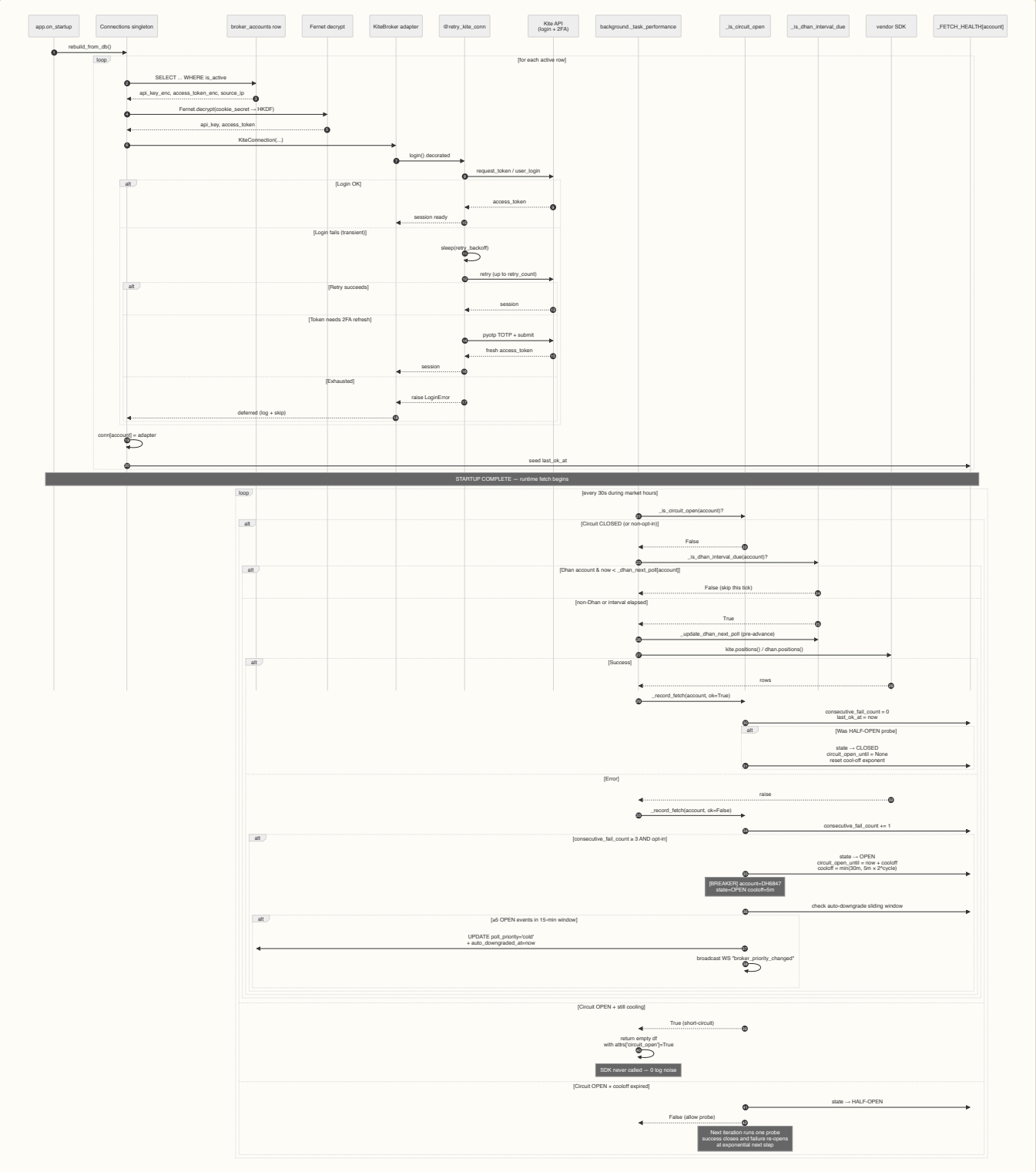
16.1 Connection retries per account — login + circuit breaker

Startup `Connections.rebuild_from_db()` iterates `broker_accounts` rows. Per account: decrypt secrets (Fernet), login via adapter, on failure the `@retry_kite_conn` decorator retries with backoff. If token refresh is needed, the 2FA flow re-mints. Post-startup, the runtime fetch path is guarded by two independent gates on `broker_apis._fetch_*_local`:

1. **Circuit breaker** (`_is_circuit_open`, opt-in per row via `circuit_breaker_enabled`) — CLOSED → normal fetch (increments `consecutive_fail_count` on error) → OPEN after 3 consecutive fails (cool-off 5m → 10m → 20m → 30m exponential) → HALF-OPEN probe → back to CLOSED on success.
2. **Dhan interval gate** (`_dhan_next_poll[account]`) — advances the next-poll timestamp BEFORE the fetch runs, so a crash mid-fetch doesn't cause a tight-retry loop. Non-Dhan accounts always pass.

3. **Dhan staggered pre-warm** — per-account delay to prevent simultaneous token refresh races.
4. **Cooloff state persistence** — circuit-breaker cooloff survives process restarts (stored to disk/DB, not in-memory only).

Auto-downgrade: when a Dhan account with both `circuit_breaker_enabled=True` AND `auto_downgrade_enabled=True` hits ≥5 breaker-OPEN events inside a 15-min sliding window, `poll_priority` flips `hot → cold` and a 5-min cooloff prevents re-firing.



Key gates & timings:

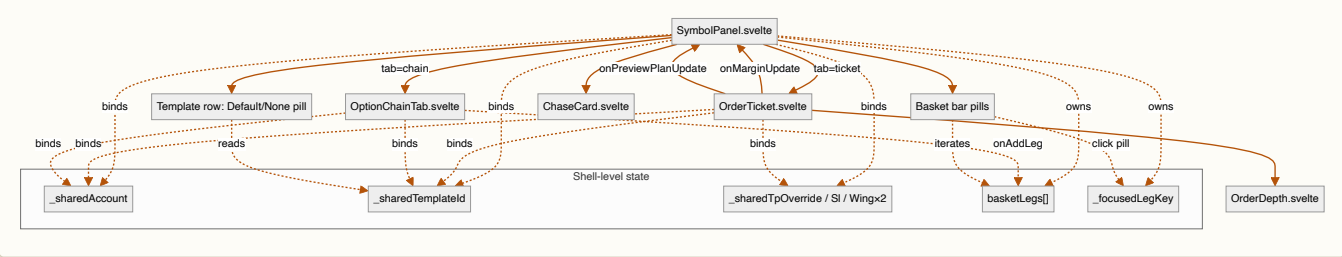
Gate	File	Behaviour
`@retry_kite_conn`	`backend/shared/helpers/decorators.py`	Login retries with backoff; falls into 2FA flow on token expiry
`_is_circuit_open(account)`	`backend/brokers/broker_apis.py`	Fast in-process dict lookup; O(1); non-opt-in accounts always return False
`_record_fetch(account, ok, error)`	same file	Increments/resets `consecutive_fail_count`, opens/closes circuit, evaluates auto-downgrade window
`_is_dhan_interval_due(account)`	same file	Dhan-only; checks `now >= _dhan_next_poll[account]`
`_update_dhan_next_poll(account, broker)`	same file	Pre-advances timestamp BEFORE fetch (crash-safe)
Cool-off schedule	same file	5m → 10m → 20m → 30m exponential (cap 30m); resets on successful HALF-OPEN probe
Auto-downgrade sliding window	same file	5 OPEN events in 15 min → `poll_priority='cold'`; 5-min re-fire cooloff
Restore endpoint	`POST /api/admin/brokers/{id}/restore-priority`	Operator resets to 'hot', clears stamps

Surfaces:

- `/api/admin/broker-health` returns `circuit_state`, `consecutive_fail_count`, `circuit_open_until`, `circuit_breaker_enabled`, `poll_priority`, `auto_downgraded_at`, `auto_downgrade_reason` per account.
- `BrokerHealthBadge.svelte` shows OPEN chip + "circuit open until HH:MM" tooltip for opt-in accounts; non-opt-in red badges show "retrying every poll".

Part V — Frontend

17. Frontend modal state



Key files:

- `frontend/src/lib/SymbolPanel.svelte` — shell + Template row + basket bar + chase card mount
- `frontend/src/lib/order/OrderTicket.svelte` — Ticket form + depth ladder + margin preview
- `frontend/src/lib/order/OptionChainTab.svelte` — strike grid + futures + chain quotes
- `frontend/src/lib/order/OrderDepth.svelte` — bid/ask depth (visibility-gated polling)

Preview chip swap rule (Chain tab):

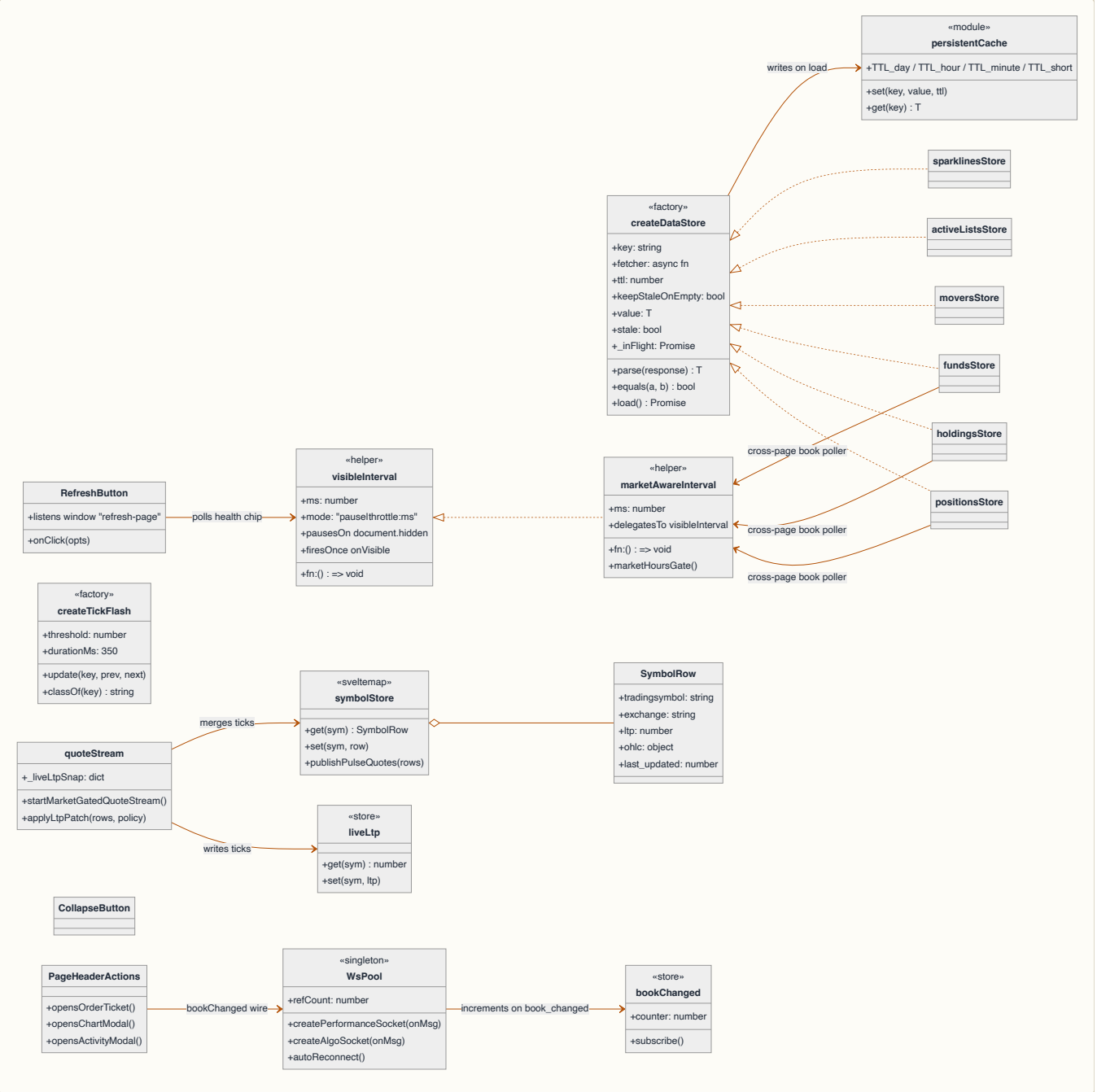
- `basketLegs.length === 0` → Ticket-form preview
- `basketLegs.length > 0` + no focus → last-leg preview
- `_focusedLegKey != null` → that specific leg's preview, badge shows `LEG N/M` ●
- Click any basket pill → set `_focusedLegKey`
- Click chip itself → cycle to next leg
- Operator × on focused leg → key clears, falls back to last-leg

⚙ **TECH — Svelte 5 `$bindable()` props** — `WHY` Two-way sync without prop-drilling or a global store. `WHAT` Child component declares `let { templateId = $bindable(null) } = $props()`; parent writes `bind:templateId={_sharedTemplateId}`. `HOW` Mutations on either side propagate. Avoid `bind:` for derived values (use a `$derived` instead). `WHERE` `SymbolPanel.svelte` ↔ `OrderTicket.svelte` ↔ `OptionChainTab.svelte` template + account props.

18. Frontend state architecture

18.0 Frontend runtime object graph

The class diagram below covers the load-bearing frontend runtime — the data-store factory, the singleton WebSocket pool, the polling helpers that hibernate on tab-hidden, and the tick-flash / symbol-store primitives that surface live prices in ag-Grid cells.



Grep map — every symbol above resolves to a single file:

Symbol	Module
`createDataStore` factory	`frontend/src/lib/data/dataStore.svelte.js`
`positionsStore` / `holdingsStore` / `fundsStore` / `moversStore` / `activeListsStore` / `sparklinesStore`	`frontend/src/lib/data/marketDataStores.svelte.js`
`positionsDayPnlStore`	`frontend/src/lib/data/positionsDayPnlStore.svelte.js`
`persistentCache`	`frontend/src/lib/data/persistentCache.js`
`symbolStore` + `publishPulseQuotes`	`frontend/src/lib/data/symbolStore.svelte.js`
`liveLtp` + `quoteStream`	`frontend/src/lib/data/quoteStream.js`
`visibleInterval` + `marketAwareInterval`	`frontend/src/lib/stores.js`
`createTickFlash`	`frontend/src/lib/data/tickFlash.svelte.js`
`createPerformanceSocket` / `createAlgoSocket` (WsPool)	`frontend/src/lib/ws.js`
`bookChanged` counter	`frontend/src/lib/stores.js`
`RefreshButton` / `CollapseButton` / `PageHeaderActions`	`frontend/src/lib/*.svelte`

18.0a positionsDayPnlStore — backward-compat shim over portfolioStore

`frontend/src/lib/data/positionsDayPnlStore.svelte.js` is now a thin backward-compat shim, not an independent aggregator — all computation lives in `portfolioStore.svelte.js` (`portfolioStore.positions`), the real SSOT. The shim exists purely so pre-existing consumers (NavCard, NavBreakdown, MarketPulse) need no import/call-site changes.

Shape (unchanged from the caller's perspective):

- `.total` → `portfolioStore.positions.total.day_pnl`
- `.byKey[sym]` → `portfolioStore.positions.byKey[sym]?.day_pnl ?? 0` (via a `Proxy` , so `Object.entries` / `Object.keys` on `.byKey` still work for callers that iterate it)
- `.byAccount` → `portfolioStore.positions.byAccount`
- `.setFromPulse()` → no-op (Pulse no longer pushes overrides; the store is the sole SSOT)

Poll-only, no live-tick delta (§1 redesign): `portfolioStore` 's underlying computation (`baseDayPnlForPosition`) is purely poll-driven (5s foreground / 30s background book-poll cadence) — `livePositionDayPnl` and its SSE-tick delta term were removed entirely, not wrapped or superseded. There is no 4Hz `symbolTickCount` -driven throttle in this store any more; positions Day P&L recomputes only when the underlying `positionsStore.value` (poll/fill) actually changes.

Consuming layers:

- **PositionStrip P pill (slot 1)**: NavStrip intraday P&L badge
- **MarketPulse per-row override**: Individual position Day P&L in the grid
- **Dashboard hero P&L**: Top-of-page portfolio summary

Consistency guarantee: By sharing a single computed value across NavStrip + MarketPulse + Dashboard, the operator sees a single canonical intraday P&L figure without drift between surfaces (compare to pre-fix where each surface computed independently).

Files:

- `frontend/src/lib/data/positionsDayPnlStore.svelte.js` — backward-compat shim
- `frontend/src/lib/data/portfolioStore.svelte.js` — real SSOT (`_posTier2` 's `baseDayPnlForPosition` call)
- `frontend/src/lib/data/nav.js` — `baseDayPnlForPosition()` computation
- Callers: PositionStrip, MarketPulse, Dashboard

18.1 Why no global store for order state?

Svelte stores would be the obvious pattern but we don't use them for order modal state. Reasons:

- **Modals are short-lived.** The operator opens, fills, submits, closes. State outlives a single modal mount maybe 5% of the time (basket persists across tab flips).
- **Component-local state with bindable props is enough.** `bind:value` on Svelte 5 runes provides bidirectional sync without the boilerplate.
- **One modal at a time.** We don't need a global "current order context" — the modal owns its context.

The exceptions: `executionMode` (navbar drives every page), `authStore` (every page), `dataCache` (PositionStrip + dashboards share), `orderTemplatesStore` (template CRUD broadcast). These are all narrow — they don't carry order-specific state.

18.2 The "shell" pattern

`SymbolPanel.svelte` is a shell. It owns:

- Header (account + symbol pickers)
- Tabs (Ticket / Chain)
- Template row (Default/None pill + override inputs + preview chip)
- Basket bar (when basket has legs)
- Common action footer (margin chip + Submit)

The actual tab content (`OrderTicket.svelte` , `OptionChainTab.svelte`) is mounted as a child. State pipes through:

- **Down via props:** shell → tab (e.g. `_sharedAccount` → `account` prop)
- **Up via callbacks:** tab → shell (e.g. `onMarginUpdate` , `onPreviewPlanUpdate`)
- **Two-way via `bind:` :** for shared mutable state (e.g. `bind:templateId={_sharedTemplateId}`)

When you add a new piece of shell-visible state, decide once:

- Is it tab-specific? → Stay in the tab component.
- Should it survive tab flips? → Lift to shell.
- Does any tab need to READ it? → Pipe down via prop.
- Does any tab need to WRITE it? → `bind:` it.

18.3 Frontend column factory SSOT — pulseColumns.js

`frontend/src/lib/data/pulseColumns.js` is the **single source of truth** for ag-Grid column definitions across the platform. Factory functions replace inline column specs, enabling consistent styling and behaviour across surfaces. New factories added this session:

Factory	Returns	Used by
`mkWeightPctCol()`	Column spec for portfolio weight %	PerformancePage, Dashboard
`mkDeltaCol()`	Column spec for P&L delta	PerformancePage
`mkThetaCol()`	Column spec for option theta	/admin/derivatives
`mkNavBreakdownCols(segments)`	Array of NAV breakdown columns	Dashboard NavBreakdown card
`mkUtilPctCol()`	Column spec for margin utilization %	/admin/funds
`mkFundsDetailCols()`	Array of funds detail columns	/admin/history Funds tab

⚙ **TECH — Centralized column specs vs inline definitions** — `WHY` Operator brief: "weight % looks different on three pages". Inline column specs (valueFormatter, width, headerName, tooltips) scatter across components; a missed update breaks visual consistency. `WHAT` Every column that appears on multiple pages gets a factory in `pulseColumns.js` . Pages call the factory instead of defining columns inline. `HOW` Factories export immutable column objects; pages `.map()` them into the ag-Grid `columnDefs` prop. `WHERE` `frontend/src/lib/data/pulseColumns.js` ; used by PerformancePage, Dashboard, /admin/derivatives, /admin/history, /admin/funds.

Usage pattern:

```
// Before: inline spec on every page
const positionCols = [
  { headerName: "Weight %", valueFormatter: pctFmt, width: 90, ... },
  { headerName: "Delta", valueFormatter: aggFmt, width: 80, ... },
];

// After: factory-based
const positionCols = [
  mkWeightPctCol(),
  mkDeltaCol(),
];
```

Shared formatters — `frontend/src/lib/format.js` now exports `aggFmtGrid({ value })` and `pctFmtGrid({ value })` — ag-Grid-compatible wrapper functions over the existing `aggCompact` and `pctFmt` formatters. These return styled strings with the correct precision; used by column factories for MarketPulse and PerformancePage grid definitions.

Files:

- `frontend/src/lib/data/pulseColumns.js` — factory SSOT
- `frontend/src/lib/format.js` — `aggFmtGrid()` and `pctFmtGrid()` ag-Grid wrappers
- `frontend/src/lib/data/nav.js` — `aggregateDayPnlForPositions()` helper (see §19.1)
- Callers: PerformancePage, Dashboard, /admin/derivatives, /admin/history

18.4 Pure module extractions — Phase 1

Four stateless JavaScript modules ship core logic isolated from component tree:

Module	Exports	Used by
`riskMath.js`	`normCdf`, `probAbove`, `expectedValueOnCurve`, `multilegPopOnCurve`, `RISK_FREE_R=0.07`	Derivatives page, `/admin/derivatives`
`templateScope.js`	`appliesToFor(side, sym)` → `'sell_option'|'sell_any'|'buy_option'|'buy_any'|'both'`	OrderTicket, SymbolPanel
`pulseGridSetup.js`	`PULSE_DEFAULT_COL_DEF`, `PULSE_SORTING_ORDER`, `pulseRowId`, `summaryRowId`, `postSortGroups`	MarketPulse (PositionStrip)
`chart/paths.js`	`smaPath`, `emaPath`, `vwapPath`, `bbPaths`, `rsiSeries`, `macdSeries`	ChartWorkspace (overlays + indicators)

Design pattern: each module is a zero-dependency utility collection; no props, no svelte imports, no stores. Enables unit-test coverage + reuse across unrelated surfaces.

Files:

- `frontend/src/lib/data/riskMath.js` — Black-Scholes, normal CDF, payoff integration
- `frontend/src/lib/data/templateScope.js` — symbol-side decision tree for default-template pickup
- `frontend/src/lib/data/pulseGridSetup.js` — ag-Grid sorting + row ID scheme + column definitions
- `frontend/src/lib/chart/paths.js` — SVG path builders for overlays + indicators

18.5 Svelte component extractions — Phase 2 + Phase 3

Phase 2 (small, light):

Component	Props	Used by
`ChaseAggPicker.svelte`	`variant='ticket'|'panel'` + value/onChange	OrderTicket (variant=ticket, default), SymbolPanel (variant=panel)
`OhlcvTooltip.svelte`	`bar, pxLeft, pxTop, pinned, onClose`	ChartWorkspace (candlestick hover)
`TickTooltip.svelte`	`tick, pxLeft, pxTop, pinned, onClose`	ChartWorkspace (live-tick overlay hover)

Phase 3 (large, Playwright-gated):

Component	Lines	Props	Used by
`OrderKnobsRow.svelte`	320	order_type, product, variety, validity (all `$bindable()`)	OrderTicket (modal)
`TemplateBar.svelte`	280	tp_pct_override, sl_pct_override, wing_qty_override, wing_leg_override (all `$bindable()`) + `sideAwareDefault` prop	SymbolPanel (shell Template row)
`CandidateLegRow.svelte` (derivatives)	841	`c`, `legsTab`, `legAnalytics` + 6 callback props	`/admin/derivatives` page, legs grid
`AddToPulseModal.svelte`	380	13 `$bindable()` state vars + 7 async callbacks	MarketPulse (add-to-watchlist)
`CardHeader.svelte`	200	`title`, `showControls`, `onDownload`, `variant` props + CSS custom property theming	9 card header sites (NavBreakdown, performance, derivatives, etc.)

Svelte 5 patterns:

- **`$bindable()` two-way:** Child declares `let { fieldName = $bindable(default) } = $props()`; parent writes `bind:fieldName={parent_var}`. Mutations sync both directions atomically.
- **Callback forwarding:** Large components expose callback props for state changes; parent wires them to stores or sibling updates. Example: `CandidateLegRow` → `onLegAnalyticsChange(newVal)` → parent `legsTab.analytics[idx] = newVal`.

Files:

- `frontend/src/lib/order/ChaseAggPicker.svelte` — variant-driven L/M/H aggression UI
- `frontend/src/lib/chart/OhlcvTooltip.svelte` + `TickTooltip.svelte` — anchored chart tooltips
- `frontend/src/lib/order/OrderKnobsRow.svelte` — extracted from OrderTicket, unified knob layout
- `frontend/src/lib/TemplateBar.svelte` — extracted from SymbolPanel, override row + Default button state
- `frontend/src/routes/(algo)/admin/derivatives/CandidateLegRow.svelte` — 841-line leg table row, internally isolated
- `frontend/src/lib/AddToPulseModal.svelte` — watchlist add-to modal, async symbol search + confirm
- `frontend/src/lib/CardHeader.svelte` — unified card title bar, CSS var theming, embedded controls

CardHeader theming: Uses CSS custom properties (`--ch-*`) set at layout level (algo-dark in `+layout.svelte`, public-light in public viewport). `showControls` prop gates the right-side icon tray; `onDownload` callback wires to `NavBreakdown.downloadCsv()` via a named method exposed on the component. Embeds `CardControls` component internally.

Bug fixes bundled:

- `GridDownloadButton.svelte` — added `autoMargin` prop (default `true`); `CardControls` passes `autoMargin={false}` so fullscreen layout spacer handles alignment instead.
- `NavBreakdown.svelte` — exposes `downloadCsv()` method; dashboard wires it through `CardHeader.onDownload`.

---

19. The preview pipeline

The on-fill preview chip (`on fill → TP ₹250 / SL ₹180 / + Wing BUY ...`) is the single most useful piece of context at submit time. It's computed via two independent pipelines:

- **OrderTicket's `_previewPlan`** ← computed against the Ticket form's symbol/side/qty/price/template.
- **SymbolPanel's `_lastLegPlan`** ← computed against the last basket leg (or operator-focused leg).

The chip render switches between them based on `_activeTab === 'chain' && basketLegs.length > 0`. **Why two pipelines?** Because the inputs are different:

- Ticket: form state, not yet a "leg"
- Last-leg: a fully-formed leg with its own account + symbol + overrides

Both call the same backend endpoint (`previewTicketTemplate`) with the same payload shape. The frontend just feeds them differently.

⚙ **TECH — Backend preview endpoint vs frontend simulation** — `WHY` Operators trust ₹ values that come from the same code path that will ACTUALLY fire on fill. Computing them in the frontend would risk drift; computing them in the backend guarantees the chip reflects reality. `WHAT` `POST /api/orders/preview-ticket-template` returns `{plan: {gtts: [...], wing: {...}, notes: [...]}}`. `HOW` Frontend debounces 200ms after any override change; backend runs `resolve_template_plan` with `apply_path="preview"` so no broker calls fire. `WHERE` `backend/api/routes/orders.py::preview_ticket_template`.

19.1 aggregateDayPnlForPositions helper

`frontend/src/lib/data/nav.js::aggregateDayPnlForPositions(rows)` reduces position rows using `baseDayPnlForPosition(r)` per row. Used wherever total Day P&L across all positions must be summed (e.g. PerformancePage TOTAL row, Dashboard hero, NavStrip P pill).

```
export function aggregateDayPnlForPositions(rows) {
  if (!rows || rows.length === 0) return 0;
  return rows.reduce((sum, r) => sum + baseDayPnlForPosition(r), 0);
}
```

Why not `sum(day_change_val)`? The `day_change_val` field omits Kite's edge cases (new positions, fully-flat intraday closes). `baseDayPnlForPosition(r)` applies the override: when `r.overnight_quantity === 0 && r.pnl !== 0`, use `r.pnl` instead. Aggregating with the helper ensures the total matches the sum of individual per-cell Day P&L values.

Callers:

- PerformancePage TOTAL row aggregation
- Dashboard hero P&L + position summary
- NavStrip P pill slot 1

Files:

- `frontend/src/lib/data/nav.js::aggregateDayPnlForPositions` + `baseDayPnlForPosition`

19.2 CSS design tokens — palette + background variants

Canonical CSS custom properties live in `frontend/src/app.css`. These tokens establish the visual language across the platform — colors, opacities, spacing. All UI changes MUST respect these tokens to maintain consistency.

Core colour palette

Token	Hex	Usage	Notes
`--algo-green`	`#4ade80`	Buy signals, profit, long positions	emerald-400
`--algo-red`	`#f87171`	Sell signals, loss, short positions	red-400
`--algo-cyan`	`#06b6d4`	Neutral data, UI accents	cyan-500
`--algo-amber`	`#fbbf24`	Warnings, TOTAL row highlight	amber-400
`--algo-blue`	`#3b82f6`	Secondary UI elements	blue-500

Background variants (opacity layering)

Token	Formula	Usage	Notes
`--algo-green-bg-mid`	`rgba(74, 222, 128, 0.14)`	Mid-intensity buy backgrounds	14% opacity for subtle pill/card backgrounds
`--algo-red-bg-mid`	`rgba(248, 113, 113, 0.14)`	Mid-intensity loss backgrounds	14% opacity for subtle pill/card backgrounds
`--c-long-14`	alias to `--algo-green-bg-mid`	Long position basket pill backgrounds	semantic alias for clarity
`--c-short-14`	alias to `--algo-red-bg-mid`	Short position basket pill backgrounds	semantic alias for clarity
`--algo-blue-tint`	`#f1f7ff`	Panel highlight tint	very light blue, used for card backgrounds

Token sourcing

All tokens are defined centrally in `frontend/src/app.css` and consumed via CSS classes or inline `var()` references. **Never hardcode colours inline** — always use a token. If a new colour is needed:

1. Add the token to `app.css` with a clear semantic name
2. Export it as a CSS custom property (`:root { --token-name: value }`)
3. Reference via `var(--token-name)` or tie to a `.class` utility

Panel backgrounds (dark theme)

- **Card body**: `#1d2a44` (dark slate, primary)
- **Card hover**: `#242f47` (slightly lighter)
- **Input background**: `#0f172a` (very dark)
- **Code/monospace**: `#1e293b` (slate-800)

Text contrast

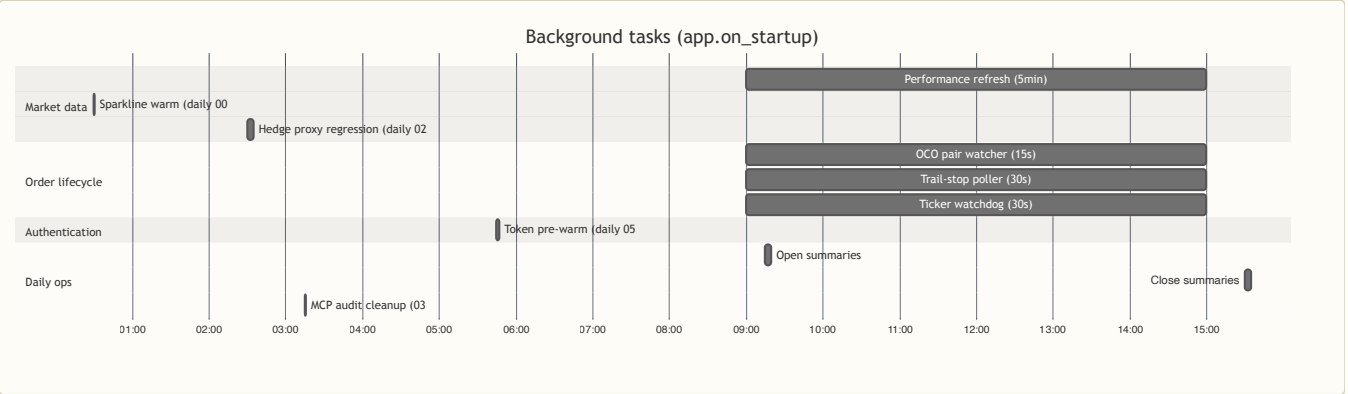
- **Primary text**: `#e2e8f0` (slate-200)
- **Secondary text**: `#94a3b8` (slate-400)
- **Disabled text**: `#64748b` (slate-500, low contrast)
- **Action text** (`--c-action`): `#fbbf24` (amber-400, matching TOTAL rows)

Files:

- `frontend/src/app.css` — CSS custom properties `:root` scope + theme utilities
- All consuming components: inline `var(--token-name)` or `.class-using-token`

Part VI — Runtime

20. Background task topology



Key files:

- `backend/api/background.py` — all task definitions
- `backend/api/app.py::on_startup` — spawn list + initialization sequence (see §4.13 `seed_and_warm`)

Tasks that touch operator orders:

- `_task_performance` (5min) — fetches positions/holdings/funds; runs `agent_engine.run_cycle`
- `_task_oco_pair_watcher` (5s) — Groww emulated OCO sibling cancel
- `_task_trail_stop` (30s) — Dhan + Kite trail SL ratchet
- `_task_open_order_watchdog` (5min) — reconcile stale OPEN live orders; fire template attach on fills
- `_task_ticker_watchdog` (30s) — KiteTicker reconnect on disconnect

Tasks that update closed-hours snapshots:

- `_task_closed_hours_refresh` (30min, post-market-close) — persists fresh broker data to `daily_book` after settlement window; invalidates live-data caches so closed-hours routes serve updated snapshots (positions, holdings, funds, trades)
- `_snapshot_probe_nse_mcx` (polling, triggered at snapshot times) — replaces hardcoded time checks. Calls `exchange_clock.sessions_with_snapshot_time_now()` to find which gates have snapshot_time in the next minute, then dispatches `trigger_close_snapshot(gate)` or `trigger_settlement_capture(gate)` accordingly (see §4.13)
- `_task_chain_instruments` (T+30s, daily 08:02 IST) — fetches NFO/MCX contracts only; populates `instruments_chain` cache for option-chain tab quote endpoint

Token lifecycle tasks:

- `_task_token_refresh` (05:45 IST daily, parked under `conn_service` guard) — fires before the 06:00 IST Kite token expiry window; calls `get_kite_conn(test_conn=True)` for all Kite accounts to pre-warm tokens and detect auth issues early. Prevents token-expiry 401 errors during market hours. When `RAMBOQ_USE_CONN_SERVICE=1`, the task is not scheduled to avoid tight-loop polling when the conn service is not running; skipped at startup.

Schedule refresh tasks:

- 04:00 IST daily — calls `exchange_clock.load_today_open_time()` (before holiday refresh) to set dynamic NSE open time from `exchange_schedule` table. The fix fires at the actual session open time (which may differ on special sessions) and does not fire on holidays. The holiday calendar sync (`_task_holiday_refresh`) is a separate job that runs at 05:30 IST (`holiday_refresh_time`), not in this 04:00 slot.

⚙ **TECH — Why poll-based + not event-based** — `WHY` Vendor postbacks are unreliable (Dhan + Groww have no inbound webhook; Kite drops 0.5-2% in our experience). Polling is the conservative floor. `WHAT` Each task runs on its own asyncio cadence; no scheduler library. `HOW` Pick interval based on operator latency tolerance: trail-stop = 30s (slow ratchet OK), OCO watcher = 15s (faster because both legs settling within window means double-fire). `WHERE` `backend/api/background.py`.

20.1 Sparkline refresh pipeline

`_task_sparkline_warm` (startup + 00:30 IST + segment opens) populates KiteTicker subscription universe from three sources:

1. **Daily book union** — all positions + holdings from past 7 days (backstop, survives conn_service restart)
2. **Watchlist symbols** — from `WatchlistItem` table (TRADES, key: `tradingsymbol` + `exchange`)
3. **Virtual root resolution** — MCX/CDS symbols resolved to active contract (e.g. `CRUDEOIL` → `CRUDEOIL26JULFUT`)

⚙ **TECH — Delayed startup warm via `_spark_delayed_startup()`** — `WHY` Immediate startup warm downloads 6 exchanges (NFO ~70k instruments) at T=0; if token count is 0, retries at T+60s — combined RSS reached 5-6GB before port 8000 bound, causing OOM kill loop. `WHAT` `_spark_delayed_startup()` wraps the task entry with `await asyncio.sleep(600)` (10 minutes, configurable `sparkline.startup_delay_seconds`). Application starts, port 8000 binds, then after the delay window the sparkline fetch begins with breathing room. `HOW` At startup, `_task_sparkline_warm` is dispatched immediately; it sleeps first. At 00:30 IST and segment opens, the fetch runs immediately (no delay). `WHERE` `backend/api/background.py:_spark_delayed_startup`, `on_startup` wiring in `backend/api/app.py`.

Three-part defect fix:

- **MCX/CDS watchlist symbols get correct exchange** — `_sparkline_universe_symbols` now queries `(tradingsymbol, exchange)` pair instead of loose `tradingsymbol` lookup
- **Virtual roots resolved before OHLCV fetch** — `snapshot_sparkline` resolves MCX/CDS virtual roots via `symbol_resolver.resolve_symbol()` before calling `ohlcv_store.get_or_fetch_daily`
- **Tier 4 fallback** — `batch_sparkline` added fallback to read from `daily_book WHERE kind='sparkline'` when ohlcv_store is cold (db_only mode)

Stale-better merge — frontend `_mergeSparkSeries` keeps cached real curve over fresh flat/degenerate series to prevent chart collapse when broker feed lags.

Mover grace-window — `loadSparklines()` in MarketPulse carries the previous mover rotation's pairs into each new call via `_prevMoverSparkPairs`. Symbols that just left the winners/losers list get a one-rotation (30s) grace period before sparklines are pruned from cache. Symbols re-entering the top 10 show instantly from cache rather than fetching fresh.

Sparkline gradient fill — sparkline SVG cells now render a gradient area fill beneath each curve. Color is trend-aware: up-trend = teal `rgba(91,142,149,0.3)` tapering to 0% at bottom, down-trend = amber `rgba(196,122,61,0.3)`, flat = slate `rgba(126,151,184,0.3)`. SVG uses `<defs><linearGradient>` with 30% opacity at the curve tapering to 0% at the cell bottom; `<polygon>` fills the area, `<polyline>` draws the line on top. Implemented in `frontend/src/lib/components/SparklineCell.svelte`.

Files:

- `backend/api/background.py::_task_sparkline_warm`, `_spark_delayed_startup`
- `backend/api/routes/sparkline.py::snapshot_sparkline`, `batch_sparkline`
- `backend/api/algo/symbol_resolver.py::resolve_symbol`
- `frontend/src/lib/PerformancePage.svelte::_mergeSparkSeries`
- `frontend/src/lib/MarketPulse.svelte::loadSparklines` — grace-window logic + `_prevMoverSparkPairs`
- `frontend/src/lib/components/SparklineCell.svelte` — gradient fill rendering

20.1.1a MarketPulse data refresh cadences

MarketPulse (PositionStrip positions/holdings grid + movers/sparklines) coordinates five distinct refresh cadences to balance responsiveness with network load:

Cadence	Interval	Data	Triggers	Notes
Book poller	5 seconds	Positions, holdings, funds via `/api/positions` etc	`marketAwareInterval`	Live-session updates; pauses on tab-hidden; post-market becomes `_task_performance` background tick
Quotes poller	30 seconds	Symbol snapshots via `/api/quotes` + `/api/pulse/quotes`	`quoteStream.startMarketGatedQuoteStream()`	LTP + OHLC for all grid cells; SSE fallback; throttled during closed hours
Movers rotation	30 seconds	Top 10 gainers/losers via `/api/pulse/movers`	`loadMovers()` in MarketPulse	Sparkline fetch deferred until next rotation (grace window for departing symbols)
Sparklines refresh	30 seconds + grace	7-day OHLCV per mover symbol via `/api/sparkline` batch	`loadSparklines()` defer + grace window	One-rotation (30s) grace period for symbols leaving top-10 list before cache prune
Settings audit	60 seconds	Chart self-heal threshold + UI prefs via `/api/admin/settings`	`loadSettings()`	Used for indicator refresh gate + column visibility toggles

Rationalization (7 cadences → 5): Eliminated 4s (too aggressive; TCP overhead), 10s (ambiguous; split between 5s and 30s), 15s (superseded by 30s for movers/sparklines). The 5s book poller is the fastest-moving piece; 30s aligns quotes + movers + sparklines to reduce network churn.

Market-close gate: All pollers respect `isMarketOpen()`. After segment close, the 5s book poller becomes the background `_task_performance` 5min loop; front-end pollers pause. SSE tick stream continues at reduced cadence (5–15min between updates) to surface settlement changes.

Files:

- `frontend/src/lib/MarketPulse.svelte` — `loadPositions()`, `loadMovers()`, `loadSparklines()`, `visibleInterval` setup
- `frontend/src/lib/PositionStrip.svelte` — book poller 5s cadence
- `frontend/src/lib/data/quoteStream.js` — 30s quotes cadence
- `backend/api/routes/pulse.py` — `/api/pulse/movers`, `/api/pulse/quotes` endpoints

20.1.1 Instruments background task — on-demand load (not startup-warmed)

The `bg-instruments` task (option chains + MCX contracts) is intentionally **NOT** started in `on_startup`. Reason: instruments data is bulky (6 exchanges, NFO alone ~70k contracts), and immediate parallel load with sparkline warm caused OOM kill loop on 2026-08-12.

Loading model:

- **On-demand via routes** — `backend/api/routes/options.py::get_or_fetch` calls the instruments store when the operator clicks into the derivatives page
- **120s startup delay guard** — if a route accidentally triggers instruments fetch at boot (before broker session warming), the fetch uses `await asyncio.sleep(120)` to defer execution and avoid resource contention with sparkline warm and token refresh
- **No timeout set** — the `get_or_fetch` call in `options.py` has no `timeout_seconds` parameter, allowing the fetch to complete without premature cancellation even if the initial load takes 15–30s
- **Lazy benefits** — most operators never touch option chains; skipping the startup warm saves 500+MB at boot

Trade-off: First visit to `/admin/derivatives` may see a 5–15s load spinner while instruments load from broker. Subsequent visits hit the in-memory cache.

Files:

- `backend/api/routes/options.py::get_or_fetch` — on-demand load (no timeout set)
- `backend/api/algo/symbol_resolver.py::InstrumentsStore` — lazy singleton

20.2 Background task async safety — broker API calls in to_thread

`backend/api/background.py::_task_warm_backfill` and other background tasks invoke broker SDK methods (`fetch_holdings()`, `fetch_positions()`) which are synchronous. These are now wrapped with `await asyncio.to_thread(...)` to prevent blocking the event loop during backfill warm or other background operations.

Pre-fix: sync broker calls in background tasks could block asyncio schedulers, causing delayed response handling on concurrent HTTP requests.

Files:

- `backend/api/background.py::_task_warm_backfill` — to_thread wrappers on broker calls

20.3 OCO pair watcher — mutual sibling pointer cleanup

`backend/api/background.py::_task_oco_pair_watcher` now clears mutual `sibling_id` pointers when both OCO entries settle simultaneously. When entry A and entry B both reach a terminal status in the same poll cycle, `sib_entry.pop("sibling_id", None)` is called for the sibling entry (in addition to the primary), so the bidirectional reference is fully cleared in the both-settled branch.

Pre-fix: only the primary entry's sibling pointer was cleared, leaving the sibling entry with a dangling reference to a potentially stale ID.

Files:

- `backend/api/background.py::_task_oco_pair_watcher` — mutual pointer cleanup

20.4 Closed-hours snapshot refresh — post-settlement data persistence

`backend/api/background.py::_task_closed_hours_refresh` runs every 30 minutes after market close to persist fresh broker data (`positions`, `holdings`, `funds`, `trades`) into `daily_book`, then invalidates the live-data cache layer so closed-hours routes serve updated snapshots.

Why this task exists: Between segment close (~15:30 IST for NSE, 23:30 IST for MCX) and settlement completion (30–60 min later), broker systems update position close prices and realised P&L. Without this task, closed-hours API routes serve stale pre-settlement snapshots until the next market open. The operator checks NAV or positions during overnight hours and sees yesterday's settlement prices + P&L, not today's reconciled values.

Guard condition: Only runs when `is_any_segment_open()` is False. Skips the write when any segment is open (avoids polluting `daily_book` with mid-session LTPs that change every second).

Execution steps:

1. Check market open state; if any segment open, sleep and retry
2. Call `snapshot_daily_book(settled=False)` to fetch current broker state (positions, holdings, funds, trades)
3. Write/upsert to `daily_book` table via idempotent UPSERT (`ON CONFLICT ... DO UPDATE`)
4. Call `cache.invalidate_batch(['positions', 'holdings', 'funds'])` to clear the live-data route cache

5. SSE broadcast `book_changed` event so frontend re-fetches fresh data

Complementary tasks:

- `_task_funds_offhours` (30 min, post-market-close) — funds-only snapshot; subset of this task
- `_task_daily_snapshot` (16:15 IST / 00:15 IST) — settlement pass; marks rows with `settled=True`
- `_task_sparkline_warm` (startup + 00:30 IST) — refreshes ticker universe from `daily_book` past 7 days

Files:

- `backend/api/background.py::_task_closed_hours_refresh`
- `backend/api/algo/daily_snapshot.py::snapshot_daily_book` — broker data collection
- `backend/api/cache.py::invalidate_batch` — cache clear

20.5 Open order watchdog — reconcile stale OPEN orders

`backend/api/background.py::_task_open_order_watchdog` runs every 5 minutes (configurable: `orders.open_order_watchdog_seconds`, default 300) to reconcile OPEN live orders that were never chased — typically due to a service restart or broker error during order placement.

Query: Every 5 min, fetch all AlgoOrder rows where `status='OPEN'`, `mode='live'`, `created_at < 5 minutes ago` (cutoff guards against false reconcile during initial chase).

Reconcile per account: Group rows by account, then call `_rco_reconcile_account(account, rows)` for each. The function queries broker order book, compares against local state, and updates rows:

- **FILLED**: appended to attach queue for template-attach fire (take-profit / stop-loss seeder)
- **CANCELLED**: marked `status='CANCELLED'` in DB
- **UNFILLED**: no change (still OPEN; will retry on next 5-min tick)

Template attach flow: After DB commit, rows marked FILLED are passed to `_maybe_fire_template_attach_for_reconcile`, which fires GTT legs asynchronously (same path as live-postback attach).

Best-effort shape: Any iteration error is logged at WARNING and skipped; the next 5-min tick retries. A single account error doesn't block other accounts.

Files:

- `backend/api/background.py::_task_open_order_watchdog` — poller + orchestration
- `backend/api/routes/orders.py::_rco_reconcile_account` — broker reconcile + DB update
- `backend/api/routes/orders_place.py::_maybe_fire_template_attach_for_reconcile` — template fire

20.6 Market Daily Window

The platform runs a fixed 24-hour market operating cycle. Every event fires exactly once per calendar day via dedup sentinels keyed on trade-date.

24-Hour Event Table

Time (IST)	Event	Exchange	Function	File:Line
00:15	MCX settlement snapshot (`daily_book`)	MCX	`trigger_settlement_capture("MCX")` → `_snapshot_fire(label="mcx-settlement")`	background.py:1836–1847
00:30	KiteTicker full stop — reactor halt, WebSocket close, subscriptions cleared	ALL	`_snapshot_stop_ticker()`	background.py:1917–1935, 2089–2099
05:30	Holiday calendar refresh + token pre-warm (all brokers)	NSE+MCX	`_task_holiday_refresh()`	background.py `_task_holiday_refresh` (~L3997)
08:00	KiteTicker restart with fresh token; `fix_daily_book_prev_close()` is marker-guarded (`task_markers.prev_close_fix_date`) and, in normal operation, updates zero rows (2026-10-20, see "Settlement re-arm rules" below)	ALL	`_snapshot_restart_ticker()` , `fix_daily_book_prev_close()`	background.py:2059–2112
09:00	MCX regular session opens	MCX	config	backend_config.yaml
09:15	NSE/BSE/NFO/CDS regular session opens	NSE/BSE	config	backend_config.yaml
15:30	NSE regular session closes	NSE/BSE	config	backend_config.yaml
15:45	NSE close snapshot (`daily_book`)	NSE/BSE	`trigger_close_snapshot("NON-MCX")` → `_snapshot_fire(label="non-mcx-close")`	exchange_clock.py:368, background.py:1824–1833
16:15	NSE settlement snapshot + non-MCX unsubscribe from KiteTicker	NSE/BSE	`trigger_settlement_capture("NON-MCX")` , `_snapshot_unsub_nonmcx()`	background.py:1836–1847, 2078–2087
23:30	MCX regular session closes	MCX	config	backend_config.yaml
23:45	MCX close snapshot (`daily_book`)	MCX	`trigger_close_snapshot("MCX")` → `_snapshot_fire(label="mcx-close")`	exchange_clock.py:380, background.py:1824–1833

Snapshot times (15:45, 16:15, 23:45, 00:15) are stored in `exchange_schedule.snapshot_time` and are operator-configurable via `/admin/exchange-schedule` .

MCX vs Non-MCX Comparison

Dimension	NSE/BSE/NFO/CDS	MCX
Session hours	09:15–15:30	09:00–23:30
Holiday calendar	`market_holidays` WHERE `exchange='NSE'`	`market_holidays` WHERE `exchange='MCX'`
Open on NSE holidays?	No	Yes (`evening_open_on_holidays: true`), except dates the NSE feed marks evening-Closed (zero-width `market_special_sessions` rows, `source='nse_auto'` , 2026-10-20)
KiteTicker subscription	Dropped at 16:15	Held until 00:30
Close snapshot	15:45 IST	23:45 IST
Settlement snapshot	16:15 IST	00:15 IST (next calendar day)

Token Refresh Lifecycle

Kite tokens carry a 23-hour vendor TTL (`conn_reset_hours: 23` , backend_config.yaml:5). The daily cycle ensures a fresh token is always available before market open:

```
05:30 IST  → _task_holiday_refresh() pre-warms tokens for all loaded accounts
             • Kite:  get_kite_conn(test_conn=True) — validates via profile() call
             • Dhan:  get_dhan_conn(test_conn=True)
             • Groww: get_groww_conn()
             On failure → retries every 30 min until 08:00 IST
             Token cache: .log/kite_tokens.json (per-account, advisory flock)

08:00 IST  → _snapshot_restart_ticker() passes freshly validated token to KiteTicker
             Guarded [08:00, 09:00) — no double-fire on daytime service restart

Per-call   → every get_kite_conn() validates via profile(); clears cache + forces
             re-auth on 401; @retry_kite_conn handles auto-recovery

Next 05:30 → cycle repeats (23h token issued at 08:00 expires at 07:00+1; pre-warm
             at 05:30 refreshes it with 90 min headroom before expiry)
```

Holiday Calendar — 4-Tier Read Hierarchy

```
Request for holiday set (exchange, date)
    |
    ├─ Tier 1: in-process LRU cache (_MEM_CACHE)     ← zero I/O, sync dict
    |          hit: return immediately
    |
    ├─ Tier 2: module-level _HOLIDAY_CACHE (daily TTL) ← pre-DB fallback
    |          hit: return
    |
    ├─ Tier 3: PostgreSQL market_holidays table       ← durable source
    |          sources: nse_auto | operator | legacy_seed
    |          refreshed daily at 05:30 IST
    |          retry every 30 min until 08:00 on failure
    |
    └─ Tier 4: NSE public API (cold-boot only)        ← 10s timeout
               fallback when DB is empty at first startup
```

Special-session override (highest precedence): if a `market_special_sessions` row exists for (exchange, date, start_time), `is_market_open()` returns `True` during that window regardless of holiday status or regular-session config.

Operator override: `PUT /admin/exchange-schedule` with `{date, gate, is_open: false}` force-closes any date. Takes effect immediately; survives restarts.

Worked Examples

Example 1 — NSE Holiday (e.g., Republic Day 2026-01-26)

- `market_holidays` has a row for `(exchange='NSE', date=2026-01-26)`.
- No row for `(exchange='MCX', date=2026-01-26)` → MCX calendar is unaffected.
- At 09:00: `is_market_open()` returns `True` for MCX segment; KiteTicker subscribes MCX tokens.
- At 09:15: `is_market_open()` returns `False` for equity segment — NSE never opens.
- Non-MCX instruments are never subscribed to KiteTicker that day.
- MCX close snapshot fires at 23:45; MCX settlement snapshot fires at 00:15+1.
- NSE `daily_book` rows carry yesterday's settlement LTP as `ltp` all day (no new snapshot).

Example 2 — Diwali Muhurat Trading (e.g., 2026-11-01)

- Both NSE and MCX are in `market_holidays` (full holiday).
- A `market_special_sessions` row adds `(exchange='NSE', date=2026-11-01, start_time=18:00, end_time=19:00, reason='Muhurat')`.
- At 18:00: `is_market_open()` returns `True` (special-session override takes precedence); KiteTicker subscriptions resume.
- At 19:00: `is_market_open()` returns `False`; displays freeze with `as_of=19:00` stamp.
- Snapshot fires at the special session's `snapshot_time` (operator-set to 19:15 in `exchange_schedule`).
- MCX is also governed by any MCX-specific `market_special_sessions` row; if none, MCX follows its holiday rule (`evening_open_on_holidays: true` → opens at 17:00 for the evening session as normal).

Settlement re-arm rules and restart hydration (2026-10-20)

`MarketLifecycle` (`market_lifecycle.py`) fires `<exch>:open`, `<exch>:close`, and `<exch>:close_settled` (~15 min after close). Each close produces one `close_settled`, which triggers `snapshot_daily_book(settled=True)`.

- **No date-rollover reset.** `_settled_fired` used to clear on every calendar-date rollover. For an exchange already closed and settled, `_last_close_at` survives midnight, so the settle window was already satisfied and `close_settled` fired a second time. That write was dated the next calendar day, and its `prev_close` equalled `ltp`. The rollover reset is deleted (`market_lifecycle.py` comment block around line 158). `_last_seen_date` is removed.

- **Re-arm points.** `_settled_fired` clears on the open transition (line ~203) and on the close transition (line ~229, added 2026-10-20 so a flag hydrated from the prior cycle cannot suppress the current cycle's settle).
- **Restart hydration.** `_hydrate_from_audit()` (line ~315) runs once per process and rebuilds `_last_close_at` and `_settled_fired` from `MarketLifecycleEvent` rows (26h lookback). An already elapsed settle window is marked handled, never fired late. Without hydration, a restart inside `[last_close, last_close + settled_offset)` would silently never fire that cycle's settle.
- **Session-boundary date.** `snapshot_daily_book(target_date=None)` defaults to `session_date_of(now_ist)` (`expiry_freeze.py:180`), which shifts back 8h, so a write just after midnight IST belongs to the prior trading session. Explicit `target_date` callers are unchanged.

Empirical status — open item. A weekday-to-weekday check (`ltp` vs next weekday `prev_close`, prod, 2026-09-01 → 2026-10-19) agrees on 88.7% of pairs (2731 of 3079). All disagreement sits in 2026-09-07 → 2026-09-18, and there are zero disagreements from 2026-09-19 onward. The September cluster is **not root-caused**. The rollover fix removes the mechanism confirmed in code, but this design section does not claim it explains that window.

Market open/closed — one call path for display routes; order paths prewarmed (2026-10-20)

- `_segment_is_open(seg_cfg, now)` (`date_time_utils.py:304`) over `is_market_open()` (line 217) is the canonical verdict for *display* routes: `/market/status`, the quote gate, and the watchlist movers gate. These hold YAML-shaped config.
- Gate-shaped callers (`_get_segments()` dicts) call `is_market_open()` directly with the same `exchange=`, `evening_open_on_holidays=`, and `special_sessions=` kwargs. They keep each gate's own `exchange_schedule` hours and do not substitute the YAML `sessions` list. This applies to the perf probe, the ticker watchdog, and the simulator guards.
- Holiday branch: the evening-open rule requires a weekday (`today.weekday() < 5`).
- **Order-path gate.** Order placement, template attach, chase, the agent close-all action, and `market_hours`-scheduled agents gated on `agent_engine._ae_segment_flags`. That function never honoured `evening_open_on_holidays` or `special_sessions`, so it falsely rejected a genuinely open MCX evening session on a holiday (live-reported 2026-10-20). It now threads both into `is_market_open()` and no longer falls back to a live probe on holidays.
- **Per-call-site status** (each line stands alone; there is no shared denominator):
  - Ticket (`_ticket_gate_market_hours_and_align_price`), basket (per-leg LIVE gate), chase (`_chase_order_impl`): **closed**. Each awaits `_prewarm_special_sessions_for_exchange()` before the sync `_build_now_ctx()`.
  - Template-attach offhours wing gate (`_check_offhours_wing_gate`, called only from the async `_apply_template_to_order_impl`): **closed**. The caller awaits the same prewarm first.
  - `market_lifecycle.poll()`: **closed**, and the highest-stakes site because it drives `close_settled` and `daily_book` snapshot writes. It awaits `fetch_special_sessions_async(segment["exchange"])` directly, not via the helper. The helper resolves `holiday_exchange`, which for the synthesized `cds` segment (raw key `"CDS"`) warms the wrong cache key.
  - `apply_plan_live`: **never exposed**. It runs via `asyncio.to_thread` in `_route_apply_path()`, so the main-loop self-deadlock guard never applies. Earlier docs listing it as exposed were wrong.
  - `_exchange_open_for_attach`: **deleted** (dead code, zero production callers). Earlier docs listed it as a live ticket-gate call site, which was wrong.
  - `run_cycle` and `actions.py`: **never exposed**. `run_cycle` builds its context in a worker thread via `asyncio.to_thread(_build_context, ...)`.
  - **Accepted residual, not fixed.** Sync main-loop callers (the `routes/agents.py` dry-run, the watchlist movers gate, the quote segment check, the simulator guards) share the warm-cache mitigation only. None is a real-money order-placement path. The list is not exhaustive.
- **Other fixes in this pass.** `_EXCHANGE_TO_SEGMENT` had no `NCO` entry, so every NCO order was rejected as market-closed; added (`a7c3375f`). Both cache functions key on the IST date (`timestamp_indian().date()`), not the system-local date, because the server runs on UTC. `fetch_special_sessions()` no longer caches a failed lookup as `[]`.
- **Known limitation.** Same-day operator edits to `market_special_sessions` are not picked up until a restart or the next 05:30 refresh.
- Probe cost: `is_trading_day()` probes only when the calendar says closed, the clock is inside 09:00–23:30, and an exchange is passed. About once a minute per exchange, cached. The newly exchange-passing sites are monitored. `probe=False` on `_segment_is_open` is the planned escape hatch and is not implemented.
- `exchange_clock`'s DB-schedule signal (EOD scheduling) remains the one mechanism not reconciled with the calendar verdict.
- **Known gap, not fixed.** The dry-run preview's `any_market_open` is always `False`: it reads a `segments` key that `_build_context()` never emits. See CLAUDE.md's "Known gap" entry.

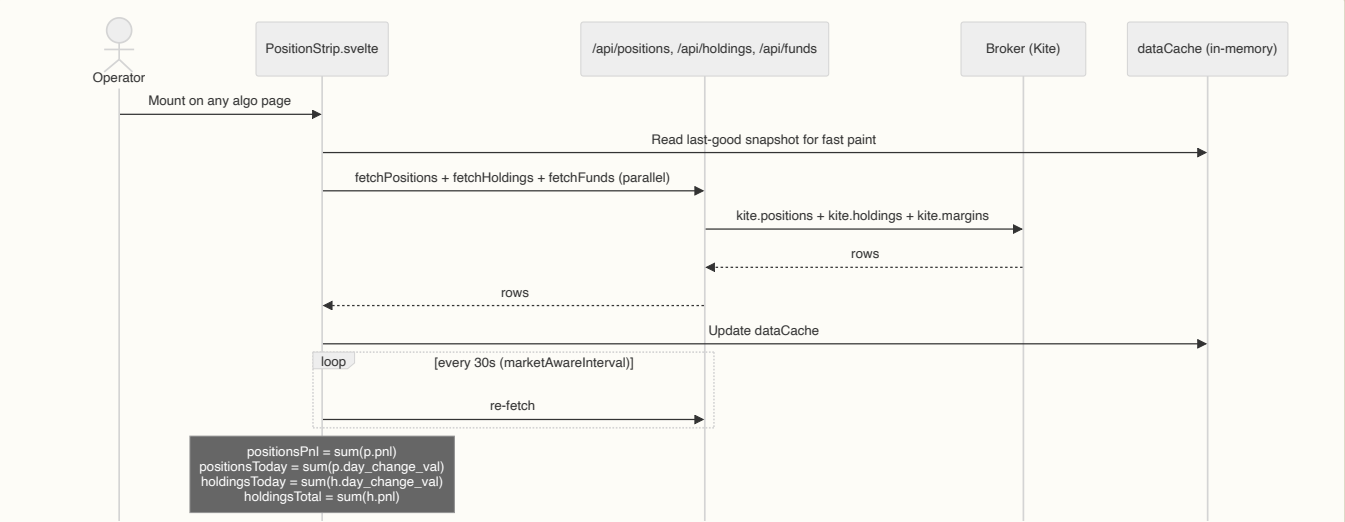
MCX evening-closed holidays (2026-10-20)

- NSE holiday-master `COM` entries carry `evening_session`. Dates marked "Closed" get a zero-width `market_special_sessions` row (`exchange='MCX'`, `source='nse_auto'`) that closes MCX all day. Weekend dates are excluded, because `next_market_open_ist()` treats row existence as open on weekends.
- The fetch returns `None` on any failure, including a malformed feed. Only a successful fetch (even an empty one) is reconciled against.
- The ON CONFLICT clause only updates `nse_auto` rows, so operator rows are never overwritten.
- The upsert reconciles: it deletes an `nse_auto` row once NSE revises that date's evening back to Open. The scope is future-or-today dates only. It also refuses to reconcile when the fresh set is empty but auto-rows exist. **Known residual edge:** that guard tests the raw closed-set, not the weekday-filtered one, so a fetch returning only weekend dates can still wipe existing rows.
- The upsert's failure path logs instead of returning silently. `morning_session` checked against the live feed: all 21 `COM` entries for 2026 carry `morning_session: "Closed"`, so there is nothing to model. The shared NSE holiday-master fetch is short-TTL memoized (5s), so the two per-refresh calls cost one HTTP round-trip.
- Holiday caches (`fetch_holidays` Tiers 1/2, `_read_market_holidays_async`) key on the IST date and year, not the system-local date.

- Failure logs for prewarm and warm-cache are at WARNING. During a persistent DB outage, `market_lifecycle.poll()` can log up to 3 warnings per 30s poll (NSE, MCX, CDS).
- Commits: prior round `85f2ade1`, `53c8da80`, `61307c47`, `125cbfea`; this round `09e53dd7`, `33e3aa18`, `eb94f67e`, `c6a947c7`.

---

21. Data refresh — PositionStrip + Dashboard



Key files:

- `frontend/src/lib/PositionStrip.svelte` — navbar strip aggregations
- `backend/api/routes/positions.py`, `holdings.py`, `funds.py` — REST endpoints
- `backend/brokers/broker_apis.py::fetch_positions / fetch_holdings / fetch_margins`
- `backend/api/cache.py` — server-side cache (per-key locking + TTL)

`/admin/derivatives` **Snapshot TOTAL reconciles to PositionStrip** by adding back the rows the page filters out (equity intraday positions + derivative-looking holdings) via `_excludedByAccount`. See `frontend/src/routes/(algo)/admin/derivatives/+page.svelte` (search `_byUnderlyingTotal`).

⚙ **TECH —** `marketAwareInterval` **polling vs WebSocket** — `WHY` Position state changes when fills happen; we already get fills via KiteTicker, but positions are aggregated server-side. Polling is cheaper than rebuilding aggregations client-side. `WHAT` `marketAwareInterval(fn, 30000)` polls every 30s during market hours, pauses on `document.hidden`. `HOW` Use the helper from `$lib/stores`; never raw `setInterval`. `WHERE` `frontend/src/lib/stores.js::marketAwareInterval`.

21.1 PositionStrip tick-border animation gate

`frontend/src/lib/PositionStrip.svelte` `tickBus.subscribe()` callback has a market-hours gate. If `!isNseOpen() && !isMcxOpen()`, the bottom-border refresh animation is skipped immediately. This prevents the tick animation from firing during closed hours and giving a false "live data refreshing" impression to the operator.

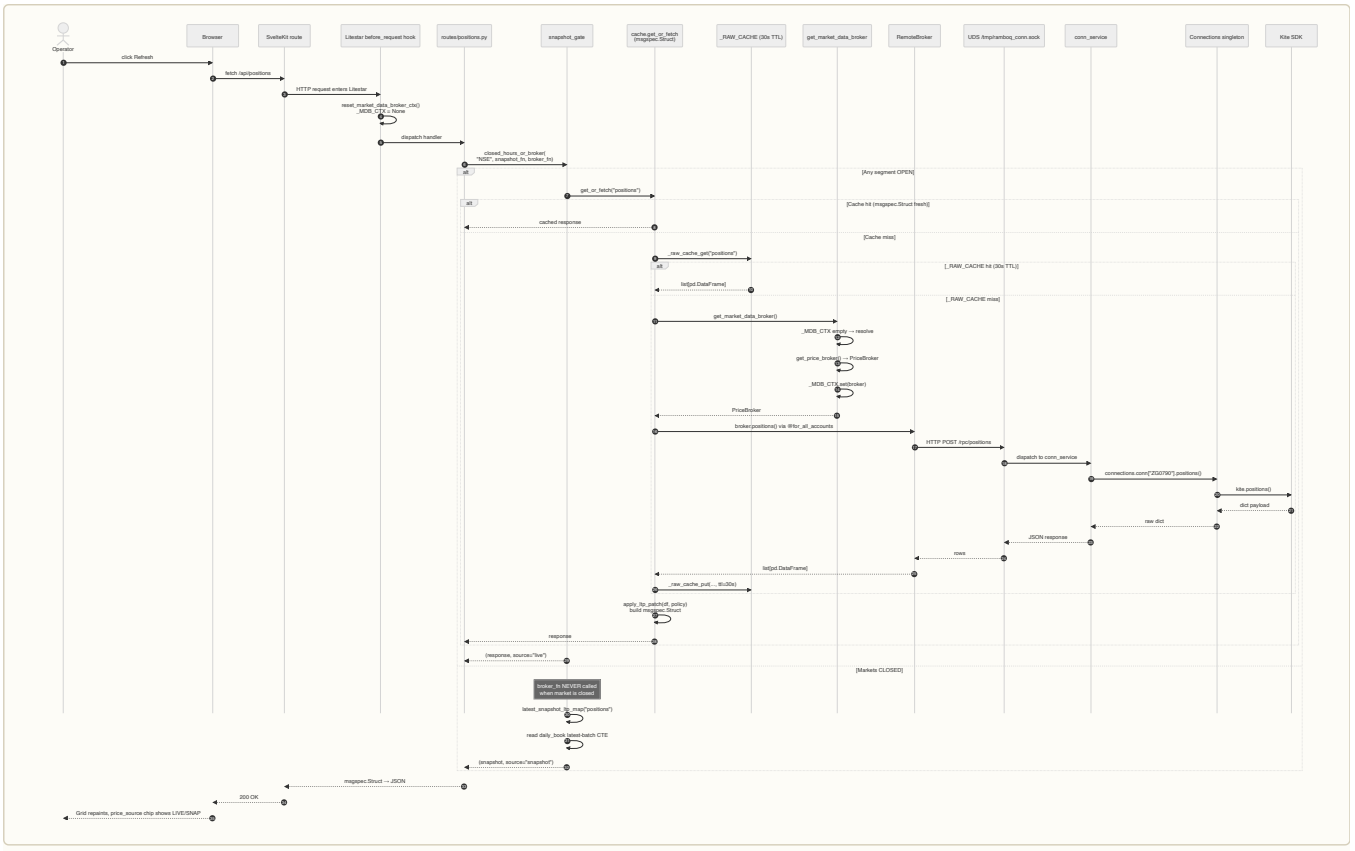
Pre-fix: animation ran continuously even when markets were closed and the WebSocket was sending stale ticks (reduced-cadence poll during quiet hours).

Files:

- `frontend/src/lib/PositionStrip.svelte::tickBus.subscribe` — market-hours gate

21.5 Frontend → broker API — full round-trip

Detailed sequence of a live `/api/positions` request when `RAMBOQ_USE_CONN_SERVICE=1`. Shows the `_MDB_CTX` contextvar reset, raw-broker-DataFrame cache (`_RAW_CACHE`, 30s TTL) on the return path, the closed-hours snapshot branch (`closed_hours_or_broker` gate), and the two-process split (main API ↔ conn_service over UDS).



Key gates:

- `reset_market_data_broker_ctx()` fires in `app.py::before_request` for **every** HTTP dispatch. All calls within one request pick the same broker (§14 + §21.7).
- `closed_hours_or_broker` — canonical gate for every live-data route. `broker_fn` is NEVER invoked when `_any_segment_open()` is False; snapshot path reads from `daily_book`.
- `_RAW_CACHE` — 30s TTL raw-DataFrame cache in `broker_apis.py`. Route-level `get_or_fetch` memoises the FORMATTED response on top. Terminal-order postbacks call `_raw_cache_invalidate(key)` so fills surface immediately.

Files:

- `backend/api/app.py` — `before_request` hook wiring
- `backend/api/helpers/snapshot_gate.py` — `closed_hours_or_broker`, `latest_snapshot_ltp_map`
- `backend/brokers/registry.py` — `get_market_data_broker`, `_MDB_CTX`, `PriceBroker._try`
- `backend/brokers/broker_apis.py` — `_RAW_CACHE`, `fetch_positions`, `@for_all_accounts`
- `backend/brokers/client/remote_broker.py` — UDS proxy
- `backend/brokers/service/app.py` — conn_service Litestar app

21.5.1 Exchange schedule — DB-backed market hours configuration

Market open/close times are now sourced from the `exchange_schedule` table rather than hardcoded hour checks. `_is_exchange_open_at()` now delegates to `exchange_clock.is_exchange_open(exchange)` (implemented in `backend/api/helpers/exchange_clock.py`), which queries the DB-backed schedule with in-process LRU caching. Changes to `exchange_schedule` propagate immediately to snapshot behavior on subsequent requests — no code redeployment required.

Default seed rows (2 rows):

- **NON-MCX**: `exchanges=[NSE,BSE,NFO,BFO,CDS]`, `open=08:00`, `close=16:00`, `snapshot=15:45`, `reset=08:00`
- **MCX**: `exchanges=[MCX]`, `open=08:00`, `close=23:30`, `snapshot=23:45`, `reset=08:00`

API CRUD protection rules — `GET|PUT|DELETE /api/admin/exchange-schedule`:

- **Default rows** (`date IS NULL`): updates allowed, deletes blocked (409 "default gate rows cannot be deleted")
- **Past-date overrides** (`date < today`): updates blocked, deletes blocked (409)
- **Future/today overrides**: full CRUD allowed
- **DTO includes** `deletable: bool` and `editable: bool` fields; frontend uses these to hide/disable controls per row

Files:

- `backend/api/helpers/exchange_clock.py::is_exchange_open` — DB query + cache
- `backend/api/routes/exchange_schedule.py` — CRUD endpoint + protection rules
- `backend/api/models.py::ExchangeSchedule` — table definition
- `backend/api/schemas.py::ExchangeScheduleDto` — response with `deletable`, `editable`

21.5.5 Day P&L SSOT — settlement-based delta

Intraday P&L requires distinguishing overnight changes from same-session ones. The **canonical SSOT** is `baseDayPnlForPosition(r)` in `frontend/src/lib/data/nav.js`, which now uses yesterday's settled P&L (`prev_settlement_pnl`) as the baseline.

Two-tier formula:

Primary path (when `prev_settlement_pnl` available on position row):

```
day_delta = pnl − prev_settlement_pnl
```

- `prev_settlement_pnl` is yesterday's `daily_book.total_pnl` for that (account, symbol)
- Backend fetches via `_backfill_prev_settlement_pnl` in `backend/api/routes/positions.py`
- Query guards `captured_at < today_midnight` to exclude today's snapshots
- Works for all cases: new intraday (delta=pnl), exited overnight (delta=realized−yesterday), continuing open (delta=(ltp−yesterday_close)×qty)

Fallback path (for positions opened today, not yet in daily_book):

```
day_delta = pnl − overnight_quantity × (close_price − average_price)
```

- When `prev_settlement_pnl` is null, compute yesterday's unrealized P&L synthetically
- Gives `pnl − 0 = pnl` for new intraday positions ✓

Backend:

- `backend/api/routes/positions.py::_override_stale_close_from_snapshot` — calls `_backfill_prev_settlement_pnl(rows)` to populate `PositionRow.prev_settlement_pnl` from `daily_book.total_pnl` (same timing guard as close_price override)
- `backend/api/models.py::PositionRow` — added `prev_settlement_pnl: Optional[float]`

Frontend SSOT:

- `baseDayPnlForPosition(r)` in `frontend/src/lib/data/nav.js` — applies the two-tier formula above; sole Day P&L formula for positions (§1 poll-only redesign — no live-tick adjustment layered on top; `livePositionDayPnl` was removed entirely, not wrapped)
- Used by:
  - PerformancePage TOTAL row (sum of daily P&L)
  - Derivatives page `_byUnderlyingTotal` loop (F&O aggregate)
  - Dashboard hero P&L + position summary
  - NavStrip P pill slot 1 (intraday P&L)
  - MarketPulse position card
  - Snapshot rows + Legs grid + Payoff overlay

21.5.6 previous_close column in daily_book — stable reference price

Kite overwrites `positions.close_price` to today's settlement price after market close, breaking the day P&L fallback formula during closed-hours snapshot reads. The `daily_book` table includes a `previous_close` column that freezes the correct reference price based on the session boundary, providing a stable overnight baseline.

Problem it solves:

- Between MCX close (23:30 IST) and next market open (08:00 IST), Kite's `positions.close_price` has already been overwritten to today's settlement (known at 00:00 IST approx).
- The closed-hours snapshot reader calls `baseDayPnlForPosition` fallback: `pnl − overnight_qty × (close_price − avg_price)`
- Using today's settlement price here → fallback computes wrong, collapses to 0 for overnight positions
- Result: NavStrip P pill + Dashboard + Performance page show zero day P&L during overnight window

Three-layer fix (commit 0da8055c):

Layer 1 — Live-path cutoff (`positions.py:_override_stale_close_from_snapshot`, `holdings.py:_override_stale_close_for_holdings`):

- Old hardcoded cutoff: `today_ist_8am if now_ist >= today_ist_8am else today_ist_midnight`

- Current: calls `await exchange_clock.settlement_cutoff_for("NSE")` which returns the last 08:00 IST boundary that has passed. Before 08:00 IST, returns yesterday's 08:00; at/after 08:00 IST returns today's 08:00. Query uses this cutoff to exclude today's EOD snapshots and properly capture prior-session settlement LTP

Layer 2 — Snapshot-path session-boundary branch (`daily_snapshot.py:snapshot_daily_book`):

- **Before 08:00 IST** (overnight): reads `daily_book.previous_close` from yesterday's rows (= prior-prior-session settlement, correctly stored by UPSERT rolling-shift). Day P&L shows: `(today's settlement − prior-prior-session settlement) × qty` (yesterday's performance)
- **At/after 08:00 IST** (new session): reads `daily_book.ltp` from yesterday's rows (= prior-session settlement = yesterday's close = new baseline). Day P&L now resets as new session opens (ltp == prev_close is intentionally valid at session open)
- Query branches at runtime based on `now_ist < today_8am` vs `>=`

Layer 3 — Data repair function (`daily_snapshot.py:fix_daily_book_prev_close`):

- Called at startup (one-time repair) and daily at 08:00 IST (new-session transition)
- **Overnight mode**: repairs rows where `|previous_close − ltp| < 0.005` (wrong data), setting `previous_close` from yesterday's `daily_book.previous_close`
- **New-session mode**: unconditionally updates today's rows to yesterday's `daily_book.ltp`
- Ensures data consistency regardless of how queries executed during the transition window
- **2026-10-20 update:** the settlement-map UPDATE targets `WHERE d.date = today` (the new session's date). Rows dated that day do not exist before 08:00, because `session_date_of()` attributes 00:00–07:59 writes to the prior session. In normal operation the UPDATE therefore matches zero rows. The "New-session mode" description above describes the code path, not normal runtime effect. The job is guarded by the DB marker `task_markers.prev_close_fix_date` (shared with the SessionGuard startup catch-up), so it runs once per IST day. Open risk: if the 08:00 run was missed and the server restarts after the same day's 15:45 or 23:45 EOD write, the catch-up UPDATE overwrites that row's `ltp`. Not verified against prod.

Solution:

- `DailyBook.previous_close: Optional[float]` column (DOUBLE PRECISION)
- **Source**: dual logic per session boundary:
  - **Before 08:00 IST**: prior-day `daily_book.previous_close` (prior-prior-session settlement)
  - **At/after 08:00 IST**: prior-day `daily_book.ltp` (prior-session settlement)
  - Both read via `prev_ltp_map` batch query in `snapshot_daily_book` ; broker REST `close_price` as fallback for first-day rows
- Written once per (date, account, kind, symbol) at first snapshot via COALESCE UPSERT
- COALESCE ensures subsequent snapshots don't overwrite the frozen value
- `_positions_snapshot` reads and passes this to `build_snapshot_position_row` , which uses it as PositionRow `close_price` when available (> 0)

Implementation details:

- **Capture point:** `backend/api/algo/daily_snapshot.py::_positions_rows()` , `_holdings_rows()` populate `previous_close` from `prev_ltp_map[(account, symbol, kind)]` which reads either `previous_close` (before 08:00) or `ltp` (at/after 08:00) based on session boundary
- **Data repair:** `backend/api/algo/daily_snapshot.py::fix_daily_book_prev_close()` repairs overnight stale data and transitions to new-session baseline at 08:00 IST
- **Persistence:** `backend/api/database.py::init_db()` creates column via idempotent `ALTER TABLE daily_book ADD COLUMN IF NOT EXISTS previous_close DOUBLE PRECISION`
- **Read point:** `backend/api/routes/positions.py::_positions_snapshot()` queries prior-day `daily_book` for each (account, symbol) and builds `prev_ltp_map` to pass to row builders
- **Use:** `backend/api/routes/positions_helpers.py::build_snapshot_position_row()` uses `previous_close` as `close_price` in the PositionRow response when `previous_close > 0`

Frontend visibility:

- Overnight window (MCX 00:15–08:00 IST): positions show day P&L from previous session (not today's settlement, not zero flash)
- At 08:00 IST: baseline resets to today's opening snapshot; day P&L correctly shows zero until intraday movement begins
- NavStrip P pill, Dashboard hero, Performance TOTAL all stay in sync across market boundaries

Files:

- `backend/api/models.py::DailyBook` — column definition
- `backend/api/database.py::init_db` — migration via ALTER TABLE
- `backend/api/algo/daily_snapshot.py::snapshot_daily_book` , `fix_daily_book_prev_close` — session-boundary branch + data repair
- `backend/api/routes/positions.py::_positions_snapshot` , `_override_stale_close_from_snapshot` , `_backfill_prev_settlement_pnl` — live + snapshot builders
- `backend/api/routes/holdings.py::_override_stale_close_for_holdings` — holdings live path
- `backend/api/routes/positions_helpers.py::build_snapshot_position_row` — use as close_price
- Frontend: no changes (uses existing PositionRow.close_price)

Key rule: never read `day_change_val` directly; always use `baseDayPnlForPosition(r)`.

Stale-snapshot guard — during closed hours, when both `close_price` and `ltp` come from the same snapshot, `baseDayPnlForPosition` detects `close_price === ltp` and returns `0` to prevent distortion (zero-flash during live→snapshot transition). This guard maintains intraday P&L continuity across market state changes.

Snapshot path parity fix — `_positions_snapshot` in `backend/api/routes/positions.py` now runs a second SQL query to fetch prior-day `daily_book` entries: `ltp AS prev_ltp, total_pnl AS prev_settlement_pnl` for each `(account, symbol)`. Builds `prev_map` and passes `prev_settlement_pnl` to `build_snapshot_position_row`, enabling Branch A (`pnl − prev_settlement_pnl`) for overnight positions viewed during closed hours (MCX overnight, weekends). Without this, the snapshot path fell back to Branch B (the formula), which could use stale intraday prices. Day P&L is now identical at market open (live path) and after close (snapshot path).

Files:

- `backend/api/routes/positions.py::_positions_snapshot` — fetches prior-day `daily_book` + passes `prev_settlement_pnl`
- `backend/api/routes/positions.py::_backfill_prev_settlement_pnl` — live path (unchanged)
- `backend/api/routes/positions_helpers.py::build_snapshot_position_row` — accepts + sets `prev_settlement_pnl` kwarg
- `backend/api/models.py::PositionRow.prev_settlement_pnl` — schema field
- `frontend/src/lib/data/nav.js::baseDayPnlForPosition` — uses Branch A when available
- All callers listed above

21.5.6 MCX snapshot multiplier fix — closed-hours position undercount

`_positions_snapshot` in `backend/api/routes/positions.py` reads closed-hours snapshots from `daily_book` and rebuilds position rows. Pre-fix, raw Kite `quantity` fields (in lots for MCX) were passed directly to the row builder, while the live path (`broker_apis.fetch_positions`) multiplies by Kite's `multiplier` field to return contracts. This asymmetry meant closed-hours MCX options showed 100× undercount (qty in lots instead of contracts).

Fix: `backend/api/routes/positions_helpers.py` exports `extract_snapshot_multiplier(snapshot_payload)` which reads the `multiplier` field from the snapshot JSON. `_positions_snapshot` applies it before building each row, ensuring closed and open hours show identical contract quantities.

Files:

- `backend/api/routes/positions_helpers.py::extract_snapshot_multiplier` — extracts + applies multiplier
- `backend/api/routes/positions.py::_positions_snapshot` — caller
- `backend/tests/test_positions_helpers.py` — 5 tests covering MCX extraction + conversion

21.5.7 Holdings snapshot `close_price` fix — preserves prior-session close

`_build_holding_row_from_snapshot` in `backend/api/routes/holdings_helpers.py` now uses `previous_close` as the primary reference (frozen via COALESCE on the first intraday snapshot), falling back to `prev_ltp` only when `previous_close` is unavailable. During closed hours, this prevents the day P&L formula from zeroing when both LTP and close_price hold the same snapshot value.

Implementation:

- `previous_close` column in `daily_book` freezes yesterday's official close at first snapshot of each trading day
- Holdings snapshot builder uses: `close_price = previous_close ?? prev_ltp ?? close_price`
- Formula remains correct: `(snapLtp − prior_close) × qty` computes legitimate overnight P&L

Files:

- `backend/api/routes/holdings_helpers.py::_build_holding_row_from_snapshot` — COALESCE logic
- `backend/api/algo/daily_snapshot.py` — writes `previous_close` at first snapshot
- `backend/api/routes/holdings.py` — caller

21.5.8 MCX 23:45 ltp=0 corruption fix — UPSERT NULL handling

Problem: MCX midnight settlement snapshot (23:45 IST) was captured with broker `ltp=0` (temporary state during settlement reconciliation), which overwrote valid Friday EOD prices. The UPSERT's plain `COALESCE(EXCLUDED.ltp, daily_book.ltp)` had no guard against zero values.

Fix — two-layer approach:

1. **Writer neutralizes zero ltp** — `backend/api/algo/daily_snapshot.py::_positions_rows()` now sets `ltp_val = None` (not skips the row) when `not mid_session and ltp_val == 0.0`. The row is still written so the `captured_at` batch join works correctly.
2. **UPSERT uses NULLIF guard** — SQL now reads:

```
ltp = CASE
      WHEN daily_book.qty = 0 AND daily_book.ltp IS NOT NULL
          THEN daily_book.ltp  -- preserve exit price for closed positions
      ELSE COALESCE(NULLIF(EXCLUDED.ltp, 0), daily_book.ltp)
    END
```

`NULLIF(EXCLUDED.ltp, 0)` converts zero to NULL so the fallback preserves the old value.

3. **Reader filters zero rows** — CTEs in snapshot paths (`latest_batch` in `positions.py` and holdings routes) now filter `(ltp IS NULL OR ltp > 0)` to exclude zero rows from delivery to positions and holdings routes.

Result: Snapshot grids and P&L remain stable through settlement windows; Friday close prices survive midnight reconciliation until replaced by Monday's 08:00 open snapshot.

Files:

- `backend/api/algo/daily_snapshot.py::_positions_rows` , `_upsert_rows` (UPSERT SQL)
- `backend/api/routes/positions.py::_positions_snapshot` (reader filter)
- `backend/api/routes/holdings.py::get_holdings` (reader filter)

21.5.9 Frontend data layer — portfolioStore + positionsDayPnlStore

The frontend's unified reactive data layer consists of two stores:

`portfolioStore` — module-level singleton in `frontend/src/lib/data/portfolioStore.svelte.js` . Computes all position and holdings aggregates via $derived.by layers (Tier 1: raw + LTP, Tier 2: day P&L

- F&O Greeks, Tier 3: chg_pct). Exports:
- `portfolioStore.positions` — `{ total, byKey, byRootPositions, byRootHoldings, byRoot, expiryByAcct }`
  - `total` — `{ day_pnl, exp_pnl, extrinsic, prev_mv, chg_pct }`
  - `byKey` — per-symbol aggregates, same structure as total
  - `byAccount` — per-account day P&L accumulation (UPPERCASE account codes) including `TOTAL` key
  - `byRoot` — for F&O, per-underlying (root symbol) aggregates with leg list
  - `byRootPositions` — cross-hedge root aggregates
  - `expiryByAcct` — expiry P&L per account (Map)
- `portfolioStore.holdings` — `{ total, byKey, byAccount, chgPctByKey }` — pulse-overridable
  - `byAccount` — per-account holdings day P&L (UPPERCASE account + TOTAL)
  - `chgPctByKey` — preserved from pulse override for accurate Chg% column
- `portfolioStore.funds` — `{ total, byAccount }`
- `setHoldingsFromPulse(byKey, total)` — MarketPulse calls this after each unified grid build to sync holdings values

SWR null-guard: When any dependency (`positionsStore` , `pulseHoldingsStore` , `fundsStore`) momentarily goes null during a 5s poll refresh, the last-known snapshot is returned instead of computing with empty arrays. Prevents derived values from zeroing during fetch transitions.

`positionsDayPnlStore` — backward-compatibility shim in `frontend/src/lib/data/positionsDayPnlStore.svelte.js` . Delegates to `portfolioStore.positions` and exports a legacy numeric API so existing consumers (NavCard, NavBreakdown, MarketPulse) need no changes:

- `.total` → number (= `portfolioStore.positions.total.day_pnl`)
- `.byKey[sym]` → number (per-symbol day P&L via Proxy)
- `.byAccount` → per-account day P&L including `TOTAL` key
- `.setFromPulse()` → no-op

Consumers:

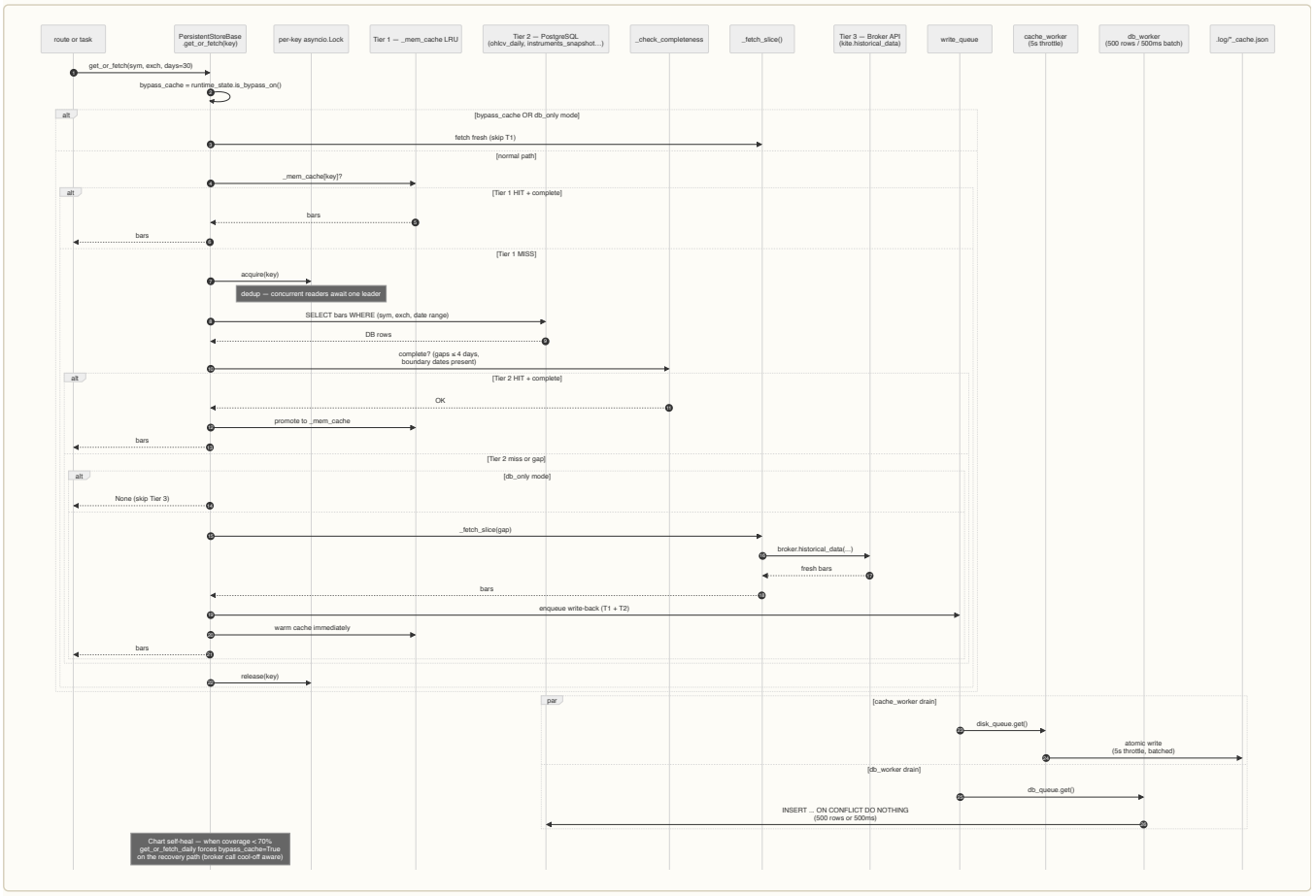
Surface	Reader	Usage
NavBreakdown	P-slot	displays `positionsDayPnlStore.byAccount[acct]` per account
MarketPulse TOTAL row	position grid footer	`positionsDayPnlStore.total`
Derivatives page	Greeks aggregate	`positionsDayPnlStore.byKey[root]` per F&O underlying

Files:

- `frontend/src/lib/data/portfolioStore.svelte.js` — unified SWR singleton
- `frontend/src/lib/data/positionsDayPnlStore.svelte.js` — legacy shim + Proxy
- `frontend/src/lib/PositionStrip.svelte` — holder of portfolioStore reference
- `frontend/src/lib/MarketPulse.svelte` — calls `setHoldingsFromPulse` after each grid build
- `frontend/src/lib/NavBreakdown.svelte` — P-slot reads `byAccount`

21.6 Persistence three-tier — cache → DB → broker

Every OHLCV / instruments / holidays / intraday read walks Tier 1 (in-memory LRU) → Tier 2 (PostgreSQL row) → Tier 3 (broker API). A per-key `asyncio.Lock` deduplicates concurrent in-flight fetches. Broker writes return immediately to the caller; persistence runs off-path via two parallel worker coroutines (`cache_worker` , `db_worker`). Refresh modes (off / soft / hard) let the operator force Tier 3 refetches.



Refresh modes (operator toggles via `POST /api/admin/persistence/mode/{off|soft|hard}`):

- `off` — normal hierarchy (default, safe)
- `soft` — Tier 1+2 bypass, fetch from broker, write-back heals both tiers
- `hard` — soft + ticker recycle (unsubscribe → reconnect → resubscribe)

Write-back workers:

- `cache_worker` — drains `disk_queue` , flushes `.log/sparkline_cache.json` etc. Throttled to 5s; last-write-wins on duplicate keys.
- `db_worker` — drains `db_queue` , batched SQL upserts per kind (500-row / 500ms boundary).
- On queue full: warn + drop; next read re-fetches from broker.

Chart self-heal (§4.5 companion) — `/api/options/historical` detects <60% coverage in DB (threshold `_SELF_HEAL_COVERAGE_THRESHOLD` in `options.py` , default 0.60) and auto-fetches from broker when ≥1 broker available. Response carries `partial: bool` for the frontend "partial data" hint.

Files:

- `backend/api/persistence/store_base.py` — `PersistentStoreBase.get_or_fetch` , per-key lock, completeness checks
- `backend/api/persistence/ohlcv_store.py` — concrete OHLCV implementation
- `backend/api/persistence/write_queue.py` — `disk_queue` + `db_queue`
- `backend/api/persistence/cache_worker.py` / `db_worker.py` — background drainers
- `backend/api/persistence/runtime_state.py` — `is_bypass_on()` , mode toggles

21.7 Stale data semantics — keepStaleOnEmpty and error recovery

Frontend data stores (`positionsStore` , `holdingsStore` , `fundsStore` , `sparklinesStore`) use **`keepStaleOnEmpty`** semantics: when a broker fetch returns empty or errors, the prior snapshot is retained client-side so grids don't flicker blank. No red banner appears on transient HTTP errors.

PositionStrip stale-data tint:

- Track consecutive fetch failures per store with `_staleFailCount`
- When `positionsStore.error || holdingsStore.error` persists for 2+ consecutive polls (`_staleFailCount >= 2`), apply CSS class `ps-stale` to the PositionStrip
- Visual: amber gradient background tint + orange border (subtle, no modal or text)
- Resets `_staleFailCount` to 0 on the next successful poll

Orders page banner gating:

- `/orders` page tracks `_orderLoadFails` (incremented per failed fetch)
- Banner "Orders feed unavailable" only renders when `_orderLoadFails >= 3`
- Suppresses noise from single transient 502s; alerts operator on sustained feed loss
- Resets counter on successful reload

Rationale: HTTP errors are often transient (network blip, broker throttle for 1-2 seconds). Keeping the stale snapshot visible prevents jarring blanks while the backend retries automatically. The PositionStrip amber tint provides subtle visual feedback that data may be stale (for 2+ polls, real signal) without alarming on every hiccup.

Files:

- `frontend/src/lib/PositionStrip.svelte` — `_staleFailCount` tracking + `ps-stale` CSS class application
- `frontend/src/routes/(algo)/orders/+page.svelte` — `_orderLoadFails` counter + 3-failure banner gate
- `frontend/src/lib/data/marketDataStores.svelte.js` — `keepStaleOnEmpty` store config

21.x Single arbitrated source per field — Pulse Chg% denominator (2026-10-20)

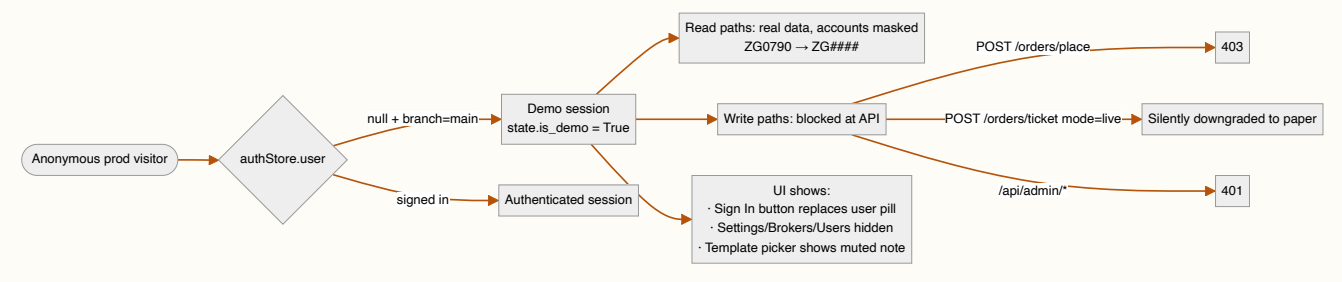
Pulse rows display `ltp` , `close` , `change_pct` , and `day_pnl` , which must move together. Each field has one arbitrated primary source. Secondary channels are fallback-only.

Field	Primary	Fallback-only
Position/holding `ltp`	SSE tick (`symbolStore` `snap.ltp`)	REST batch quote (`liveQ.ltp`)
Day P&L % denominator	Backend `prev_close` , accumulated per leg into `_prevCloseSsot` / `_prevCloseHoldSsot` (`pulseUnified.js`)	Cost basis `|avg × qty|` when no leg has a prior settlement
Underlying-anchor row fields	`_applyQuoteFields(row, snap, q)` , the same entry point real rows use	None. The bespoke `_applyUnderlyingQuoteFields` is deleted

- `row.close` stays live-quote-sourced for bid/ask/depth display. It is not the denominator.
- Two columns are headed "Chg %". The left `change_pct` column is the raw symbol change with no quantity weighting, live-quote-sourced by design. The right `day_pnl_pct` column is Day P&L ÷ prev market value and is the one this change affects.
- Movers keep their own poller. The consolidation does not cover them.
- Spec: PULSE_SPEC §11.3.

---

22. Demo mode



Key files:

- `backend/api/auth_guard.py::is_demo_request` + `auth_or_demo_guard`
- `frontend/src/routes/(algo)/+layout.svelte` — demo nav-link gating

- `frontend/src/lib/SymbolPanel.svelte` — template row demo gate (L-3)
- `backend/brokers/broker_apis.py::mask_column` — for demo + public

22.5. Investor portal — token-as-credential

Public read-only NAV surface for LPs. The URL `/investor/<token>` IS the credential — no login, no password. Operator mints from `/admin` per-user Portal button, copies the URL, forwards through their own channel (WhatsApp / email).

Why token-as-credential

The boutique fund has 1–5 LPs. Asking each LP to manage a password for a quarterly NAV check is friction nobody wants. Carta, SS&C/GP-Link, and Yieldstreet all converge on the same pattern for LP-facing statements: a long-lived per-LP URL that's revocable on suspicion of leakage. Same threat model as a long-lived API key — if you trust the recipient with the URL, the URL is fine.

⚙ **TECH — Long-lived URL token vs JWT magic-link** — `WHY` JWT magic-links are short-lived (5-15 min); they're great for a one-time "log in to this session" handshake but useless for "bookmark this URL and re-check the value every Friday." The investor portal is a recurring read-only surface, not a session. `WHAT` 32-byte `secrets.token_hex` → 64-char string, 90-day default expiry, revocable. `HOW` Stored raw in `investor_tokens.token` (same convention as `AuthToken` — the token IS the URL slug; hashing adds no security since possession of the URL == access). `WHERE` `backend/api/models.py::InvestorToken`, `backend/api/routes/investor.py`.

Schema

```
class InvestorToken(Base):
    __tablename__ = "investor_tokens"
    id, user_id (FK users.id), token (64-char unique),
    expires_at, revoked_at (nullable),
    last_visit_at, visit_count (operator visibility),
    note (admin label), created_by (FK users.id), created_at
```

`Base.metadata.create_all` picks it up on next deploy — no migration.

Active check

`_is_active(row, now) := row.revoked_at is None AND row.expires_at > now`. Three terminal states surfaced in the admin UI: ACTIVE (green) / REVOKED (red) / EXPIRED (slate).

Endpoints

Admin (`manage_investor_tokens` cap, admin-only):

- `GET /api/admin/users/{id}/investor-tokens` — list (preview only, never full token)
- `POST /api/admin/users/{id}/investor-tokens` — mint (returns full token + portal URL ONCE)
- `DELETE /api/admin/users/{id}/investor-tokens/{tid}` — revoke

Public (no auth — token in URL is the credential):

- `GET /api/investor/{token}/slice` → `InvestorSliceResponse` (same math as `/api/nav/me`)
- `GET /api/investor/{token}/history?days=180` → curve

Visit tracking

`_resolve_token()` bumps `last_visit_at` + `visit_count` on every successful resolve via a best-effort `UPDATE` (try/except, rollback on failure). The operator's admin modal surfaces the timestamp + count so they know "this LP last looked 3 weeks ago" without leaving the page. The counter increments per endpoint hit, so a single page load bumps it by 2 (slice + history).

Frontend separation

The portal page lives at `frontend/src/routes/investor/[token]/+page.svelte` — sibling of `(public)` and `(algo)` route groups. It inherits only the root `+layout.svelte` (which is empty — just `{@render children()}`), so it gets none of the algo navbar or the public marketing nav. Cream + champagne palette matching the marketing site so LPs land on a "professional statement" page, not a Bloomberg-style trading desk.

Robots `noindex,nofollow` in `<svelte:head>` so leaked URLs don't end up in search engines.

Revocation model

Revoke is destructive but trivially reversible — admin clicks Revoke (confirms via `ConfirmModal`), `revoked_at` flips to `now()`, next visit 401s. Re-mint creates a new row; the old row stays for the audit trail.

Idempotent: revoking an already-revoked row is a no-op (the original `revoked_at` timestamp is preserved so "when did we revoke this?" remains accurate).

Source files

- `backend/api/models.py::InvestorToken`
- `backend/api/routes/investor.py` — both controllers
- `backend/api/rbac.py::CAPS["manage_investor_tokens"]`
- `frontend/src/routes/investor/[token]/+page.svelte` — LP-facing page
- `frontend/src/routes/(algo)/admin/+page.svelte` — `openPortal()` modal + mint flow
- `frontend/src/lib/api.js::{fetchInvestorTokens, mintInvestorToken, revokeInvestorToken}`

22.6. Investor portal — units-based NAV math

The v1 model (`slice = share_pct × firm_nav`) is **retired**. Every LP slice / cost-basis / P&L computation now flows through the standard fund-accounting units model:

```
units_held(user, t)   = Σ units_delta for user's events <= t
total_units(t)        = Σ units_held across every LP
nav_per_unit(t)       = firm_nav(t) / total_units(t)
slice(user, t)        = units_held × nav_per_unit
cost_basis(user, t)   = Σ amount (sub+bootstrap) − Σ amount (redemption)
pnl(user, t)          = slice − cost_basis
```

⚙ **TECH — Units vs static share_pct** — `WHY` static share_pct breaks when an LP joins mid-period (their cost basis is the day they bought in, not "since fund inception") or partially redeems (the remaining slice's basis must shrink in proportion). `WHAT` Each LP holds a count of partnership units; the fund publishes a per-unit value daily; slices are products. `HOW` Subscription buys units at the day's nav_per_unit, redemption sells at the day's nav_per_unit, gains accrue automatically through firm_nav growth. `WHERE` `backend/api/algo/investor_units.py` is the single math source; four callsites consume it.

Single source of math

`backend/api/algo/investor_units.py` exposes:

- `ensure_user_bootstrap(s, user)` — idempotent synthetic-event seed for v1 → units migration
- `ensure_all_bootstrapped(s)` — covers every eligible LP; called at the start of every units compute
- `units_held(events, as_of)` / `cost_basis(events, as_of)` — pure-function primitives
- `slice_value(user_events, all_events, firm_nav, as_of)` — returns `(slice, nav_per_unit)`
- `compute_slice(s, user, firm_nav, as_of)` — DB-aware wrapper
- `compute_slice_history(user_events, all_events, firm_curve)` — pre-fetched, walks dates in pure Python

Switched callsites (all four):

Surface	Old	New
`/api/nav/me` (authenticated LP)	`share_pct × firm_nav / 100`	`compute_slice()`
`/api/nav/me/history`	scaled curve	`compute_slice_history()`
`/api/investor/{token}/slice` + `/history` (public portal)	scaled curve	`compute_slice` / `compute_slice_history`
`compute_statement()` (monthly PDF)	period-anchored scale	event-walked slice

Auto-bootstrap rule

On every units compute, `ensure_all_bootstrapped(s)` scans every eligible LP (`is_active=True AND share_pct > 0`) and inserts a synthetic event for any without one. The bootstrap row encodes the v1 state:

```
InvestorEvent(
    event_type   = "bootstrap",
    units_delta  = user.share_pct,
    amount       = user.contribution,
    nav_per_unit = contribution / share_pct  # 1.0 fallback when contribution=0
    event_date   = contribution_date or created_at.date() or today,
)
```

This guarantees that **the first request after this code lands reproduces v1 numbers identically** when share_pcts sum to 100 across all eligible LPs. When the sum != 100 (operator-residual case with implicit ownership), the units model redistributes proportionally and slices sum to `firm_nav` by construction — slightly different numbers, but internally consistent + correct going forward.

Day-delta semantics under units

`day_delta_share` on `/api/nav/me` is computed as `slice(today) − slice(prior)`, both via the SAME event set. This means:

- A pure market gain shows up as positive day-delta ✓
- A subscription between the two snapshots inflates `slice(today)` but ALSO appears in cost_basis, so it doesn't read as P&L on the LP's portal
- A redemption deflates `slice(today)` symmetrically

The portal UI labels this as "Day Δ" but the operator should read it as "change in your slice's market value since yesterday's snapshot."

Smoke-test invariants

Math is verified end-to-end against a fixture (see commit `322f0c22`):

Scenario	Invariant	Verified
All LPs at bootstrap, share_pcts sum to 100	slices match v1 exactly	✓
Fund grows N%	each LP's slice grows N% on their basis	✓
LP_A subscribes mid-period at higher nav_per_unit	extra subscription doesn't double-count as P&L	✓
Any state	Σ slices == firm_nav (modulo rounding)	✓

Bootstrap edit / correction path

Operator wants to fix a bootstrap row (wrong contribution, wrong share_pct in `users` row, missing LP):

1. Edit `User.contribution` / `share_pct` in `/admin` (existing flow)
2. Delete the bootstrap event in `/admin` → Portal → Events tab
3. Next compute auto-bootstraps with the corrected User columns

The delete-then-recompute cycle preserves history (bootstrap event has the old timestamp) without manual reconciliation.

Source files

- `backend/api/algo/investor_units.py` — math + bootstrap
- `backend/api/algo/investor_statement.py::compute_statement` — PDF math via units
- `backend/api/routes/nav.py::my_slice` + `my_history` — authenticated LP endpoints
- `backend/api/routes/investor.py::slice` + `history` — public portal endpoints
- `backend/api/models.py::InvestorEvent` — events journal table

22.7. Audit log — forensic trail

Single `audit_log` table catches every mutating event the platform produces — HTTP mutations (via middleware), broker fills (via postback handler), agent-initiated actions (via the action dispatcher), and background-task events (NAV compute, monthly statement send). Read surface at `/admin/audit` is cap-gated to `view_audit` (designated / admin / risk).

⚙ **TECH — ASGI middleware + fire-and-forget writes** — `WHY` SEBI Cat-III's "every mutating event" requirement can't be satisfied with per-route decorators (easy to forget; impossible to enforce). A middleware catches everything by default. `WHAT` `AuditMiddleware` wraps every HTTP response; on a mutating 2xx/3xx it schedules a `_write_audit` coroutine via `asyncio.create_task`. Response leaves the server immediately; the DB write lands shortly after. `HOW` Failed audit writes log a warning and drop — the user's request never blocks on the audit pipeline. `WHERE` `backend/api/audit.py`.

Schema

```
class AuditLog(Base):
    id, actor_user_id (FK users.id), actor_username, actor_role,
    action, category, method, path,
    target_type, target_id,
    status_code, summary,
    request_id (UUID, mirrored in X-Request-ID),
    client_ip, user_agent, created_at
```

- `actor_*` fields are SNAPSHOTTED — a later role demotion doesn't rewrite history.
- `category` is a coarse tag (`order.fill`, `agent.action`, `system.nav`, ...) populated by `_derive_category_from_path` for HTTP rows and explicitly by `write_audit_event` callers.
- `request_id` correlates each audit row with the API log line for the same request.

Two write paths

1. HTTP middleware (default) — `AuditMiddleware.handle` watches every response. Skips non-mutating methods + `_SUPPRESS_PREFIXES`. Captures actor from JWT, status code from the wrapped `send`, body summary from the first 1 KB of response. Path-derived category via `_derive_category_from_path`.

2. Non-HTTP helper (added Jun 2026) — `write_audit_event(category, action, ...)` is the public API for any code path that mutates state without going through HTTP:

Caller	Category	Actor
Broker postback handler (`orders.py`)	`order.fill` / `order.cancel` / `order.reject`	`broker`
Agent action dispatcher (`actions.py::execute`)	`agent.action`	`agent:<slug>`
Monthly statement send (`background.py`)	`system.statement`	`system`
NAV compute (`background.py`)	`system.nav`	`system`

The helper is fire-and-forget (`asyncio.get_running_loop().create_task(...)`); failed writes log a warning and drop. Sim-mode actions are intentionally NOT audited — they're already isolated in the sim event log and don't touch real state.

Failed mutations toggle

`audit.log_failed_mutations` setting (default `False`). When ON, the middleware also writes audit rows for 4xx/5xx mutating responses. Use for defect tracking ("operator hit SUBMIT and saw 422 — what blocked?"); toggle off otherwise to avoid volume spikes from validation errors.

Category routing

`_PATH_CATEGORY_RULES` in `audit.py` is the prefix-match table. Adding a new business surface is one tuple. Example:

```
_PATH_CATEGORY_RULES: tuple[tuple[str, str], ...] = (
    ("/api/orders/ticket",                  "order.place"),
    ("/api/orders/postback",                "order.fill"),
    ...
)
```

For `PUT/PATCH /api/orders/{id}` and `DELETE /api/orders/{id}`, the middleware narrows the generic `order` category to `order.modify` / `order.cancel` based on HTTP method.

UI — filter pills

`/admin/audit` carries a row of category pills (All / Orders / Agents / Users / Config / System) above the existing column filters. Each pill passes a comma-separated category list to the backend; SQL `IN (...)` does the rest. The Category column in the table carries per-bucket tints (green orders / cyan agents / amber users / violet config / slate system).

Performance contract

- Every audit write is `asyncio.create_task(_write_audit(...))` — no `await`. The middleware's `handle()` returns immediately after the wrapped response.
- The DB insert pays no transaction beyond its own session; failures swallow into a logger warning.
- Helper invocations from background tasks pay the same fire-and-forget cost.
- Read path is one paginated query with `LIMIT/OFFSET`; the `(category, created_at)` and `(actor_user_id, created_at)` indexes cover the common UI queries.

Source files

- `backend/api/audit.py` — middleware + `write_audit_event` helper
- `backend/api/models.py::AuditLog` — table
- `backend/api/routes/audit.py` — read surface + filters
- `frontend/src/routes/(algo)/admin/audit/+page.svelte` — viewer + pills

22.8. Postback fan-out — book_changed bus

Single coordinated refresh trigger for every position-derived surface after a broker postback. Replaces the prior pattern where each surface polled its own cadence and downstream aggregates (Snapshot grid totals, strategy analytics, payoff curve) lagged the per-cell qty patch by 5–15 s.

⚙ **TECH — Coordinated invalidation vs per-page polling** — `WHY` the postback handler historically invalidated only the `orders` cache. `positions` and `holdings` had their own 30 s TTL, and the strategy endpoint memoised its own analytics — so the Snapshot grid showed patched per-cell qty (via `position_filled` optimistic patch) but stale aggregates until the next per-page poll cycle. Operator's report: "snapshot grid updated two

iterations." `WHAT` Terminal postbacks now invalidate every dependent cache atomically + broadcast a single `book_changed` event. A frontend singleton subscriber re-emits via a Svelte store; every position-derived page subscribes once and refetches its primary loader. `HOW` 200 ms upstream debounce coalesces basket-order bursts. Monotonic counter store lets `$effect` re-run on every increment without payload comparison. `WHERE` `backend/api/routes/orders.py::order_postback` + `frontend/src/lib/data/bookChanged.js`.

Backend chain

`POST /api/orders/postback` on any terminal status (COMPLETE / CANCELLED / REJECTED / EXPIRED):

```
invalidate("orders")
if _terminal:
    invalidate("positions")
    invalidate("holdings")
    broadcast({
        "event": "book_changed",
        "account": masked, "exchange": ..., "tradingsymbol": ...,
        "reason": status, "ts": int(time() * 1000),
    })
if status == "COMPLETE":
    broadcast({"event": "position_filled", "qty": signed_delta, ...})
```

`position_filled` is preserved alongside the new event — it carries the qty delta the per-cell optimistic-patch path on Pulse + Performance reads. `book_changed` is the broader coordination signal that also covers CANCELLED / REJECTED paths where there's no qty to patch.

Frontend bus

`$lib/data/bookChanged.js`:

- Singleton subscriber via `createPerformanceSocket`. Started from `(algo)/+layout.svelte::onMount` so every algo page sees it. Idempotent — multiple `startBookChangedBus()` calls share one WS.
- Listens for `book_changed`, debounces 200 ms, increments `bookChanged` (a monotonic counter writable store) + sets `lastBookEvent` (latest payload).
- Pages subscribe with a `$effect` that watches the counter and calls their primary loader once per increment. Counter pattern (vs payload comparison) lets the effect re-run trivially on every change.

Subscription map

Page	Loaders called on increment
`/admin/derivatives`	`loadPositions()` + `loadStrategy()`
`/dashboard`	`loadHero()`
`/pulse`	`loadPulse()`
`/orders`	`_debouncedLoadOrders()`
`/performance`	`loadAll({ fresh: true })`

Recipe — wire a new page to the bus

```
<script>
  import { bookChanged } from '$lib/data/bookChanged';

  async function loadXxx() { /* page's primary loader */ }

  let _bookCounter = 0;
  $effect(() => {
    const n = $bookChanged;
    if (n <= _bookCounter) return;
    _bookCounter = n;
    loadXxx();
  });
</script>
```

That's it. The counter guard prevents re-entry; the upstream debounce handles burst coalescing.

Performance contract

- Backend fan-out runs inside the postback handler's existing `_asyncio.create_task` block — zero added latency on the broker ack path.
- One extra JSON broadcast per terminal status (~150 bytes wire). At 10 fills/sec that's 1.5 KB/sec across all WS clients combined.
- Frontend debounce keeps loader calls to one per 200 ms window per page.
- Pages without the wiring fall back to their existing pollers — additive, never breaks the prior path.

Source files

- `backend/api/routes/orders.py::order_postback` — invalidation chain + broadcasts
- `frontend/src/lib/data/bookChanged.js` — singleton subscriber + stores
- `frontend/src/routes/(algo)/+layout.svelte` — `startBookChangedBus()` callsite
- Wired pages: admin/derivatives, dashboard, MarketPulse, orders, PerformancePage

---

22.9. History — multi-day orders / trades / funds

`/admin/history` is the row-level forensic surface — three tabs over the platform's append-only datasets: Orders (`algo_orders`), Trades (`daily_book.kind='trades'`), Funds (`daily_book.kind='funds'`). Companion to `/admin/audit` (event-level log); same `view_audit` cap, different storage shape.

⚙ **TECH — Append-only daily_book vs broker SDK** — `WHY` Kite Connect (and most Indian broker SDKs) expose ONLY today's orders + trades; historical data must be scraped from the Console UI. RamboQuant snapshots every loaded account at 15:35 IST into `daily_book` so the platform owns the multi-day record-of-truth without depending on the broker's UI export. `WHAT` One row per (date, account, kind, symbol) with a unique constraint that lets re-runs upsert idempotently. `HOW` `_task_daily_snapshot` background task fires at 15:35 IST + on startup. `WHERE` `backend/api/algo/daily_snapshot.py` + `backend/api/models.py::DailyBook` .

Endpoint surface

`backend/api/routes/history.py::HistoryController` — `view_audit` cap, three reads:

Endpoint	Source	Default range	Pagination
`GET /api/admin/history/orders`	`algo_orders`	30 days	50/page, cap 500
`GET /api/admin/history/trades`	`daily_book[kind='trades']`	30 days	50/page, cap 500
`GET /api/admin/history/funds`	`daily_book[kind='funds']`	90 days	unpaged

Shared params: `from_date / to_date / accounts / symbols` (comma-separated lists for accounts + symbols). Orders adds `status / mode` . Funds drops `symbols` .

Response shape highlights

- **Orders**: `counts` field is a SQL-side `GROUP BY status` histogram. UI renders as summary pills without paginating.
- **Trades**: `summary.total_notional` is `Σ qty × avg_cost` across the FILTERED set, computed via `_func.sum()` so pagination doesn't degrade accuracy.
- **Funds**: `earliest_date` is `MIN(daily_book.date) WHERE kind='funds'` — the UI's "tracking started X" chip uses it to set expectations while historical backfill catches up.

DailyBook schema

Core `daily_book` table columns — one row per `(date, account, kind, symbol)` :

Column	Type	Purpose
`id`	SERIAL PK	Row identifier
`date`	DATE	Snapshot date (IST)
`account`	VARCHAR(32)	Broker account (indexed)
`segment`	VARCHAR(16)	'equity' | 'commodity' | 'currency' | 'derivatives'
`kind`	VARCHAR(16)	'holdings' | 'positions' | 'trades' | 'funds' (indexed with account + symbol)
`symbol`	VARCHAR(64)	Trading symbol (indexed with account, kind)
`exchange`	VARCHAR(16) NULLABLE	Exchange code (e.g. 'NSE', 'MCX')
`qty`	INTEGER	Quantity (CONTRACTS, after lot normalization via `_annotate_lot_size`)
`lots`	INTEGER	Lot count (qty / lot_size); informational
`lot_size`	INTEGER	Lot size for this instrument; informational
`avg_cost`	NUMERIC(16,4) NULLABLE	Average fill cost
`ltp`	NUMERIC(16,4) NULLABLE	Last trade price at snapshot time
`day_pnl`	NUMERIC(16,4) NULLABLE	Intraday P&L (frozen at session close)
`total_pnl`	NUMERIC(16,4) NULLABLE	Cumulative P&L since inception
`previous_close`	NUMERIC(16,4) NULLABLE	Prior session settlement LTP (frozen, first-write-wins via COALESCE in UPSERT)
`previous_close_backup`	NUMERIC(16,4) NULLABLE	Copy of previous_close saved before fix_daily_book_prev_close overwrites it; COALESCE guard prevents re-overwrite on restart
`payload_json`	TEXT NULLABLE	Raw broker row for forensics (e.g. full KiteTicker payload)
`captured_at`	TIMESTAMP(TZ)	When the snapshot was taken (UTC)

Unique constraint: `(date, account, kind, symbol)` — allows idempotent re-runs of `_task_daily_snapshot`.

Key invariant: `previous_close` is set exactly once per (account, symbol, date) via `COALESCE(..., previous_close)` in the UPSERT. Subsequent passes never overwrite a non-NULL value. This ensures day P&L formulas always reference the same prior-session settlement LTP throughout the trading day.

Funds capture (new — Jun 2026)

`_funds_rows` — runs alongside the existing holdings / positions / trades capture inside `_task_daily_snapshot`. Per account, per segment (equity / commodity), one row per day. Idempotent via the existing `daily_book` ON CONFLICT clause.

Column mapping (re-using the generic `daily_book` schema to avoid a new table):

`daily_book` column	Funds semantic
`qty`	`utilised.debits` (₹ debited today)
`avg_cost`	`available.cash`
`ltp`	`available.opening_balance`
`day_pnl`	`utilised.realised_m2m`
`total_pnl`	`net` (segment net worth)
`symbol`	`'__seg__'` sentinel (unique constraint requires non-null)
`exchange`	segment label uppercased

The mapping is intentionally pragmatic — `daily_book` is denormalised by design, and adding a separate `funds_book` table would duplicate the schema without adding value. The semantics are clear from `kind='funds'`.

Drill, delta, backfill — closed limits

Per-row audit drill — closed. `algo_orders.request_id` (nullable VARCHAR(36), indexed) captured on `POST /api/orders/ticket` from `request.scope.state.request_id` stamped by `AuditMiddleware`. `GET /api/admin/audit` accepts a `request_id` filter param; the audit page

reads `?request_id=…` URL param on mount + widens `since_hours` to 90 days. History Orders tab grows an `Audit ↗` column per row that opens `/admin/audit?request_id=<uuid>` pre-filtered.

Cashbook Δ on Funds tab — closed. `FundsRow.cash_delta` computed server-side: `HistoryController.list_funds` walks rows in O(N), groups by `(account, segment)`, sorts ASC by date, sets `prior_cash` to the previous row's `cash_available` each step. Response keeps DESC order (newest first) for the UI; per-row delta carries the move within the (account, segment) series. UI tints positive green / negative red / em-dash for the first row in a series.

Funds backfill — endpoint + Dhan adapter both wired.

Adapter contract:

```
def funds_ledger(self, from_date: str, to_date: str) -> list[dict]:
    """Return a list of normalised per-(date, segment) rows:
        [{date, segment, cash_available, opening_balance,
          debits, realised_m2m, net, payload}, ...]
    """
```

Endpoint flow:

```
@post("/funds/backfill", guards=[admin_guard])
async def backfill_funds(...) -> FundsBackfillResponse:
    broker = get_broker(account)
    if not hasattr(broker, "funds_ledger"):
        raise HTTPException(status_code=501, detail=...)
    entries = await loop.run_in_executor(
        None, lambda: broker.funds_ledger(from_iso, to_iso))
    # INSERT ... ON CONFLICT DO UPDATE per entry — same column
    # mapping as _funds_rows in the live snapshot path.
```

Broker support matrix:

- **Kite (zerodha_kite)** — no programmatic ledger. Always 501.
- **Dhan** — wired (DhanBroker.funds_ledger). SDK method discovery probes `get_ledger_report` (v2) / `get_funds_ledger` / `ledger_report` (fork variants) with kwarg→positional fallback. Aggregates voucher-level entries per `(voucherdate, segment)`; `_DHAN_SEGMENT_MAP` collapses Dhan exchange codes to our 2-segment vocabulary.
- **Groww** — pending. Same single-file pattern: add `funds_ledger(from, to)` to `GrowwBroker` returning the normalised shape.

⚙ TECH — Voucher-level aggregation vs daily snapshot

Dhan's `/v2/statement/ledger` returns voucher-level entries (one per transaction: a trade settlement, a brokerage debit, an MTM credit), not daily summaries. The adapter aggregates because `daily_book[kind='funds']` is intentionally per-day per-segment — re-using the existing snapshot schema instead of adding a `funds_ledger_voucher` table.

Aggregation logic:

- Group entries by `(voucherdate, segment)`.
- Sum `debit` + `credit` separately per group.
- Track first + last `runbal` as SOD / EOD proxies (Dhan returns entries in chronological order within a day).
- Output `cash_available = close_runbal`, `opening_balance = close_runbal − (credits − debits)`, `realised_m2m = credits − debits` (semantically "net daily cash move", not pure MTM — voucher entries include brokerage / STT / exchange charges that operator should not interpret as P&L).

Idempotency on backfill

The backfill loop uses `INSERT ... ON CONFLICT (date, account, kind, symbol) DO UPDATE SET ...` — same clause as `_upsert_rows` in the daily snapshot path. Re-running a backfill with a wider date range overwrites existing rows with the canonical Dhan numbers (intentional — if both the live snapshot AND a backfill cover the same day, the backfill's voucher-aggregated numbers are more accurate than a single broker.margins() snapshot taken at 15:35 IST).

Per-row try/except + single bulk commit at the end: a single bad voucher entry doesn't lose a multi-month pull. Failed rows log to debug + increment the `skipped` counter; the response surfaces both counts.

Remaining limit

- **Cashbook view as a separate tab** — running-balance walk that reconciles trade-leg deltas against funds snapshots row by row. The Δ column gives the daily move; a dedicated tab could enumerate the trade contributions that produced it. Not in scope for this slice; a follow-up SQL view + 4th tab.

Source files

- `backend/api/routes/history.py` — controller + 3 endpoints
- `backend/api/algo/daily_snapshot.py` — `_funds_rows` + pipeline wiring
- `backend/api/models.py::DailyBook` — table (unchanged; just a new `kind` value)
- `frontend/src/routes/(algo)/admin/history/+page.svelte` — viewer
- `frontend/src/lib/api.js` — `fetchHistoryOrders/Trades/Funds` wrappers

22.10. Order placement latency — preflight + tick cache + paper-skip

Closes the order-placement deterioration the operator flagged ("placement feels slow now"). Three orthogonal fixes ship together because they target the same hot path:

⚙ **TECH — Sequential awaits vs asyncio.gather** — `WHY` `await` on a `run_in_executor` blocks the route until the broker SDK returns; four sequential awaits = ~800-1200ms on Kite's typical 200-300ms round-trip. Even though each call is itself async-scheduled, the await chain serializes them. `WHAT` Wrap each independent broker call in its own helper coroutine with self-contained exception handling, then fire all four with `asyncio.gather`. Total wall-time becomes `max(individual call)` instead of `sum(individual calls)`. `HOW` Each helper returns a plain Python value on success or a sentinel (None / tuple-with-error) on failure, so the consumer can branch on result type rather than handle exceptions across the gather boundary. `WHERE` `backend/api/algo/actions_preflight.py::run_preflight` — fan-out helpers now take explicit args: `_preflight_fetch_profile(broker, loop, account)`, `_preflight_fetch_instruments(broker, loop, exchange, qty, account)`, `_preflight_fetch_basket_margin(broker, loop, basket_orders)`, `_preflight_fetch_account_margins(broker, loop, segment)`.

Fix 1 — preflight parallelization

`run_preflight` previously ran four broker calls in strict sequence:

```
profile = await loop.run_in_executor(None, broker.profile)         # ~300ms
instruments = await loop.run_in_executor(None, broker.instruments, exchange)  # ~250ms
bm_result = await loop.run_in_executor(None, broker.basket_order_margins, …)  # ~300ms
m = await loop.run_in_executor(None, broker.margins)               # ~300ms
# Total: ~1150ms sequential
```

The new structure:

```
# Stage 1 (synchronous) — build basket_orders (uses cached get_lot_size).
# Stage 2 (parallel) — gather the 4 independent broker calls:
profile_res, instruments_res, bm_res, margins_res = await asyncio.gather(
    _preflight_fetch_profile(broker, loop, account),
    _preflight_fetch_instruments(broker, loop, exchange, qty, account),
    _preflight_fetch_basket_margin(broker, loop, basket_orders),
    _preflight_fetch_account_margins(broker, loop, segment),
)
# Total: ~max(300ms) parallel
```

Each helper handles its own exceptions so a single broker failure surfaces as a logged warning + None result rather than tearing down the gather. `_fetch_basket_margin` returns the exception object (not raising) so the consumer can re-raise into the existing MARGIN_SHORTFALL block's try/except — minimal change to the downstream handler.

Dhan flat-dict margin fix — `_fetch_account_margins` now detects when Dhan returns a flat margin dict (presence of 'net' or 'available' key) and returns it unchanged, bypassing Kite's nested segment-key lookup. The `_EXCHANGE_SEGMENT` dict routing (MCX/NCO→commodity, CDS/BCD→currency) applies only to the Kite nested-dict path.

Fix 2 — tick-size index

`_align_price_to_tick` looked up the contract's tick_size via a linear scan:

```
for inst in items:              # items = 10–50k rows
    if inst.s == sym_u and inst.e == ex_u:
        tick = float(inst.ts or 0)
        break
```

Ticket route called this twice per order (price + trigger), so a single ticket paid ~100k linear iterations.

Now a module-level `_TICK_INDEX: dict[tuple[str,str], float]` is built lazily from the instruments cache. `_TICK_INDEX_STAMP` holds the cached `InstrumentsResponse` object; identity comparison (`resp is not _TICK_INDEX_STAMP`) detects cache refresh and triggers a rebuild. Subsequent ticket calls are O(1) dict lookups.

Trade-off: the rebuild itself is still O(N) — one scan per cache refresh (typical TTL ~10 minutes). Hot-path savings dominate; ~50ms per ticket recovered.

Fix 3 — PAPER skips route-level preflight

`PaperTradeEngine.register_open_order` already runs `basket_order_margins` internally — it's the gate that decides REJECTED vs OPEN on an open order, and writes the broker's exact error string into `AlgoOrder.detail` when the basket margin check fails. The route-level preflight before that was running the SAME basket_margin call (plus three others) for the SAME order, costing ~800ms with zero additional correctness.

The PAPER branch of `ticket_order` no longer calls `run_preflight()`. LIVE preflight stays — it's the only chance to block before `kite.place_order` actually fires.

Combined ticket-path savings

Path	Before	After
LIVE ticket	~1200ms preflight + ~150ms route + ~300ms place_order ≈ 1.65s	~300ms preflight + ~150ms route + ~300ms place_order ≈ 0.75s
PAPER ticket	~1200ms preflight + ~150ms route + ~50ms engine register ≈ 1.40s	~150ms route + ~50ms engine register ≈ 0.20s

PAPER is the bigger win because the entire preflight goes away; LIVE saves roughly half the latency.

Source files

- `backend/api/algo/actions_preflight.py::run_preflight` — parallel gather
- `backend/api/routes/orders.py::_align_price_to_tick` + `_TICK_INDEX` — O(1) tick lookup
- `backend/api/routes/orders.py::ticket_order` — PAPER preflight skip block

22.11. Navbar audit — rename + resequence

Operator-requested audit of the algo navbar.

Renames:

- `modes` group → `explore`. The old name was vestigial from the sim/paper/live/shadow/replay terminology before the mode toggles moved to the navbar dropdown (Wave C). Group now contains just `Sandbox`; can grow when Replay gets its own dedicated entry.
- `Lab` label → `Sandbox`. Industry-standard term across QuantConnect / Streak / Sensibull; reads faster to first-time visitors than the prior internal jargon. URL `/admin/execution` unchanged — bookmarks + deep links preserved.

Monitor resequence (rationale = daily-trader workflow frequency):

```
old: Tour Pulse Dashboard Derivatives Strategies NAV Orders Charts Automation
new: Tour Pulse Dashboard Orders        Derivatives Charts Automation Strategies NAV
```

Orders moved ahead of analysis surfaces (Derivatives / Charts) since active trading is the trader's primary entry point. Strategies + NAV move to the end — attribution + LP-facing views are weekly, not minute-by-minute.

Implementation:

```
// frontend/src/routes/(algo)/+layout.svelte
const GROUP_LABELS = {
  monitor: 'Monitor',
  analyze: 'Analyze',
  explore: 'Explore',
  build:   'Build',
  config:  'Config',
};
const INLINE_GROUPS = new Set(['monitor', 'analyze', 'explore']);
```

`INLINE_GROUPS` controls which groups render inline in the desktop nav; the rest collapse to dropdown triggers. Mobile drawer shows every group with a `GROUP_LABELS` caption for scan-by-intent navigation.

22.12. #audit workflow + Dhan / Groww postback scaffold

The operator runs periodic comprehensive audits by writing the literal hashtag `#audit` in chat. The coding agent dispatches 8 parallel `audit` subagents (one per dimension: performance / defects / stale code / UX consistency / palette / broker-API parity / data layer / docs) and synthesizes findings into a severity-tagged punch list. Audit findings ship as `audit slice <letter>` commits.

⚙ **TECH — Parallel audit subagents vs single deep review** — `WHY` A single agent asked to cover 8 dimensions produces shallow work on each. Eight focused subagents each get a tight scope brief, run concurrently, and report independently. `WHAT` 8 Agent tool-calls in one message with `subagent_type=audit` (read-only), each prompt cites the canonical patterns to check against. `HOW` Each subagent returns a focused punch list

with severity tags (HIGH / MED / LOW); the coordinator synthesizes into a remediation plan rather than firing fixes unilaterally. `WHERE` Memory file `~/.claude/projects/-Users-ramanambore-projects-ramboq/memory/feedback_audit_tag.md` documents the workflow.

Audit slices A, B, C (shipped Jun 2026)

Slice A — quick wins: doc drift in CLAUDE.md + README.md (navbar renames `modes` → `explore`, `Lab` → `Sandbox`, monitor sequence); stale code purges (`OrderDetail.svelte`, `margin_optimizer.py`, `shadow.py` + `ShadowController`, 4 api.js shadow stubs, `fetchPnlRange` — all confirmed unreferenced); palette fixes (LogPanel border `#10b981` → `#4ade80`; RefreshButton badges 600-level→400-level).

Slice B — perf + data layer:

- `_task_performance` ticker subscribe loop, `_task_trail_stop`, `_task_oco_pair_watcher`: sequential per-account awaits → `asyncio.gather`. Each saves ~200-300ms × N accounts.
- `get_lot_size` O(N) → O(1) via `_LOT_INDEX` dict (same identity-stamp invalidation pattern as `routes/orders.py::_TICK_INDEX`).
- 3 DB indexes added in `init_db` migration block (idempotent `CREATE INDEX IF NOT EXISTS`):
  - `ix_algo_orders_trail_stop` — partial on (mode, status) WHERE attached_gtts_json IS NOT NULL
  - `ix_news_headlines_published_at` — DESC
  - `ix_strategy_lots_open` — composite missing from create_all on pre-existing tables
- Operator-visible interrupt fixes batched in:
  - `/admin/history` + `/admin/audit`: $effect-gated load (was `onMount` checking `_canView` once at false; load never fired)
  - Algo layout: removed blanket "non-admin → /signin" redirect (vestigial from old admin/partner tier; broke tour for trader/risk/admin roles — pre-rename names were ops/observer)
  - "Prev Close" → "Close" rename across PerformancePage, MarketPulse, /admin/derivatives

Slice C — defects + Dhan/Groww postback scaffold:

- `list_active_chases` template-attach gap: live-reconcile path flipped FILLED but never called `_maybe_fire_template_attach_for_reconcile`. Now captures `_reconciled_filled` rows + fires after commit.
- `chase.py:806` partial-fill slippage formula: `quantity` → `filled_qty or quantity` (the old formula overstated slippage when partials happened).
- New routes `POST /api/orders/{dhan,groww}_postback` with shared `_process_broker_postback` helper that mirrors the Kite path's fan-out (AlgoOrder sync + audit log + cache invalidate + WS broadcasts). Best-effort; logs raw payload on first hit so parser can be tuned.

`_process_broker_postback` shared helper

`backend/api/routes/orders.py::_process_broker_postback` — extracted from the Kite postback inline logic so Dhan + Groww routes call the same fan-out:

```
async def _process_broker_postback(
    *, broker_id, order_id, status, account, symbol, txn, qty, price,
    exchange="", status_message="",
):
    # 1. AlgoOrder row sync by broker_order_id (status, fill_price, filled_at)
    # 2. order_events row (broker_postback kind)
    # 3. audit_log entry tagged order.fill|cancel|reject
    # 4. invalidate orders / positions / holdings on terminal
    # 5. broadcast order_update + position_filled + book_changed
```

Status normalization uses `_DHAN_STATUS_TO_KITE` and `_GROWW_STATUS_TO_KITE` tables already in the broker adapters. Best-effort throughout; never 5xx so the broker doesn't retry.

Source files

- `backend/api/algo/actions_preflight.py::run_preflight` — parallel gather (slice 22.10)
- `backend/api/background.py::_task_{performance,trail_stop,oco_pair_watcher}` — slice B parallel gathers
- `backend/brokers/adapters/kite.py::get_lot_size` + `_LOT_INDEX` — slice B O(1) lookup
- `backend/api/database.py::init_db` — slice B index migrations
- `backend/api/routes/orders.py::list_active_chases` — slice C template-attach fix
- `backend/api/algo/chase.py:806` — slice C slippage formula
- `backend/api/routes/orders.py::order_postback_{dhan,groww}` + `_process_broker_postback` — slice C postback scaffolds

22.13. Audit slice D — UX consistency + palette consolidation + 2 defects

Closes the remaining MED-severity items from the #audit run. Pattern: converge on canonical components + canonical CSS-custom-prop alphas rather than introducing new code.

UX consistency — adopting canonical components

Three pages were running their own bespoke implementations of patterns the codebase already had canonical components for. Each replacement deletes the bespoke chrome + the CSS that supported it:

Page	Bespoke pattern	Canonical replacement
`/admin/history`	`.hist-tabs` + `.hist-tab` (+ active border + count badge)	`<AlgoTabs tabs={[{id, label, badge}, ...]} bind:value={tab} onChange={setTab} />`
`/admin/statements`	native `<select class="ms-select">` + `.ms-select` CSS block	`<Select bind:value={selectedPeriod} options={_months} />`
`PageHeaderActions` amber+cyan hover bg	`rgba(_, 0.12)` (drift)	`rgba(_, 0.14)` matching `--algo-amber-bg` / `--algo-cyan-bg` canonical

⚙ **TECH — Canonical-component adoption vs maintaining bespoke chrome** — `WHY` Every bespoke tab strip / select / pill the operator can't visually distinguish from the canonical ones is friction on muscle memory. Slice D's audit found 4 parallel pill-strip implementations across audit / statements / templates / agent-templates pages — same job, four different CSS classes. `WHAT` Replace bespoke implementations with the existing canonical component when the contract matches; the canonical component does the styling once and every page inherits. `HOW` `AlgoTabs` already exposes `tabs: [{id, label, badge, color}]` so the History tab badge logic (showing total only on the active tab) maps trivially. `Select` accepts `options: [{value, label}]` so the period dropdown's already-correct data shape is a one-line swap. `WHERE` `frontend/src/lib/AlgoTabs.svelte`, `frontend/src/lib/Select.svelte`.

Palette alpha consolidation

The cyan-bg alpha `0.10` was drifting across 5 files (8 callsites) while the canonical `--algo-cyan-bg` is `0.14`. CommandBar / OrderCard / OrderTicket / SymbolPanel all converged. Same fix on PageHeaderActions amber `0.12` → `0.14`.

Net effect: a chip background on one page now visually matches the same conceptual element on another page. Pre-fix the operator's brain had to disambiguate "is this cyan-12 or cyan-14?" — now both reach `var(--algo-cyan-bg)`.

Defects

`paper.py::reset()` race — pre-fix the three dict-replace operations (`_open_orders`, `_price_history`, `_underlying_history`) ran unlocked. If a concurrent `step()` snapshot or `_capture_price_history` write happened during the replace, the price-history chart data for the first tick of a new sim could silently land in the OLD dict reference while the new sim queried the empty replacement. Fix: acquire `self._lock` around the replacements.

`history.py::backfill_funds` docstring — was claiming "idempotent — existing rows are not overwritten (ON CONFLICT DO NOTHING)" but the SQL is `DO UPDATE SET ...` (full overwrite). Operator-surprise risk: a hand-edit to a funds row gets clobbered on the next backfill with the same date range. Docstring now accurately documents the overwrite + warns the operator to treat funds rows as read-only.

Source files

- `frontend/src/routes/(algo)/admin/history/+page.svelte` — `<AlgoTabs>` adoption
- `frontend/src/routes/(algo)/admin/statements/+page.svelte` — `<Select>` adoption
- `frontend/src/lib/PageHeaderActions.svelte` — amber/cyan 0.12→0.14
- `frontend/src/lib/{CommandBar,SymbolPanel}.svelte` + `order/{OrderCard,OrderTicket}.svelte` — cyan 0.10→0.14
- `backend/api/algo/paper.py::reset` — `self._lock` acquisition
- `backend/api/routes/history.py::backfill_funds` — docstring correction

22.14. Market-status — broker API beats bellwether-quote probe

The agent engine's `market_hours` schedule gate and the daily snapshot pipeline both consult `probe_market_active(exchange)` to decide whether the market is currently trading. Pre-slice-E the probe used a workaround: call `kite.quote()` on bellwether symbols (NIFTY 50 + NIFTY BANK for NSE/NFO, SENSEX for BSE/BFO, or the dynamically-resolved nearest MCX commodity futures contract — not hardcoded CRUDEOIL), check `last_trade_time` freshness within a 15-minute window. Worked, but spent Kite's quote budget on a question Kite's API can't answer directly.

⚙ **TECH — Authoritative broker API vs inferred bellwether probe** — `WHY` Kite Connect has no market-status endpoint; the only signal Kite exposes is a quote with `last_trade_time`. Dhan and Groww both ship a direct market-status API (`get_market_status` and variants). When an authoritative answer is one round-trip away, prefer it over an inferred one — bellwether probes have edge cases (illiquid contracts, weekend Muhurat sessions, MCX evening sessions) where the inference disagrees with the broker. `WHAT` Extend the `Broker` ABC with an optional `market_status(exchange) -> bool | None` method. Adapters that have the API override and return True/False. The probe layer iterates brokers, takes the first definitive answer, falls back to the bellwether path when no broker answers. `HOW` SDK-method discovery probes (`getattr(sdk, 'get_market_status', None)` etc.) handle adapter version drift. Per-exchange 60s cache absorbs the per-tick gate evaluation. `WHERE` `backend/brokers/base.py::market_status`, `backend/brokers/adapters/dhan.py::market_status`, `backend/brokers/adapters/groww.py::market_status`, `backend/shared/helpers/market_probe.py::probe_market_active`.

Resolution order

```
probe_market_active(exchange):
  if cache hit and fresh (60s TTL): return cached
  for broker in all_brokers():
    verdict = broker.market_status(exchange)   # ← step 1
    if isinstance(verdict, bool):
      cache[exchange] = verdict
      return verdict
  # step 2: fall back to bellwether-quote probe (unchanged path)
  kite = resolve_kite_handle()
  if kite is None: return None
  bellwethers = _candidates(exchange, broker)
  q = kite.quote(bellwethers)
  active = any(row.last_trade_time >= now - 15min for row in q)
  cache[exchange] = active
  return active
```

Adapter contract

```
class Broker(ABC):
    def market_status(self, exchange: str) -> bool | None:
        """True / False if broker exposes a market-status endpoint
        for `exchange`; None when adapter doesn't implement one or
        the call fails. Optional method, not abstract."""
        return None
```

Both Dhan and Groww implementations probe known SDK method names (`get_market_status` / `market_status` / `get_exchange_status`) across SDK version drift; iterate the response rows (whatever shape the SDK returns); map our exchange vocabulary (NSE / BSE / NFO / BFO / CDS / MCX) to the broker's segment codes (Dhan: `NSE_EQ` / `BSE_EQ` / `NSE_FNO` / `BSE_FNO` / `NSE_CURRENCY` / `MCX_COMM` ; Groww: similar with variants); return `True` if ANY mapped segment reports active.

Side effects on quote budget

A typical Kite-only deployment hits the bellwether path on every cache miss → 4 symbols × 1 quote call ≈ 4 instruments off the 10-req/sec quote budget per probe. A Dhan-loaded deployment skips that entirely for any exchange Dhan covers; only the rare cache-miss-with-Dhan-down case falls through.

Adding the API to a new adapter

```
def market_status(self, exchange: str) -> bool | None:
    sdk = self.client
    status_fn = (getattr(sdk, "get_market_status", None)
                 or getattr(sdk, "market_status", None))
    if status_fn is None:
        return None
    try:
        resp = self._sdk.status_fn()
    except Exception as e:
        logger.debug(f"{self.broker_id}.market_status failed: {e}")
        return None
    # map your broker's segment codes → our vocabulary
    # return True if any mapped segment reports active
    # return False if all closed
    # return None if no mapping matched (fall through to next broker)
```

Source files

- `backend/brokers/base.py::market_status` — ABC default
- `backend/brokers/adapters/dhan.py::market_status` — Dhan implementation
- `backend/brokers/adapters/groww.py::market_status` — Groww implementation
- `backend/shared/helpers/market_probe.py::probe_market_active` — resolution chain + cache

22.15. Chart indicator system — pure module + overlay persistence

Technical indicators (SMA, EMA, VWAP, Bollinger Bands, RSI, MACD) live in a single pure module rather than being inlined in ChartWorkspace.

⚙ **TECH — Indicators as a pure stateless module** — `WHY` Inline math inside a 2000-line Svelte component is untestable with `node --test` (no DOM, no Svelte runtime needed). Extracting the math to a pure module means a 32-test suite can verify hand-calculated reference values, edge cases (empty arrays, N=0, constant series), and Wilder-smoothing correctness without a browser. `WHAT` `frontend/src/lib/chart/indicators.js`

exports `sma`, `ema`, `vwap`, `bollinger`, `rsi`, `macd`. Each function takes an OHLCV bars array, returns a typed series array. First (n-1) entries are `{ts, value: null}` — warmup convention. `HOW` `_assertN(n)` throws `RangeError` for non-positive or non-integer periods. MACD throws when `fast >= slow`. All functions are pure (no side-effects, no imports). `WHERE` `frontend/src/lib/chart/indicators.js`; tests at `frontend/scripts/indicators.test.js`.

Overlay palette (canonical colours, do not vary):

Overlay	CSS class	Colour	Notes
SMA 20	`overlay-sma`	`#7dd3fc` sky-blue	dashed 4-3
SMA 50	`overlay-sma`	`#c084fc` violet	dashed 6-3
EMA 20	`overlay-ema`	`#4ade80` green	solid
EMA 50	`overlay-ema`	`#fb923c` orange	dashed 6-3
VWAP	`overlay-vwap`	`#7dd3fc` cyan	solid 1.4px
BB mid	`overlay-bb`	`#7dd3fc` cyan	solid 1px
BB upper/lower	`overlay-bb`	`#7dd3fc` cyan	dashed 3-2
BB fill	`overlay-bb`	`rgba(125,211,252,0.06)`	no stroke
RSI line	`overlay-rsi`	`#fbbf24` amber	solid 1.5px
MACD line	`overlay-macd`	`#fbbf24` amber	solid 1.4px
MACD signal	`overlay-macd`	`#f87171` red	dashed 3-2
MACD histogram	(line elements)	`rgba(74,222,128,0.55)` / `rgba(248,113,113,0.55)`	green above zero, red below

Sub-panel geometry (SVG user-unit constants in ChartWorkspace.svelte):

- `RSI_H = 48` — RSI panel height
- `MACD_H = 56` — MACD panel height
- `_bandH = (_showRsi ? RSI_H : 0) + (_showMacd ? MACD_H : 0)` — reserved bottom space
- `_innerH = chartH - CPAD_T - CPAD_B - _bandH` — usable height for the price panel

Overlay persistence:

- LocalStorage key: `rbq.cache.chart-overlays.v1` (JSON array of string keys)
- Init: `$state([])` — empty server-side. Never `$state(_loadPrefs())` because `$state()` init runs during SSR where `localStorage` is undefined.
- Hydration: `onMount` reads and validates the stored array against `_OVERLAY_OPTS`. Sets `_overlaysHydrated = true` after reading.
- Save: `$effect` watches `_overlays` but guards with `if (!_overlaysHydrated) return` to prevent overwriting stored prefs during the first render frame.

VWAP note — indices (NIFTY 50, NIFTY BANK etc.) carry `volume=0` on every bar. `calcVwap()` returns `null` for all points when `cumVol=0`. The `{#if _vwapPath}` block in the SVG template silently suppresses the element. This is correct: VWAP is a price/volume metric that has no meaning for non-tradeable indices.

Buy / sell signal markers

⚙ **TECH — Signal detection as pure helpers, render layer pure SVG** — `WHY` Operator brief: surface buy/sell points TradingView-style for each active indicator. Detection logic must be testable (`node --test`) and the marker layer must respect the canonical algo palette so the visual vocabulary matches the rest of the app. `WHAT` Five exports in `indicators.js` — `emaSignals(fast, slow)`, `vwapSignals(closes, vwapArr)`, `bollingerSignals(closes, bb)`, `rsiSignals(arr, oversold=30, overbought=70)`, `macdSignals(macdLine, signalLine)`. Each returns `[{i, type:'buy'|'sell'}]`. Inputs are duck-typed — they accept raw number arrays, `{value}` arrays (real ema output), or `{close}` bars (raw OHLCV). `HOW` In ChartWorkspace, `_signalMarkers` is a `$derived.by` that re-runs only when `_bars` or `_overlays` change; `_signalLayout` translates events to `{x, y, type, tag, tooltip, stack}` records with same-bar stacking. SVG renders one `<g class="signal-marker signal-{type}">` per event with a triangle + 9px monospace tag. `WHERE` `frontend/src/lib/chart/indicators.js::emaSignals|vwapSignals|bollingerSignals|rsiSignals|macdSignals`; `frontend/src/lib/ChartWorkspace.svelte::_signalMarkers|_signalLayout`.

Marker palette + geometry (canonical, do not vary):

Element	Spec
Buy triangle	filled `#4ade80` emerald-400, 10×8 px, anchored at bar's low + 8 px pad, tip-up
Sell triangle	filled `#f87171` red-400, 10×8 px, anchored at bar's high − 8 px pad, tip-down
Stroke	`#0a0a0a` 0.5 px (subtle outline so triangles read against the chart background tint)
Tag font	9 px monospace, weight 700, paint-order stroke-then-fill with 2.5 px dark stroke for legibility against bars
Tag colour	matches triangle (`#4ade80` buy, `#f87171` sell)
Stack offset	16 px vertical between markers on the same bar (split: buys below, sells above)
Indicator tag text	`EMA↑` / `EMA↓` / `VWAP↑` / `VWAP↓` / `BB↓` (buy = lower band) / `BB↑` (sell = upper band) / `RSI↑` / `RSI↓` / `MACD↑` / `MACD↓`
Tooltip (`<title>` element)	`Buy signal — RSI 14 @ 2026-04-15` (verb + indicator + bar timestamp)
Density throttle	per-indicator cap of 12 events on dense ranges (`_bars.length >= 180`); most-recent events kept
Bollinger throttle	first bar of a contiguous lower / upper band run only — prevents 5-marker spam on multi-bar breaks

Signal detection rules (peer-platform standard — TradingView / Sensibull / Streak / Upstox):

Indicator	Buy	Sell
EMA cross	fast > slow AND prev fast ≤ prev slow (golden cross)	fast < slow AND prev fast ≥ prev slow (death cross)
VWAP	close > vwap AND prev close ≤ prev vwap	close < vwap AND prev close ≥ prev vwap
Bollinger	close ≤ lower band (first bar)	close ≥ upper band (first bar)
RSI 14	rsi > 30 AND prev rsi ≤ 30	rsi < 70 AND prev rsi ≥ 70
MACD 12/26/9	macd > signal AND prev macd ≤ prev signal	macd < signal AND prev macd ≥ prev signal

Toggle UX — Signals chip in chart toolbar renders only when `_overlays.length > 0` (no markers to show without an indicator). Default ON, persisted to `localStorage` key `rbq.cache.chart-signals.v1`. Same height + active-state palette (cyan-400) as the Intraday chip — toolbar height SSOT (`--chart-toolbar-h`) preserved.

Source files

- `frontend/src/lib/chart/indicators.js` — pure indicator + signal functions
- `frontend/src/lib/chart/paths.js` — SVG path builders extracted from ChartWorkspace: `smaPath`, `emaPath`, `vwapPath`, `bbPaths`, `rsiSeries`, `macdSeries`. Pure functions; zero dependencies.
- `frontend/src/lib/ChartWorkspace.svelte` — imports `calcEma`, `calcVwap`, `calcMacd`, `calcBollinger`, `calcRsi`, + 5 signal helpers; uses `paths.js` functions to render SVG; all overlay paths and `_signalMarkers` / `_signalLayout` are `$derived`
- `frontend/scripts/indicators.test.js` — 52-test unit suite (`node --test`) covering indicator math + 5 signal helpers
- `frontend/e2e/chart_overlays.spec.js` — indicator paths Playwright spec
- `frontend/e2e/chart_signals.spec.js` — buy/sell markers Playwright spec (chromium-desktop + mobile-portrait)

22.16. Derivatives page — cold-start and payoff improvements

Three improvements accelerate page load + rendering fidelity on the `/admin/derivatives` page:

Picker ordering — `_getCandidates()` now prioritizes symbols in descending order of relevance: options positions → futures positions → holdings → popular underlyings. On page mount, the first candidate is auto-selected, ensuring the most-likely underlying is prefilled without waiting for candidates to load.

Cold-start seed — Page no longer waits for `await loadInstruments()` to complete before rendering. Instead, `OptionsPayoff.svelte` is seeded immediately with `POPULAR_UNDERLYINGS[0]` (e.g. NIFTY 50), so the payoff chart renders instantly. When instruments arrive, the page resolves to the operator's preferred symbol or re-runs `loadStrategy()`.

Stub today_value — `_clientPayoffStub` in `backend/api/routes/derivatives.py` now returns `today_value: null` instead of `expiry_value`. This prevents the frontend's "today" curve from rendering with an incorrect negative slope before Black-Scholes pricing arrives. `OptionsPayoff.svelte` skips the today curve entirely when `hasTodayValues=false`.

Payoff curve stale-while-revalidate — On underlying symbol switch, the old payoff curve stays visible under a loading overlay instead of immediately showing a wrong intrinsic-only stub. This preserves context during the fetch latency, preventing the "chart collapsed to a single line" flash that confused operators mid-analysis. When the fresh payload arrives, the overlay clears and the new curve renders.

Files:

- `frontend/src/routes/(algo)/admin/derivatives/+page.svelte` — picker ordering + cold-start seed + stale-while-revalidate overlay

- `frontend/src/routes/(algo)/admin/derivatives/CandidateLegRow.svelte` — extracted leg-grid row component (841 lines, 9 props, 6 callbacks)
- `frontend/src/lib/OptionsPayoff.svelte` — `hasTodayValues` check
- `backend/api/routes/derivatives.py` — stub today_value change

22.17. Derivatives page — `_throttledTick` market-close gate

`_throttledTick` is a counter that drives all `$derived.by` blocks on the derivatives page (liveSpot, _clientPayoffStub, _legsExpPnlTotal, _expiryProfit, etc.). Each increment triggers a payoff chart re-render.

Bug: Pre-fix, the counter incremented on every `symbolTickCount` event unconditionally — even during closed hours when `MarketPulse`'s reduced-cadence WebSocket poll still fires. Each increment caused `_clientPayoffStub` to produce a new array reference, forcing `OptionsPayoff.svelte` to re-animate continuously overnight (expensive + misleading).

Fix: `_throttledTick++` is now gated behind `if (isMarketOpen())` in both `derivatives/+page.svelte` and `PositionStrip.svelte`. During closed hours, the counter stays frozen. The edge-detect clock in `marketAwareInterval` fires `loadStrategy()` once on the closed→open transition.

Files:

- `frontend/src/routes/(algo)/admin/derivatives/+page.svelte` — market-close gate
- `frontend/src/lib/PositionStrip.svelte` — same gate for consistency

22.18. NavStrip — lifetime slot SSOT aligned with MarketPulse TOTAL

Problem: P pill slot 2 (lifetime P&L) and H pill slots 2–3 (holdings value / lifetime P&L) all had an SSE tick-delta added on top of the raw broker-snapshot sum. MarketPulse positions and holdings grids compute their TOTAL rows as `Σ row.pnl` (raw snapshot), producing a different number for the same data.

Root cause: `_livePositionsPnl`, `_liveHoldingsTotal`, and `_liveHoldingsValue` in `PositionStrip.svelte` each added `_positionsDelta` / `_holdingsDelta` to the snapshot sum. Only slot 1 (today Day P&L) should carry the SSE delta; lifetime values are broker-snapshot aggregates and must not accumulate intraday ticks.

Fix: Removed the delta from all three lifetime derived values. Only `_liveDayPnl` (P slot 1) retains the delta so the intraday position responds immediately to LTP moves.

SSOT invariant: NavStrip lifetime values ≡ `Σ row.pnl` in MarketPulse TOTAL row (within aggCompact rounding, ≤ 0.1%). Day P&L (slot 1) may diverge up to 2% during open hours due to SSE delta.

Test coverage:

- `frontend/e2e/test_navstrip_consistency.spec.js` — P slot 2 matches TOTAL within 0.1%; slot 1 within 2%
- `frontend/e2e/test_navstrip_frozen.spec.js` — closed-hours snapshot freeze

Files:

- `frontend/src/lib/PositionStrip.svelte` — lines 415–461, three fixed `$derived.by` blocks

22.18a. NavStrip H slot 2 — SSOT switch to pulseHoldingsStore + live ltp×qty formula

Problem: NavStrip H slot 2 (Holdings Value) showed a different value than MarketPulse TOTAL Holdings Value column (e.g. 1.55c vs 1.72c). Two independent root causes:

1. **Wrong data store:** NavStrip read from `holdingsStore` (`md.holdings` cache key, separate TTL timer) while MarketPulse uses `pulseHoldingsStore` (`md.pulse.holdings`). Even with identical data, the TTL timers are independent, so the stores can hold data fetched at different times.
2. **Stale value formula:** `_liveHoldingsValue` summed `h.cur_val` — a broker-computed field frozen at fetch time. MarketPulse's `finalizeRows` in `pulseUnified.js:697` instead computes `live_ltp × qty_hold` per tick, reacting to every LTP update.

Fix:

- Switched `holdings` state in `PositionStrip.svelte` from `holdingsStore.value` to `pulseHoldingsStore.value` (same store used by MarketPulse). Both the existing `_liveHoldingsTotal` and the fixed `_liveHoldingsValue` now iterate from this single source.
- `_liveHoldingsValue` rewritten to compute `getSnapshot(sym)?.ltp × qty` (matching `finalizeRows`), falling back to `h.cur_val` when LTP is unavailable.

SSOT invariant: NavStrip H slot 2 ≡ MarketPulse TOTAL Holdings Value (within tick timing, < 0.1%).

Files:

- `frontend/src/lib/PositionStrip.svelte` — `holdings` state assignment + `_liveHoldingsValue` `$derived.by` block

22.19a. Orders fetch timeout + Chase hang prevention

Problem: `_fetch_orders()` in `orders_helpers.py` used `ThreadPoolExecutor.pool.map()` which blocks the entire map until the slowest broker finishes. A hanging Dhan `orders()` call blocks indefinitely. `get_or_fetch` holds a per-key lock, so all subsequent `/chases/active` polls queue behind it — ChaseCard polls every 3s → 10+ coroutines pile up → event loop saturated → entire site unresponsive.

Fix — three layers:

1. **Per-broker 8s timeout** (`orders_helpers.py`): Replaced `pool.map()` with explicit `fut.result(timeout=8)` per broker. On timeout, logs a warning and contributes empty list for that account. `pool.shutdown(wait=False, cancel_futures=True)` ensures abandoned futures don't block exit.
2. **Chase snapshot 10s timeout** (`orders.py`): Wrapped `get_or_fetch("orders", ...)` in `asyncio.wait_for(timeout=10.0)` in `_chase_snapshot_broker_status_by_id` . On `asyncio.TimeoutError` , returns `{}` immediately so the route always responds within 10s.
3. **ChaseCard in-flight guard** (`ChaseCard.svelte`): Added `let _fetching = false` + `_poll()` wrapper. If a prior fetch is still in flight when the 3s interval fires, the new invocation returns immediately without issuing a second request. Prevents coroutine pile-up even under slow-network conditions.

Files:

- `backend/api/routes/orders_helpers.py` — `_BROKER_ORDERS_TIMEOUT = 8` , explicit futures loop
- `backend/api/routes/orders.py` — `asyncio.wait_for` in `_chase_snapshot_broker_status_by_id`
- `frontend/src/lib/order/ChaseCard.svelte` — `_poll()` in-flight guard

22.19. CSV export button — all ag-Grid card headers

Every data card across the app now has a one-click CSV download button.

Component: `GridDownloadButton.svelte` — 1.4 rem × 1.4 rem, `var(--algo-cyan-bg)` palette, down-arrow SVG. Self-hides when `onClick` is null. Sibling CSS eliminates the left margin gap produced by `GridSearchButton + GridDownloadButton` adjacency.

CardControls integration: `CardControls.svelte` gained an `onDownload = null` prop. When non-null, the button renders between Search and Collapse. Canonical toolbar order across all cards:

```
Refresh · Search · Download · Collapse · DefaultSize · Fullscreen
```

Coverage by surface:

Surface	Card	Export target
Pulse	Pinned/Watchlist	active tab grid
Pulse	Winners / Losers	gridWin / gridLose
Pulse	Positions	gridPositions
Pulse	Holdings	gridHoldings
Dashboard	Capital (Funds + Margins)	_fundsGrid then _marginGrid
Dashboard	Equity (Pos + Hold)	_eqPosGrid then _eqHoldGrid
PerformancePage	NAV/Funds strip	navGrid or fundsGrid (tab-aware)
PerformancePage	Positions Summary	positionsSummaryGrid
PerformancePage	Positions Breakdown	positionsAllGrid
PerformancePage	Holdings Summary	holdingsSummaryGrid
PerformancePage	Holdings Breakdown	holdingsAllGrid
NavBreakdown	NAV breakdown	hand-rolled via exportRowsToCsv
Derivatives	Legs card	hand-rolled via exportRowsToCsv
Derivatives	Snapshot card	hand-rolled via exportRowsToCsv
Derivatives	Payoff card	excluded (SVG chart, no tabular data)

Hand-rolled grid utility: `frontend/src/lib/utils/csvExport.js` exports `exportRowsToCsv(rows, columns, filename)`. Accepts a column descriptor array `{header, key, format?}`, builds RFC 4180 CSV blob, triggers browser download. Used for NavBreakdown, Derivatives Legs, and Derivatives Snapshot (none of which use ag-Grid).

Files:

- `frontend/src/lib/GridDownloadButton.svelte` — new component
- `frontend/src/lib/CardControls.svelte` — `onDownload` prop
- `frontend/src/lib/utils/csvExport.js` — new hand-rolled export utility
- `frontend/src/lib/MarketPulse.svelte` — Pulse card wiring
- `frontend/src/routes/(algo)/dashboard/+page.svelte` — Dashboard wiring
- `frontend/src/lib/PerformancePage.svelte` — PerformancePage wiring
- `frontend/src/lib/NavBreakdown.svelte` — NavBreakdown wiring
- `frontend/src/routes/(algo)/admin/derivatives/+page.svelte` — Derivatives wiring

22.20. docs/ folder reorganisation + exhaustive spec files

Docs structure: All markdown documentation (except `CLAUDE.md`, `CLAUDE_HISTORY.md`, `README.md`) moved to `docs/`:

Path	Contents
`docs/specs/`	Feature behavioral contracts — PULSE_SPEC, BROKER_SPEC, NAVSTRIP_SPEC, ...
`docs/guides/`	Operator guides — USER_GUIDE, ADMIN_GUIDE, AGENTS_GUIDE, LAB_MCP_GUIDE, SIMULATOR_GUIDE
`docs/audits/`	Point-in-time audit snapshots
`docs/DESIGN_GUIDE.md`	This file
`docs/MIGRATION.md`	DB migration history
`docs/deployment.md`	Ops runbook

Spec files (21 total): Every major surface now has a behavioral contract spec in `docs/specs/`. Each spec documents: API endpoint signatures and params, frontend store variables (`$state` / `$derived` / `$effect`), explicit SSOT chain for every displayed value, edge cases, and test coverage map.

Spec files are **not** auto-loaded — Claude reads them explicitly when working on or testing the relevant surface. They are the source of truth for expected behaviour and complement `CLAUDE.md` (which describes how to build) with what each surface does.

Spec	Surface
ACTIVITY_SPEC	Activity log modal / surface
AGENTS_SPEC	Agent engine — condition tree, actions, grammar
AUDIT_SPEC	Audit log + history page
AUTOMATION_SPEC	Automation page — agents, templates, tokens
BROKER_SPEC	Broker connection layer
CHART_SPEC	Chart workspace — OHLCV, indicators, signals
DASHBOARD_SPEC	Dashboard — hero metrics, grids, agent log
DERIVATIVES_SPEC	Derivatives analytics — Greeks, payoff, strategy
EXECUTION_SPEC	Execution modes 1–5 (sim/paper/live/replay/shadow)
GTT_SPEC	GTT book + template attach pipeline
HEDGE_SPEC	Proxy hedges + beta regression
LAB_SPEC	MCP server + research lab
NAV_SPEC	NAV formula v4 + investor portal
NAVSTRIP_SPEC	NavStrip pill cluster
ORDERS_SPEC	Order placement — ticket + basket
PERSISTENCE_SPEC	Persistence pipeline — stores, modes, retention
PULSE_SPEC	MarketPulse page (expanded to ~840 lines, §11–24)
REPLAY_SPEC	Sim replay driver
SETTINGS_SPEC	DB-backed settings
SIMULATOR_SPEC	Market simulator
SYMBOLS_SPEC	Symbol resolution + virtual roots

PULSE_SPEC expansion: §11–24 added covering the unified pipeline, bucket sort, column definitions, row grouping, LTP tick flash, sparkline merge strategy, context menu, watchlist management, account multi-select, persistent cache layer, closed-hours snapshot behavior, and CardControls cluster.

Files:

- `docs/specs/*.md` — 21 spec files
- `docs/guides/*.md` — operator guides (moved from repo root)
- `generate_pdf.py` — updated path to `docs/DESIGN_GUIDE.md`
- `tools/pdf-gen/README.md` — updated path reference
- `CLAUDE.md` — docs layout table + Common Tasks doc rows added

22.21. Spec files expanded — Orders, Activity, Charts (modal + page coverage)

Three behavioral contract specs rewritten from v1.0 to v2.0 with exhaustive coverage of both the modal and bookmarkable-page variants of each surface.

ORDERS_SPEC.md v2.0 additions:

- Surface variants: OrderTicket modal (keyboard `t`, context-menu prefill, alert deep-link) vs /orders history page (filter bar, status histogram, CSV export)
- Full OrderTicket state machine: IDLE → LOADING_PREFLIGHT → PREFLIGHT_OK/FAIL → SUBMITTING → SUBMITTED/ERROR
- Field-level validation spec (symbol, exchange, product, order type, qty/lots, price, trigger, account)
- 15 audit checkpoints (G2 cap, DRAFT no-broker, basket partial fail, etc.)
- Playwright + pytest test coverage map with known gaps

ACTIVITY_SPEC.md v2.0 additions:

- 5 mount points: ActivityLogModal, /activity page, Orders card, Dashboard card, Automation inline — with shared vs isolated filter state documented
- Full `activityStore` API: `$state` variables, getters/setters, `openActivityModal(tab?)`
- 7-tab catalog with endpoint, level-parsing rule, and column list per tab
- LogPanel component props (15 configurable props), scroll behavior, lazy polling

- Deep-link override: `openActivityModal('conn')` from BrokerHealthBadge bypasses persisted tab; no-arg call leaves persisted tab unchanged
- Filter sharing rules: shared (modal + /activity page); isolated (Orders card, Dashboard card)

CHART_SPEC.md v2.0 additions:

- ChartModal (keyboard `k`, optional symbol/exchange prefill) vs /charts page (URL param sync: `?symbol=&exchange=&range=`, replaceState on change, cold-start deep-link)
- Full `chartStore` SSOT: `clearData()` wipes ohlcv + lastFetched atomically on symbol change; `clearOhlcv()` wipes bars only; `isFresh()` 30s TTL guard
- All 8 indicators fully specified (SMA 20/50, EMA 20/50 Wilder, VWAP vol-guard, Bollinger ±2σ 20-period, RSI 14, MACD 12/26/9)
- Known defects:
  - **P1 — Chart hang on null/unresolved symbol:** RESOLVED in commit `cafcf0f7`. Root cause was `$effect` calling `clearData()` on empty-string symbol + `_loadHistorical` early return skipping the loading-reset logic. Fix: `$effect` guards against `!symbol`; early return now explicitly resets loading state before returning. Spec: `docs/specs/CHART_SPEC.md` §15.
  - **P2 — MCX rollover stale bars:** virtual root may resolve to expiring contract on rollover day; workaround is manual range change to force refetch.

Files:

- `docs/specs/ORDERS_SPEC.md` — v2.0
- `docs/specs/ACTIVITY_SPEC.md` — v2.0
- `docs/specs/CHART_SPEC.md` — v2.0

22.22. Agent execution — actions.py updates (NCO guard + async close)

Three changes to `backend/api/algo/actions.py` ensure agent-driven orders are guarded correctly:

NCO (NSE Commodity) added to G1/G2 exchange guard — `run_preflight` previously bypassed the LOT_MULTIPLE (G1) and FAT_FINGER (G2) guards for F&O on "unknown" exchanges. NCO was added to the exchange set `("MCX", "NCO", "NFO", "BFO", "CDS")`, so NCO orders now receive the same lot validation as MCX orders. Pre-fix: NCO orders bypassed both guards, risking over-placement.

Unbound broker guard in _action_live_close_position — `broker` variable was not initialized before the try block. On exception, `diagnose_live_failure(broker=broker)` would reference an undefined variable. Fix: `broker = None` before try; guarded `if broker is not None` before calling diagnose. Pre-fix: exceptions swallowed silently in Python < 3.11, or masked by unrelated NameError in 3.11+.

Files:

- `backend/api/algo/actions_preflight.py::run_preflight` — NCO added to exchange guard
- `backend/api/algo/actions.py::_action_live_close_position` — broker initialization + guard

22.23. Template attach — parent_lot_size NFO/BFO/CDS support

`backend/api/algo/template_attach.py::apply_plan_live` now resolves `parent_lot_size` for all F&O exchanges: `("MCX", "NCO", "NFO", "BFO", "CDS")`. Previously, lot-size resolution was limited to MCX/NCO only, causing the G1 (LOT_MULTIPLE) guard to be dead for NFO/BFO/CDS GTT template legs.

How it works:

1. `apply_template_to_order` calls `await get_lot_size(symbol, exchange)` for all F&O exchanges
2. `plan.parent_lot_size` is always resolved (never 0) before `apply_plan_live` is called
3. `apply_plan_live` G1 check verifies every GTT leg + wing qty against `parent_lot_size`
4. On failure, returns `AttachResult.errors` immediately (upstream of broker call)

Pre-fix: G1 was dead for NFO/BFO/CDS because lot_size was unknown; orders could be placed with qty not a multiple of lot_size, causing rejects or silent qty truncation by the broker.

Files:

- `backend/api/algo/template_attach.py::apply_plan_live` — parent_lot_size resolution extended

22.24. Agent engine — topic suppression doesn't burn lifespan

`backend/api/algo/agent_engine.py::_v2_record()` call and the triggered-branch DB mutation block (increment `trigger_count`, set status to `cooldown`, clear `condition_first_true_at`) are now deferred from the per-agent loop into a post-loop survivor section (after `_compute_topic_suppression` runs). Non-triggered paths (cooldown→active transition, debounce latch persist) remain inline.

Behavioral change: a low-priority agent suppressed by a higher-tier agent on the same topic no longer has its trigger quota burned. It remains `active` and can fire on a future tick when the high-priority agent is not suppressing. Pre-fix: suppressed agents still consumed their quota as if they fired, leading to premature cooldown of secondary strategies.

Example scenario:

- Agent A (loss-hedge, tier=high): fires on -5% loss
- Agent B (loss-rebalance, tier=low, topic="loss"): fires on -3% loss
- Tick 1: A fires, suppresses B
- Tick 2: loss hits -3% but A is in cooldown → B should fire now

Pre-fix: B's quota was burned on tick 1 (suppressed), so it wouldn't fire on tick 2 even though A was cooling.

Files:

- `backend/api/algo/agent_engine.py::run_cycle` — survivor-section mutation deferral

22.25. Demo banner + Showcase page + Nav + Fullscreen + Derivatives Exp Close

Demo banner moved to layout

`feat(demo)` commit a6d5e2f0: Demo banner ("Rambo Terminal — live production...") moved from Dashboard-only to `frontend/src/routes/(algo)/+layout.svelte` so it appears on every algo page (Pulse, Dashboard, Derivatives, Orders, Charts, Performance, Automation).

- Positioned `fixed; top: 3rem; height: 2rem; z-index: 46` — does not displace content
- `fix(layout)` commit b8a5203b: Added `:has(.demo-banner)` CSS rules to adjust `.page-header` and `.algo-content` vertical spacing (same pattern as existing `:has(.ps-strip)` rules)

Showcase page — merged About + Tour

`feat(showcase)` commit 2479ddf1: `/showcase` page replaces separate TourModal + HireMeModal with unified recruiter/investor landing.

- Contains: hero (name, credentials, roles, contact CTAs), facts grid, 9 architecture cards, TourModal (60-second auto-tour still available via `/showcase?tour=1`)
- `chore` commit 417880a2: `HireMeModal.svelte` deleted (was 295 lines); HireMe reference removed
- `fix(nav)` commit 2f51706f: Nav link "Tour" → "About"; standalone About button removed from desktop + mobile hamburger menus

Fullscreen button cluster reorder

`fix(fullscreen)` commit 714f3394: In fullscreen card mode (`.fs-card-on`), button cluster order is now Refresh (leftmost) → Download → Search. Changed by adjusting the `right: calc(2rem + 0.65rem + 1.7rem * N)` slot assignments in `app.css`.

Derivatives — Exp Close per-underlying spot resolver + TOTAL row decoration

`fix(derivatives)` commits 9bb890bd + b8a5203b:

- **Exp Close spot resolution**: `annotateOptionCandidates` in `derivativesMath.js` now accepts `spot` as a number (backward-compatible) OR `(underlying: string) => number` function (v2). Internally: `const resolveSpot = typeof spot === 'function' ? spot : () => spot;`
- **Full-book expiry analysis**: `expiryCloseAnalysis` uses a `spotResolver` closure that resolves spot per-underlying via SSE snapshot → batchQuote cache → 0 fallback. The `!spot` early-return gate is removed — full-book expiry close now runs across all positions regardless of selectedUnderlying selection. Enables mixed-underlying baskets (NIFTY + BANKNIFTY + CRUDEOIL simultaneously).
- **TOTAL row CSS convention**: Container holds `display:grid + subgrid + grid-column:1/-1` for alignment only. Amber decoration (background, border-top/bottom, font-size, font-family, text-align) lives on `> span` children — matches `.byund-row-total > span` Snapshot pattern. Container alignment is orthogonal to cell styling.

Files changed:

- `frontend/src/routes/(algo)/+layout.svelte` — demo banner markup + styling
- `app.css` — demo banner + fullscreen button cluster + TOTAL row CSS
- `frontend/src/routes/(algo)/admin/showcase/+page.svelte` — new page
- `frontend/src/lib/HireMeModal.svelte` — deleted
- `frontend/src/routes/(algo)/+layout.svelte` — nav Tour→About rename
- `frontend/src/lib/data/derivativesMath.js::annotateOptionCandidates` — spot parameter accepts function
- `frontend/src/routes/(algo)/admin/derivatives/+page.svelte` — spotResolver closure in expiryCloseAnalysis

22.26. Derivatives rows + LogPanel CSS Grid — border-bottom + 2-col grid layout

Two CSS refinements improve visual clarity and fix layout bugs.

Derivatives candidate grid — row separation via border-bottom

Problem: `.cand-grid` used `row-gap: 0.2rem` for visual separation between rows. The gap is transparent, so the parent grid's dark navy background bled through, creating unintended dark horizontal stripes.

Fix: Switched to `border-bottom: 1px solid rgba(126,151,184,0.10)` on each `.cand-row` (matches `.byund-row` pattern in Snapshot grid). Result: clean row-level separation using the row's own background, not the parent's. Row visual definition is now explicit and isolated.

Files: `frontend/src/routes/(algo)/admin/derivatives/+page.svelte` — `.cand-grid` CSS + `.cand-row` CSS

Unified amber TOTAL row stratum (single CSS rule, two mechanisms)

Problem: TOTAL rows in two grids (Legs + Snapshot) were styled separately, with subtle color inconsistencies.

Solution: Single unified CSS rule now covers both:

```
.cand-row.cand-row-total,
.byund-row-total > span {
  background: linear-gradient(rgba(251,191,36,0.22), rgba(251,191,36,0.22)), #1d2a44 !important;
  border-top: 2px solid rgba(251,191,36,0.70);
  border-bottom: 1px solid rgba(251,191,36,0.40);
  color: var(--c-action);
  font-weight: 700;
}
```

Why two different mechanisms achieve the same visual:

- **Legs grid (`.cand-row-total`)**: Uses `display: grid; grid-template-columns: subgrid; grid-column: 1/-1` with `column-gap: 0.6rem` . Amber rule applied to the **container** so it covers gap areas. Child `> span` elements receive only typography overrides (no color/background/border).
- **Snapshot grid (`.byund-row-total`)**: Uses `display: contents` with zero column-gap. Amber rule applied to per-span children directly (each span gets its own amber background, border-top/bottom, color, font-weight).

Why this split design? — The container-level rule works for subgrid (grid-gap areas need coverage from parent). Per-span rules work for `display: contents` (eliminates the parent layer, leaving bare span children). By applying the same CSS rule to both selectors, we get identical visual styling across two different grid architectures — reducing maintenance burden and ensuring TOTAL rows render consistently everywhere.

Files: `frontend/src/routes/(algo)/admin/derivatives/+page.svelte` — shared `.cand-row.cand-row-total` + `.byund-row-total > span` CSS rule in app.css

LogPanel 2-column layout — CSS Grid instead of column-count

Problem: `LogPanel.svelte` used CSS `column-count: 2` for multi-column magazine layout on Agent/Terminal/System/Conn tabs (≥900px). Magazine layouts rely on the browser computing column heights automatically — but this breaks inside `overflow-y: auto` scrollable containers. The browser can't determine column heights when the container itself can scroll, causing layout thrashing.

Fix: Switched to CSS Grid: `display: grid; grid-template-columns: 1fr 1fr; column-gap: 1.5rem; align-content: start` . Grid handles dynamic row heights cleanly; scroll axis is independent from layout computation. Column divider via `background-image: linear-gradient(...)` (CSS `column-rule` only works with `column-count` , not Grid).

Breakpoint: ≥900px uses Grid; <900px uses `display: block` (single-column).

Template gate: `{multiColumn && logTab !== 'order' ? 'lp-multicol' : ''}` — Orders tab always single-column (preserves traditional table layout).

Files: `frontend/src/lib/LogPanel.svelte` — `.lp-multicol` CSS + template gate

22.27. Modal consolidation — ActivityLogModal sheet pattern + fullscreen inset flush

Three UI refinements converge modal/overlay architecture onto canonical patterns and improve fullscreen card edge-to-edge rendering.

ActivityLogModal — canonical sheet-modal overlay

Problem: `ActivityLogModal.svelte` used a custom `<ModalShell>` component that center-aligned the modal. This diverged from the app's other sheet-modals (ChartModal, OrderTicket) which use a fixed `.canonical-modal-overlay` CSS class for top-anchored sheet behavior.

Fix: Replaced `<ModalShell>` with `.canonical-modal-overlay` + `use:portal`. New structure:

- **Container:** `<div class="canonical-modal-overlay" use:portal>`
- **CSS:** `position: fixed; inset: 0; align-items: flex-start; padding-top: var(--modal-sheet-top, calc(3rem + 1.8rem))`
- **Backdrop:** `position: absolute; inset: 0; backdrop-filter: blur(2px)` sibling
- **Sheet:** `position: relative; z-index: 1` — slides down from top

Result: ActivityLogModal now visually matches the rest of the sheet-modal ecosystem (ChartModal, OrderTicket). No center-aligned break-glass modal — all modals anchor top, slide down on open, scroll internally if content exceeds viewport.

Files: `frontend/src/lib/ActivityLogModal.svelte` — replaced `<ModalShell>` with overlay div

Fullscreen card — inset flush to edges + close button removal

Problem: Fullscreen cards (`.fs-card-on` class) had `inset: 1.5rem` (floating card with rounded corners, border, shadow). The separate `PageFullscreenButton.svelte` component created a portalled close button that sat in a fixed overlay. Two issues: (a) floating inset looked wrong on edge-case mobile layouts, (b) close button lifecycle was fragile (portalled elements can unmount unexpectedly on tab switch).

Fix: Two changes—

1. **Inset changed to flush-to-edges:** `.fs-card-on` now uses `inset: var(--modal-sheet-top, calc(3rem + 1.8rem)) 0 0 0` so fullscreen cards reach the top (respecting the demo banner if present) and flush-extend left/right/bottom. No floating border-radius, no margin. Card content renders edge-to-edge, maximizing usable space.
2. **Close button inlined in card:** Removed the portalled `PageFullscreenButton.svelte` component entirely. Close button now lives in `DefaultSizeButton.svelte` (which already existed for the "reset to default size" action). Button lifecycle is now tied to the card itself, not a separate portal.

Result: fullscreen cards are now true full-screen (edge-to-edge, no floating padding), and close button lifecycle is simpler and more predictable.

Files changed:

- `frontend/src/app.css` — `.fs-card-on` inset changed
- `frontend/src/lib/DefaultSizeButton.svelte` — embedded close button
- `frontend/src/lib/PageFullscreenButton.svelte` — deleted (dead code)

Payoff analytics — spot price SSOT via frontend to backend

Problem: The derivatives payoff chart (`/admin/derivatives` page) called `fetchStrategyAnalytics(cleanLegs)` without passing the live spot price. Backend `_resolve_spot()` then attempted to resolve spot via the broker's PriceBroker chain, iterating 5 brokers on cache miss (5–30s latency on token expiry).

Fix: Frontend now passes `{ spot: liveSpot ?? null }` to the backend endpoint. Backend `_resolve_spot()` short-circuits immediately when spot is provided, skipping the entire broker quote call chain.

Result: payoff chart refreshes instantly when underlying spot is already available in the frontend's real-time SSE stream, rather than incurring a full broker round-trip every time the user changes the underlying symbol.

Files changed:

- `frontend/src/routes/(algo)/admin/derivatives/+page.svelte` — passes `spot: liveSpot` to backend
- `backend/api/routes/derivatives.py::_resolve_spot` — early return when spot is provided

22.28. Agent engine — baseline window initialization + mixed-condition gating

`backend/api/algo/agent_engine.py` fixes baseline-window handling in the v2 grammar alert system to prevent loss-alert fires during the 15-minute opening-gap period.

Session-start baseline initialization: `_update_pnl_history()` now sets `alert_state['session_start']` on the first call each day (and on cold start / process restart), enabling `_v2_baseline_live()` to correctly gate the 15-minute opening-gap baseline window. Without the session_start marker, agents firing on loss thresholds would trigger immediately at market open before establishing an intraday baseline, producing false-positive alerts. The `session_start` timestamp acts as the fence; conditions are suppressed until `now - session_start >= 900s` (15 minutes).

Mixed-condition baseline gating: `_cycle_baseline_not_ready()` now uses `_v2_all_rate_metric()` (requires ALL leaves to be rate metrics) instead of `_v2_has_rate_metric()` (any rate metric), so mixed-condition agents (e.g. `pnl ≤ threshold OR pnl_rate_abs ≤ threshold`) fire immediately on their static conditions without waiting for the baseline window. Static conditions like `pnl ≤ −5%` need no baseline; only purely rate-based conditions (e.g. `pnl_rate > 10%/min`) depend on a warm-up window. This prevents long-position loss-hedge agents (which have mixed static + rate conditions) from being silently suppressed during market open.

Files:

- `backend/api/algo/agent_engine.py::_update_pnl_history` — session_start initialization
- `backend/api/algo/agent_engine.py::_v2_baseline_live` — 15-minute window gating
- `backend/api/algo/agent_engine.py::_cycle_baseline_not_ready` — all_rate_metric check

22.29. MmapTickReader — virtual-root alias resolution for MCX SSE subscription

`backend/brokers/mmap_ticker.py::MmapTickReader` now exposes two methods for managing MCX virtual-root symbol aliases in the mmap shared-memory tick buffer, enabling per-symbol SSE subscription without alias resolution overhead.

New method: `get_token_for_sym(sym)` — Returns the instrument token for a symbol, or None if not found. Used by SSE subscription paths to confirm token existence before registering listeners.

New method: `set_virtual_root_alias(tok, root)` — Overrides the internal `_token_to_sym[tok]` mapping so the mmap poll loop emits the root symbol (e.g. `"CRUDEOIL"`) instead of the futures contract tradingsymbol (e.g. `"CRUDEOIL26JULFUT"`). This allows `_perf_subscribe_book_symbols` to register MCX virtual root aliases for SSE at 250ms cadence without requiring resolution callbacks at subscription time. The mmap writer continues to receive ticks under the contract token; the reader translates them to the canonical root symbol before broadcasting to subscribers.

Integration point: Background task `_perf_subscribe_book_symbols` now calls `set_virtual_root_alias(kite_token, virtual_root)` for each MCX position discovered during performance refresh, so SSE clients receive ticks under the semantically correct root symbol rather than seeing both the concrete contract and root in separate streams.

Files:

- `backend/brokers/mmap_ticker.py::MmapTickReader.get_token_for_sym` — new method
- `backend/brokers/mmap_ticker.py::MmapTickReader.set_virtual_root_alias` — new method
- `backend/api/background.py::_perf_subscribe_book_symbols` — caller

22.30. MmapTickReader self-healing supervision + tick-staleness indicator

In conn-service mode (`RAMBOQ_USE_CONN_SERVICE=1`) the API process runs a local tick-forwarding poll task inside `MmapTickReader` (`mmap_ticker.py`). It reads `/dev/shm/ramboq_ticks` and publishes to `BroadcastBus` for SSE. That local task could die silently: a nightly lifecycle bug (`hasattr()`-based duck-typing in the ticker stop/restart jobs) could tear it down, and nothing re-armed it. Live prices froze with no operator-visible cue. This section covers the fix: local liveness supervision, correct mode detection in the nightly jobs, and a frontend staleness indicator.

Two supervision layers, two concerns. Conn-service supervises the upstream KiteTicker WebSocket to Kite. The API-side watchdog supervises only the LOCAL poll task that forwards ticks from the mmap buffer into the API process. A healthy conn-service does not imply a healthy local forwarder, and the reverse holds too.

`MmapTickReader.check_and_heal()` — watchdog entry point:

- If the poll task is alive, returns `True` and clears the "already logged" gate so the next real death logs again.
- If the task is dead, logs one `CRITICAL` line per dead→alive transition (`[MMAP-WATCHDOG] poll task found dead — restarting`), including the causing exception type and message when the task ended with one. A prolonged outage does not spam the log.
- Restarts via `ensure_started()`.
- Keeps a rolling record of heal timestamps. Three or more heals inside 10 minutes escalate to a second `CRITICAL` line. This separates a known, self-resolving blip from a deeper problem (mmap producer side or conn-service) that needs investigation.

Caller: `_task_ticker_watchdog` (`background.py`). In conn-service mode the task runs `_watchdog_cutover_heal_loop()`, which calls `check_and_heal()` every 30 seconds. It skips the heal when the engine is idle (dev, no mode selected) because `_start_kite_ticker` deliberately never starts the reader in that state. The loop never returns on its own, since `_supervised` would otherwise restart it in a tight loop.

Nightly and morning lifecycle jobs mode-check with `is_cutover_on()`. Previously these jobs detected the architecture by `hasattr()` duck-typing on the ticker object, which let the stop job reach the local `MmapTickReader` and kill its poll task. All three jobs now branch on `is_cutover_on()` and are true no-ops on the local task in conn-service mode:

Job	Schedule	Direct mode (KiteTicker owns socket)	Conn-service mode
`_snapshot_stop_ticker()`	MCX close + 1 min (23:31 IST)	`ticker.stop()`	no-op (reactor and socket live in conn-service)
`_snapshot_restart_ticker()`	08:00 IST	Restart with rotated daily token	`check_and_heal()` unless engine idle
`unsubscribe_non_mcx()`	16:15 IST	Trim non-MCX subscriptions	no-op (no socket to trim)

Frontend staleness indicator — `tickStaleness.svelte.js`. Backend self-heal cannot be the only signal. A healthy-looking SSE connection with a dead tick bridge is exactly the failure this section addresses, so the frontend watches the stream itself:

- Tracks `lastLiveTickAt` from `quoteStream.js`. It advances only on a real `snapshot` or `tick` SSE event carrying a valid LTP. It does not advance on heartbeats or on background book-poller batches, which take a separate write path.
- A 5-second checker sets the stale flag when no real tick has landed for 75 seconds. The flag is AND-gated on `isNseOpen() || isMcxOpen()`, so a quiet, correct closed market never fires.
- Operator-visible output: an amber `TICKS STALE` badge rendered by `RefreshButton.svelte`, plus one warning toast per episode ("Live ticks stale — no updates 75s+").
- Toast re-arm rule: only a genuine new tick clears the toast-fired flag. Closing the market hides the badge, but a still-dead stream reappearing after reopen is the same episode and does not re-toast.
- Test threshold override: `window.__rbq_tickStaleMs` (set via `addInitScript`) lets Playwright avoid the 75-second wait.

Files:

- `backend/brokers/mmap_ticker.py::MmapTickReader.check_and_heal` — liveness check, restart, escalation
- `backend/api/background.py::_watchdog_cutover_heal_loop`, `_task_ticker_watchdog` — 30s caller
- `backend/api/background.py::_snapshot_stop_ticker`, `_snapshot_restart_ticker` — `is_cutover_on()` gates
- `frontend/src/lib/data/tickStaleness.svelte.js` — client-side staleness watchdog
- `frontend/src/lib/RefreshButton.svelte` — amber badge surface

22.31. Margin gate — Dhan/Groww activation, close exemption, close-path defaults (2026-10-20)

What it is: the pre-placement margin-shortfall check (`run_preflight` in `actions_preflight.py`). Kite's gate was always enforced and correct. For Dhan and Groww it was structurally inert until this round.

Root cause — flat response shape

`_preflight_leg_required` read only Kite's nested `final.total` / `initial.total` and a top-level `required`. Dhan's `_margin_for_order` and Groww's `basket_order_margins` return a flat `{"total", "var", "exposure", "available", "raw"}`. No checked key matched, so `required` was `0` for every Dhan/Groww order. `INSUFFICIENT_FUNDS` needs `required > 0` and could never fire. The unit tests stubbed only Kite's nested shape. Both adapters also coerced an absent field to `0.0`, which broke missing-vs-zero.

Parser and adapter fixes

- `_preflight_leg_required` falls back to a flat top-level `total`, checked after Kite's keys so Kite parses unchanged. An unparseable value returns `None`.
- A multi-leg basket with any `None` leg is `None` for the whole basket (fail open on the whole basket, never a partial sum). Per-leg `error` strings still feed the keyword-classification fallback.
- `_dhan_num_or_none` and `_gf_or_none` return `None` for a genuinely absent field.

Close exemption — server-verified, inside the shared gate

Every margin blocker goes through `_preflight_emit_margin_blocker`, the one site that can append a MARGIN_SHORTFALL or INSUFFICIENT_FUNDS blocker. For Dhan and Groww only, it calls `_verify_close_intent` (sign AND magnitude against the real position). A verified close is exempt. A verification error falls through to the normal blocker. The caller's raw intent is never trusted at this layer. Kite is excluded by an explicit `{"dhan", "groww"}` allowlist, because `consider_positions=True` already nets closes on Kite.

Enforce vs log-only

Setting (`/admin/settings`)	Default	Effect when `False`
`preflight.enforce_margin_dhan`	`False`	Would-be blocker goes to `diag["margin_would_block"]` and a WARNING. Order not blocked
`preflight.enforce_margin_groww`	`False`	Same, for Groww

Kite is unaffected by either setting. It is always enforced. Flip a key with no deploy, after confirming real numbers from the `[MARGIN-RAW-DHAN]` / `[MARGIN-RAW-GROWW]` lines.

Close-path defaults

`_parse_broker_call_fields` defaults a missing side to `"BUY"` and a missing product to `"NRML"`. `_action_live_close_position` and `_chase_build_tasks` omitted side. This priced SELL-closes as fresh BUYs, so Kite demanded full SPAN and tight-margin loss-limit closes were blocked. Also, `_chase_build_tasks` hardcoded `product="NRML"`, so MIS closes were placed under the wrong product and did not net against the real position. Fixed in `12b8e166` (side) and `3a28bcbd` (product from the matched position row, via `_fetch_net_position_product` in `orders_place.py`).

Fixed same round (`2c53e43f`): `_chase_build_tasks` had the same NaN-truthiness bug (`str(p.get("product") or "NRML")` → the literal `"nan"` for a missing cell). Fixed with a no-pandas-import `x != x` NaN check (this file has no other pandas dependency), same effect as the `pd.isna` guard used in `_fetch_net_position_product`.

Preview matches enforcement

The ticket margin preview is naked. It sends no `paired_legs`, matching the submit-time check. A template wing is placed only on parent fill. The preview also tracks the resolved exchange, sends lots to `/orders/ticket/preview`, and plans bare-underlying templates against the resolved contract. `/preflight` still accepts `paired_legs` for read-only API callers.

Source files

- `backend/api/algo/actions_preflight.py` — parser, `_preflight_margin_policy`, `_preflight_emit_margin_blocker`, `run_preflight`
- `backend/brokers/adapters/dhan.py` (`_margin_for_order`, `basket_order_margins`) and `backend/brokers/adapters/groww.py` (`basket_order_margins`)
- `backend/api/routes/orders_place.py` — `_verify_close_intent`, `_fetch_net_position_product`
- `backend/api/algo/actions_live.py` — `_action_live_close_position`, `_chase_build_tasks`
- `backend/shared/helpers/settings.py` — the two `enforce_margin_*` keys
- `frontend/src/lib/order/OrderTicket.svelte` — naked margin preview

Part VII — Operations

23. How to add a new template field

Templates have grown organically. The current schema is wide (5 mandatory + 7 optional fields). To add a new one:

Backend

1. **Add the column** to `OrderTemplate` in `backend/api/models.py`.
2. **Idempotent ALTER TABLE** in `backend/api/database.py::init_db`.
3. **Schema fields** in `backend/api/schemas.py` — `OrderTemplate` (response), `OrderTemplateCreate`, `OrderTemplatePatch`. Also `TicketOrderRequest` + `BasketLeg` if you want a per-submit override.
4. **`_build_overrides_json`** in `orders.py` (search for the function name) — add the override → JSON key.
5. **`resolve_template_plan`** in `template_attach.py` — add the `_pick()` call and the GTT spec emission.
6. **Seeded defaults** — update `SYSTEM_TEMPLATES` in `templates_seed.py` if your field should ship with a value.

Frontend

7. **Template management UI** at `/automation?tab=brackets` (BracketsTab) — add the input.
8. **Override input** at the shell-level Template container in `SymbolPanel.svelte` — add the override field + reset on template change.
9. **Preview** — `previewTicketTemplate` should already wire it because the backend handles it; double-check the chip render handles the new shape.

Documents

10. **Add a row** to §10 if it's a default field.
11. **Update §11** if your field has unusual merge semantics.

---

24. Testing philosophy

The codebase has fewer tests than ideal — that's a known debt. Where tests exist:

- **`backend/tests/`** — pytest + pytest-asyncio. Run via `pytest backend/tests/`.
- **`frontend/e2e/`** — Playwright. Run via `cd frontend && npx playwright test`.
- **No unit tests for frontend** — relies on `svelte-check` + manual flows + e2e.

The Playwright tests run against `dev.ramboq.com` (deployed dev branch). They're slow but high-confidence. Use them for any UX flow that changes; backend pytest for any algo/broker change.

Rule of thumb: if you're touching `chase.py`, `template_attach.py`, or any broker adapter, add a pytest test. If you're touching SymbolPanel / OrderTicket flow, add a Playwright spec.

⚙ **TECH — Why Playwright over Cypress** — `WHY` Multi-tab support, native browser context isolation, better async waits. Cypress's same-origin restrictions don't fit our auth flow (OAuth-like JWT). `WHAT` Specs in `frontend/e2e/*.spec.js`. Run with `--workers=1` so dev DB writes don't race. `HOW` Use `expect(...).toContainText(...)` for chip assertions; `toHaveAttribute('placeholder', ...)` for input placeholders. `WHERE` `frontend/e2e/`.

---

25. Logging discipline

Three log files matter:

- `api_log_file` — full API log (5MB rotating × 5). Read this first when debugging.
- `api_error_file` — stdout+stderr tee from systemd. Catches uncaught exceptions.
- `hook.log` — webhook listener output.

Log levels by intent:

- `DEBUG` — for trace-style detail. Verbose; filtered out in prod.
- `INFO` — operator-visible events. Order placed, agent fired, chase replaced.
- `WARNING` — recoverable failures. Broker auth retry, asymmetric GTT, partial OCO failure.
- `ERROR` — uncaught exceptions, lost state. Should also trigger Telegram.

Don't log inside hot loops without a rate limit. `_task_performance` ran a `logger.info` per row early on; quickly buried `api_log_file` under non-actionable noise.

---

26. Deployment notes

Both `dev` and `main` deploy via webhook. Push triggers:

```
GitHub push → webhook.ramboq.com → /etc/webhook/dispatch.sh
  → main:  /opt/ramboq/webhook/deploy.sh prod main
  → other: /opt/ramboq_dev/webhook/deploy.sh dev <branch>
```

`deploy.sh` (per env):

1. `git pull`
2. `pip install` (production deps)
3. `npm run build` (vite)
4. `systemctl restart ramboq_api.service` / `ramboq_dev_api.service`
5. `notify_deploy.py` (Telegram + ntfy since July 2026)

Per-environment serialisation: a host-wide `/tmp/ramboq_deploy.lock` prevents concurrent prod + dev builds from race-condition npm conflicts. `nice -n 19 ionice -c 3` on npm so background builds never starve API responsiveness.

Manual server work after SSH: always `chown -R www-data:www-data /opt/ramboq /opt/ramboq_dev`. Webhook deploys fail silently if file owner is wrong.

⚙ **TECH — Webhook-based deploy vs CI/CD platform** — `WHY` We're a single-server setup; GitHub Actions would add 30-60s to every deploy plus a $/runner cost. The webhook is bash + git, zero dependencies. `WHAT` `webhook` (Adnan Hajdarbegovic's daemon) listens on port 9000, validates the HMAC, runs `dispatch.sh`. `HOW` Push to a watched branch triggers it automatically. Logs in `hook.log`. `WHERE` `/etc/webhook/hooks.json` (on server); `webhook/dispatch.sh` + `webhook/deploy.sh` (in repo).

26.1 Pre-commit hook for PDF sync

New file `tools/hooks/pre-commit` — bash hook that auto-regenerates `DESIGN_GUIDE.pdf` when `docs/DESIGN_GUIDE.md` is staged for commit. PDF is staged and included in the same commit. Commit is aborted if generation fails (prevents stale PDF landing with fresh .md).

Install:

```
cp tools/hooks/pre-commit .git/hooks/pre-commit
chmod +x .git/hooks/pre-commit
```

Prerequisites:

- `cd tools/pdf-gen && npm install && npx playwright install chromium`
- `python3` with `PIL` / `Pillow` (standard in venv)

How it works:

1. Hook runs on `git commit`
2. Checks if `docs/DESIGN_GUIDE.md` is in the staging area
3. If yes, runs `python3 tools/pdf-gen/generate_pdf.py` to rebuild PDF
4. Stages `docs/DESIGN_GUIDE.pdf` if successful
5. Commits both .md and .pdf together

6. Fails commit if PDF generation errors (forces operator to fix .md syntax before pushing)

Operator experience:

- `git add docs/DESIGN_GUIDE.md` (and any other changes)
- `git commit -m "docs(...): update architecture section"`
- Hook automatically regenerates PDF and stages it
- PDF is in the commit; no separate manual step needed

Files:

- `tools/hooks/pre-commit` — new hook script
- `tools/pdf-gen/generate_pdf.py` — unchanged PDF generation logic

26.5 Recent fixes and operational improvements (Jul 2026)

Item	Issue	Fix location
Auth: `email_verified` silent drop	Non-designated actors could attempt to change `email_verified` field on `PUT /admin/users/:username` without error	`backend/api/routes/admin.py::update_user_details` — added 403 check before PATCH
Startup: PerfSnapshot NameError	`_backfill_from_disk` fired before `PerfSnapshot` imported	`backend/api/background.py::_backfill_from_disk` — added import inside function
F&O orders: lots-first API (P0)	API now accepts LOTS for `lot_size > 1` instruments; converts to contracts at boundary	§5.1; `backend/api/routes/orders_place.py`, `orders_basket.py`, `template_attach.py`
Day P&L SSOT → settlement delta	Compute day P&L as `pnl − prev_settlement_pnl` (yesterday's daily_book total); fallback to synthetic unrealized for new positions	§21.5.5; `backend/api/routes/positions.py::_backfill_prev_settlement_pnl` + `frontend/src/lib/data/nav.js::baseDayPnlForPosition`
Sparkline: MCX/CDS watchlist exchange	MCX/CDS symbols lost correct exchange during universe warmup	§20.1; `_sparkline_universe_symbols` now queries `(tradingsymbol, exchange)` pair
Sparkline: virtual root resolution	MCX/CDS watchlist items not resolved to active contract before OHLCV fetch	§20.1; `snapshot_sparkline` calls `symbol_resolver.resolve_symbol()`
Sparkline: Tier 4 fallback	OHLCV store cold-starts had no fallback during db_only mode	§20.1; `batch_sparkline` added `_fill_from_daily_book_sparkline` path
BSE ticker: 50-cap truncation	BSE holdings after 50th unresolved NSE entry weren't subscribed	§14.5.9.5; `_perf_subscribe_book_symbols` uses chunked loop, not `[:50]` slice
BSE equity token: NSE→BSE fallback	Quote/sparkline lookup for equity exchanges didn't check companion immediately	§14.5.9.5; `_resolve_token_for_sym` pairs NSE↔BSE before derivatives walk
BFO F&O: routing verified	Confirmed BSE F&O (BFO) wired correctly for subscription + gating	§14.5.9.5; BFO in sparkline exchanges, maps to NSE segment, token at index 0
NavStrip: lifetime slot SSOT fix	P slot 2 + H slots 2-3 accumulated SSE delta on lifetime values, diverging from MarketPulse TOTAL rows	§22.18; `frontend/src/lib/PositionStrip.svelte`
CSV export: all ag-Grid cards	No download button on Dashboard, PerformancePage, NavBreakdown, Derivatives Legs/Snapshot	§22.19; `GridDownloadButton.svelte` + `CardControls.svelte` + `csvExport.js`
Docs: reorganise + 21 spec files	All .md docs moved to `docs/`; 21 exhaustive behavioral contract specs added	§22.20; `docs/specs/*.md`, `docs/guides/*.md`
Specs v2.0: Orders, Activity, Chart	v1.0 specs lacked modal/page surface split, state machine, audit cases, and test map	§22.21; `docs/specs/ORDERS_SPEC.md`, `ACTIVITY_SPEC.md`, `CHART_SPEC.md`
NavStrip H slot 2 SSOT + Orders timeout	NavStrip H Holdings Value now uses pulseHoldingsStore + live ltp×qty formula (matches MarketPulse); orders fetch 8s per-broker timeout + chase 10s asyncio guard + ChaseCard in-flight guard	§22.18a, §22.19a

---

27. Sprint history + audit fixes

Previous fixes are documented in-code via comments. Key milestones:

Phase/Sprint	Key fixes	Lookup
Phase 0–3	Template attach pipeline (resolve → plan → GTT place)	grep `Phase \d`
Sprint A–E	Reconcile paths, partial fills, Dhan/Groww OCO, rate limits	grep `Sprint [A-E]`
Gap closure (B–L)	28 audit fixes across categories	`git log --grep="audit fix" -i`
Jul 2026	F&O lots convention, Day P&L SSOT, sparkline + BSE ticker, NavStrip SSOT, CSV export, docs/specs reorganisation, spec v2.0 (Orders/Activity/Chart)	See §26.5 + commit bodies
Jul 2026 (CC sprint)	Cyclomatic complexity reduction across full codebase: 0 D/E/F-grade functions, C-grade reduced from 393→216. C→B helper extraction pattern applied to all layers (brokers/, api/routes/, api/algo/, shared/). TLM CCWATCH now gates all merges.	`git log --grep="refactor(cc)"`

See commit bodies for specific gap IDs (e.g. B-1 = Dhan status map, C-3 = postback fallback window, H-5 = cap warnings). These are documented in code as defensive comments.

Part VIII — Wrap-up

28. Reading order for a new developer

If you've got a week to onboard:

Day 1 — understand the shape:

- This doc end-to-end
- `CLAUDE.md` skim (it's the operator-facing manual; some route URLs may reference `/agents/*` which has been redirected to `/automation/*` — see §29)
- `docs/specs/<SURFACE>_SPEC.md` — read the spec for the surface you're working on; each one documents API endpoints, frontend stores, and SSOT chains
- `backend/api/app.py` startup wiring
- `backend/api/models.py` schema

Day 2 — order flow:

- `frontend/src/lib/SymbolPanel.svelte` + `OrderTicket.svelte` (the modal)
- `backend/api/routes/orders.py::ticket_order` (single submit path)
- `backend/api/algo/chase.py::chase_order` (the loop)

Day 3 — templates:

- `backend/api/algo/template_attach.py` (resolve + apply)
- `backend/api/algo/templates_seed.py` (the matrix)
- Trace one BUY CE order from click → fill → attach end-to-end

Day 4 — brokers:

- `backend/brokers/base.py` (the ABC)
- `backend/brokers/adapters/kite.py` (reference impl)
- `backend/brokers/adapters/dhan.py` + `groww.py` (vendor quirks)

Day 5 — background + extras:

- `backend/api/background.py` (every task)
- `backend/api/algo/actions.py` (agent action handlers)
- `frontend/src/lib/order/ChaseCard.svelte` + `OrderCard.svelte` (display)

If you've got a day: read §7 (chase loop) above, then read `chase.py::chase_order` source. Everything else extends from that one function.

29. When in doubt

Open an `Agent` with `subagent_type=audit` and ask it to trace your specific scenario. The audit agents in this codebase are well-calibrated for finding subtle issues. Don't merge a change to `chase.py` or `template_attach.py` without one.

Known doc-drift in CLAUDE.md (as of the most recent doc audit): the older operator manual still references `/agents`, `/agents/activity`, `/agents/fragments` URLs. These have been redirected to `/automation`, `/automation/activity`, etc. The redirect routes still work; the URLs in CLAUDE.md are just stale. The current canonical URLs are under `/automation/*`.

30. Operator's mental model — the one-page summary

Action	Read this section
"What happens when I click Submit on Ticket?"	§5 — single ticket sequence
"What does the chase loop do between attempts?"	§7 — chase lifecycle
"How does TP/SL get attached?"	§9 — template attach pipeline
"Why is my SL not ratcheting on Dhan?"	§13 — trail-stop subsystem
"How does the Default pill pick the right template?"	§10 — 4-default matrix
"When does the preview chip swap on Chain?"	§17 — frontend modal state
"What runs in the background?"	§20 — task topology
"Why does the navbar strip not match the dashboard?"	§21 — data refresh paths
"What can a demo visitor do?"	§22 — demo mode flow
"How do I add a new broker?"	§15
"How do I add a new template field?"	§23
"What's the tech stack?"	§2 — overview; also inline ⚙ TECH callouts throughout

Part IX — Change recipes (cookbook)

This section turns the design knowledge above into runnable change recipes. Each recipe lists the **exact files to edit**, the **exact pattern to copy**, and the **verification step** before commit. Use these as templates — copy-paste, rename, tweak.

The cookbook is intentionally prescriptive. You do not need to read the full doc above to follow a recipe; you only need §3 (architectural principles) for the philosophy, then jump straight here.

31. Recipe: add a new route

Scenario: you want a new endpoint, e.g. `GET /api/positions/heatmap` that returns aggregated per-symbol stats.

Steps

1. **Pick the right route file.** `backend/api/routes/` is grouped by domain (`orders.py`, `positions.py`, `holdings.py`, `agents.py`, etc.). Add the new route to the matching file. New domain? Create `heatmap.py`.
2. **Write the msgspec response type** in `backend/api/schemas.py`:

```
class HeatmapRow(msgspec.Struct):
    symbol: str
    pnl: float
    weight: float
class HeatmapResponse(msgspec.Struct):
    rows: list[HeatmapRow]
    refreshed_at: str
```

3. **Add the route** to the controller (mirror an existing simple route as a template — e.g. `PositionsController.list_positions` in `backend/api/routes/positions.py`):

```
@get("/heatmap", guards=[auth_or_demo_guard])
async def heatmap(self, request: Request) -> HeatmapResponse:
    is_demo = request.state.is_demo
    rows = await self._build_heatmap()
    if is_demo:
        rows = [_mask_row(r) for r in rows]
    return HeatmapResponse(rows=rows, refreshed_at=timestamp_display())
```

4. **Register the controller** in `backend/api/app.py` if the file is new. Existing controllers don't need re-registration.
5. **Frontend wrapper** in `frontend/src/lib/api.js`:

```
export const fetchHeatmap = () => _get('/api/positions/heatmap');
```

   The wrapper handles auth, retries, demo masking display, and error trimming automatically. **Never call `fetch()` directly from a component** — always go through `api.js`.
6. **Demo masking.** Read paths must mask account values for demo sessions (§22). Use `mask_column(col)` helper, never roll your own.
7. **Verify.** `pytest backend/tests/test_routes_smoke.py` (add a smoke test if the route is non-trivial) + manual curl.

---

32. Recipe: add a column to an existing table

Scenario: you want `algo_orders.last_chase_quote` to store the last broker depth snapshot.

Steps

1. **Edit `backend/api/models.py`.** Add the field to the SQLAlchemy model:

```
last_chase_quote: Mapped[str | None] = mapped_column(Text, nullable=True)
```

2. **Add an idempotent ALTER** to `backend/api/database.py::init_db`:

```
await conn.execute(text(
    "ALTER TABLE algo_orders ADD COLUMN IF NOT EXISTS last_chase_quote TEXT"
))
```

   The `IF NOT EXISTS` is non-negotiable — `init_db` runs on every startup, idempotency required.
3. **Update msgspec schema** in `backend/api/schemas.py` if the column should be returned over the wire:

```
class AlgoOrderInfo(msgspec.Struct, kw_only=True):
    ...
    last_chase_quote: str | None = None
```

   `kw_only=True` is non-negotiable — the struct interleaves required + optional fields, and Python 3.13's stricter msgspec refuses that without it. Don't strip the modifier when copy-pasting.
4. **Populate it.** Decide which code path writes to it. For our example, `chase.py::chase_order` writes after each depth quote fetch.
5. **Frontend render** (optional). Add a column to `OrderCard.svelte` or `OrderTab.svelte` ag-Grid config; render via cellRenderer.
6. **Verify.** `psql -d ramboq_dev -c "\d algo_orders"` shows the new column; `pytest`; e2e if frontend-visible.

⚠️ **Never write a migration script.** The `init_db` `ALTER TABLE ... IF NOT EXISTS` pattern is the migration mechanism. We don't use Alembic.

---

33. Recipe: add a new background task

Scenario: you want `_task_unrealized_pnl_alert` that fires every 60s during market hours.

Steps

1. **Define the coroutine** in `backend/api/background.py`. Copy the shape of `_task_oco_pair_watcher` — it's the simplest template:

```
async def _task_unrealized_pnl_alert():
    interval = 60
    while True:
        try:
            await _run_unrealized_pnl_check()
        except Exception as e:
            logger.exception(f"_task_unrealized_pnl_alert failed: {e}")
        await asyncio.sleep(interval)
```

The try/except around the loop body is non-negotiable — without it, an uncaught exception silently kills the task forever.

2. **Spawn it at startup.** In `backend/api/app.py::on_startup`:

```
asyncio.create_task(_task_unrealized_pnl_alert())
```

3. **Gate by market hours** if appropriate. Use `is_any_segment_open()` from `backend/shared/helpers/date_time_utils.py`:

```
if not is_any_segment_open():
    await asyncio.sleep(interval)
    continue
```

4. **Gate by capability flag.** If the task hits an external service, wrap in `is_enabled('telegram' | 'mail' | …)` so dev branches don't spam.

5. **Verify.** Start dev (`uvicorn backend.api.app:app`), watch `.log/api_log_file` for the task's INFO/DEBUG logs, kill, restart, confirm it picks up cleanly.

⚠️ **Do not use `time.sleep`.** Always `await asyncio.sleep(...)`. Sync sleep blocks the entire event loop.

---

34. Recipe: add a new agent action

Scenario: you want `square_off_underlying` so an agent can close every position on a given underlying. This is a broker-hitting (live) action.

Steps

1. **Decide: live or noop?** Broker-hitting actions (place, cancel, close, chase, etc.) are live. Non-broker actions (send summary, log, deactivate agent) are noop. Our example is live.

2. **Add to `BROKER_ACTIONS`** in `backend/api/algo/actions.py` (line ~33):

```
BROKER_ACTIONS = {
    "place_order", ..., "square_off_underlying",  # add here
}
```

3. **Add the handler** in `backend/api/algo/actions_live.py`. Live handlers take `(agent, context, params)`:

```
async def _action_live_square_off_underlying(
    agent: Agent,
    context: dict[str, Any],
    params: dict,
) -> None:
    underlying = params.get("underlying")
    if not underlying:
        logger.error(f"Agent [{agent.slug}]: missing underlying param")
        raise ValueError("missing underlying")
    # Broker call: get positions for underlying, close all
    ...
```

4. **Register in `_LIVE_ACTION_HANDLERS`** in `actions.py` (line ~196):

```
_LIVE_ACTION_HANDLERS: dict[str, _HandlerRef] = {
    ...,
    "square_off_underlying": _HandlerRef(
        "backend.api.algo.actions_live",
        "_action_live_square_off_underlying"
    ),
}
```

5. **Add token to agent grammar** in `backend/config/grammars/agent_grammar.yaml`:

```
- grammar_kind: action
  token_kind: action_type
  token: square_off_underlying
  value_type: string
  description: Close all positions on a given underlying
  resolver: backend.api.algo.actions_live._action_live_square_off_underlying
  params_schema:
    required: [underlying]
    properties:
      underlying:
        type: string
        description: "E.g. NIFTY, BANKNIFTY"
```

6. **Optional: add sim/paper branches** in `backend/api/algo/actions_sim.py` and `actions_paper.py` if your action has trading logic. Check if those files switch on action type — if so, add your type.
7. **Verify.** Create a test agent via `/admin/agents`, set action to `square_off_underlying`, fire-in-simulator, confirm event logged + (for live mode) broker call executed.

---

35. Recipe: add a new template field (worked example)

Scenario: add `tp_breakeven_lock: bool` — when true, after TP1 fires, modify SL to entry price (free trade).

Steps

1. **Backend column** in `backend/api/models.py::OrderTemplate`:

```
tp_breakeven_lock: Mapped[bool | None] = mapped_column(Boolean, nullable=True)
```

2. **Idempotent ALTER** in `backend/api/database.py::init_db`:

```
await conn.execute(text(
    "ALTER TABLE order_templates ADD COLUMN IF NOT EXISTS tp_breakeven_lock BOOLEAN"
))
```

3. **Schemas** in `backend/api/schemas.py`:
   - `OrderTemplate` (response): `tp_breakeven_lock: bool | None = None`
   - `OrderTemplateCreate`, `OrderTemplatePatch`: same field
   - `TicketOrderRequest`: optional `tp_breakeven_lock_override: bool | None = None` if you want per-submit override
   - `BasketLeg`: same per-leg
4. **Override JSON build** in `backend/api/routes/orders.py::_build_overrides_json` (search for the function):

```
if data.tp_breakeven_lock_override is not None:
    result["tp_breakeven_lock"] = data.tp_breakeven_lock_override
```

5. **Plan resolution** in `backend/api/algo/template_attach.py::resolve_template_plan`:

```
breakeven_lock = _pick("tp_breakeven_lock", template, overrides)
```

   Then emit the appropriate behavior — for breakeven lock, you'd persist it into `attached_gtts_json` and watch for TP1 fill events.
6. **Seeded defaults** in `backend/api/algo/templates_seed.py::SYSTEM_TEMPLATES` — add to whichever defaults should ship with it ON.
7. **Frontend management UI** at `frontend/src/lib/automation/BracketsTab.svelte`:
   - Add a toggle to the create/edit form
   - Surface it in the listing
8. **Frontend override input** in `frontend/src/lib/SymbolPanel.svelte`:
   - Add a `_sharedTpBreakevenLockOverride = $state(null)` shell-level
   - Render in the Template row alongside the existing override inputs
   - Reset on template change (search the `$effect` block that watches `_sharedTemplateId`)
   - Pass through to OrderTicket and per-leg basket logic
9. **Preview chip.** Backend handles the math; frontend just renders. The preview will automatically pick up the new override because `previewTicketTemplate` passes through the full overrides dict.
10. **Update this doc.** Add a row to §10 (4-default matrix) if any default ships with it, and to §11 (merge order) if your field has unusual merge semantics.

11. **Verify.** `svelte-check`, `pytest`, e2e on `dev.ramboq.com`: create a template with the field, place a paper order, watch the chain of events in `.log/api_log_file`.

---

36. Recipe: add a new broker capability flag

Scenario: you want to track whether each broker supports iceberg orders (currently not in the matrix).

Steps

1. **Add the field** to `backend/brokers/capabilities.py::BrokerCapabilities`:

```
@dataclass(frozen=True)
class BrokerCapabilities:
    ...                       # existing fields
    iceberg_order: bool       # No default — force explicit setting per broker
```

   Note the discipline: real fields don't get defaults (audit B-5 lesson). Every broker constant must set every field.

2. **Set per-broker explicitly** in **all four** constants — KITE/DHAN/GROWW plus the `UNKNOWN_CAPS` fallback at the bottom of `capabilities.py`. Missing UNKNOWN_CAPS will crash the unknown-broker code path at runtime:

```
KITE_CAPS    = BrokerCapabilities(..., iceberg_order=True)
DHAN_CAPS    = BrokerCapabilities(..., iceberg_order=False)
GROWW_CAPS   = BrokerCapabilities(..., iceberg_order=False)
UNKNOWN_CAPS = BrokerCapabilities(..., iceberg_order=False)  # conservative default
```

3. **Capability registry** in `capabilities.py::CAPS_BY_BROKER_ID` already routes by `broker_id` — no change needed.

4. **Frontend warning helper** in `frontend/src/lib/data/brokerCapWarnings.js`:
   - Update the warning-aggregation logic to surface a warning when a template asks for an iceberg leg against a broker where `!caps.iceberg_order`.

5. **Consumer code** queries via `get_broker(account).capabilities.iceberg_order` or the HTTP endpoint `/api/admin/brokers/{account}/capabilities`.

6. **Verify.** Inspect `/admin/brokers` page; the new cap should surface in the row.

---

37. Recipe: add a new page

Scenario: new admin page at `/admin/funds-history`.

Steps

1. **Create the route file** `frontend/src/routes/(algo)/admin/funds-history/+page.svelte`. Copy structure from `/admin/brokers/+page.svelte` — it's the simplest admin page template.

2. **Page header.** Use the canonical pattern (see "Page-header rule" in CLAUDE.md or §17 of this doc):

```
<div class="page-header">
  <span class="algo-title-group">
    <h1 class="page-title-chip">Funds History</h1>
  </span>
  <span class="algo-ts">{$nowStamp}</span>
  <span class="ml-auto"></span>
  <span class="page-header-actions">
    <RefreshButton onClick={load} loading={_loading} label="Refresh" />
    <PageHeaderActions />
  </span>
</div>
```

3. **Navbar entry** in `frontend/src/routes/(algo)/+layout.svelte` `navItems`. Pick a group (`monitor` | `analyze` | `modes` | `build` | `config`). Set `adminOnly: true` if it should hide in demo mode.

4. **Data loading.** Always:
   - Use `$effect` for mount + cleanup (not legacy onMount)
   - Use `marketAwareInterval` from `$lib/stores` for polling (not raw setInterval)
   - Read via `$lib/api.js` wrappers

5. **Verify.** Visit the route, check navbar entry, confirm Refresh works, watch console for errors. Playwright spec if non-trivial.

---

38. Recipe: add a setting

Scenario: add `chase.max_consecutive_errors` so operator can tune the abort threshold (currently hardcoded `_MAX_CHASE_ERRORS=5`).

Steps

1. **Add to `SEEDS`** in `backend/shared/helpers/settings.py`:

```
("chase", "chase.max_consecutive_errors", "int", 5,
 "Abort a chase after this many consecutive API failures", None, "errors"),
```

2. **Read it** wherever the constant was used. Replace `_MAX_CHASE_ERRORS` with `get_int('chase.max_consecutive_errors', 5)`. The cache invalidates on every PATCH; reads are O(1).

3. **Verify.** Visit `/admin/settings` → confirm new row in the Chase bucket. Edit it, watch the change take effect on next chase iteration without restart.

⚙ **TECH** — the seeder preserves operator overrides on deploy; only the description / schema / default_value refresh. So bumping the default in code only affects fresh installs.

---

39. Recipe: change an existing default template

Scenario: operator wants `default-long-option` to have SL −40% instead of no SL.

Steps

1. **Edit `SYSTEM_TEMPLATES`** in `backend/api/algo/templates_seed.py`:

```
{
    "name": "default-long-option",
    ...
    "sl_pct": 40,  # was None
    "sl_type": "LIMIT",
}
```

2. **Re-seeder behavior.** On startup, `seed_templates` (in `templates_seed.py`) rebuilds system templates by `name` — operator's edits to custom templates are preserved, but system templates are overwritten. **The operator's pulls of `default-long-option` will get the new SL on next deploy.**

3. **If operator has saved-instance edits** (i.e. clicked Edit on a system template and saved), those land in a separate row keyed by user_id. They survive system re-seed. To force-refresh, the operator deletes their saved copy.

4. **Verify.** Restart dev, hit `/automation?tab=brackets`, confirm SL value is updated. Place a test order — fill should trigger SL GTT placement.

---

40. Recipe: wire a new notification channel

Scenario: add Slack as a notify channel alongside Telegram + email.

Steps

1. **Add config keys** in `backend/config/secrets.yaml` (manually on server, gitignored):

```
slack_webhook_url: "https://hooks.slack.com/services/..."
```

2. **Add capability flag** in `backend/config/backend_config.yaml::cap_in_dev`:

```
slack: True
```

`is_enabled('slack')` returns `True` on main always, else respects this flag.

3. **Add the helper** in `backend/shared/helpers/alert_utils.py`:

```
def _send_slack(message: str) -> None:
    if not is_enabled('slack'):
        return
    webhook = secrets.get('slack_webhook_url')
    if not webhook:
        return
    requests.post(webhook, json={"text": message}, timeout=5)
```

4. **Add the grammar token** for the notify channel in `backend/config/grammars/agent_grammar.yaml`:

```
- grammar_kind: notify
  token_kind: channel
  token: slack
  description: Send alert via Slack webhook
  resolver: backend.shared.helpers.alert_utils._send_slack
```

5. **Wire it in `_dispatch`** in `alert_utils.py` — for each notify event, check `channel == 'slack'` and call `_send_slack`.
6. **UI checkbox** in agent editor (`frontend/src/routes/(algo)/automation/+page.svelte`) — add Slack to the events grid.
7. **Verify.** Create a test agent with Slack notify, fire in simulator, confirm message lands.

---

41. Recipe: ship a fix to dev + main

Scenario: you've finished a small fix on `dev` branch and want to deploy to prod.

Steps

1. **Run pre-flight checks locally.**
   - `cd backend && pytest` (or scoped to affected files)
   - `cd frontend && npm run check` (svelte-check)
   - `cd frontend && npx playwright test --workers=1` if frontend-touching
2. **Commit on dev.** Use the conventional format:

```
<Sprint/scope>: <one-line summary>

<optional body explaining why>

Co-Authored-By: Claude Opus 4.7 (1M context) <noreply@anthropic.com>
```

3. **Push dev.** `git push origin dev`. Webhook triggers dev deploy automatically. Watch logs:

```
ssh ramboq "tail -f /opt/ramboq_dev/.log/api_log_file"
```

4. **Verify on `dev.ramboq.com`.** Manual smoke test of the changed feature.
5. **Merge to main.**

```
git checkout main
git merge --ff-only dev
git push origin main
git checkout dev  # back to dev for next change
```

6. **Webhook deploys prod automatically.** Watch `/opt/ramboq/.log/api_log_file`.
7. **Telegram deploy ping** lands in `RamboQuant Alerts` group — confirm.

⚠️ Don't squash-merge. We keep linear history via `--ff-only`. ⚠️ Don't tag releases; we deploy on every push.

---

42. Cross-cutting checklist before every commit

Run mentally before every commit. Skipping any of these has burned us before:

- ☐ **Demo mode honored?** Read paths mask accounts; write paths return 403 or downgrade to paper (§22).
- ☐ **Idempotency on side-effects?** Anything that places orders/GTTs needs a guard (§3.2).
- ☐ **Mode resolution?** Order code reads `row.mode` instead of branching by `if live` (§3.4).
- ☐ **Logger discipline?** No `print()`; no logger calls in hot loops (§25).

- [ ] **Hot-loop sleep?** `asyncio.sleep`, never `time.sleep` (§4.3).
- [ ] **TODO/FIXME?** None in code paths; finish or extract.
- [ ] **Old tests still pass?** `pytest` on the closest test file.
- [ ] **svelte-check clean?** No new errors introduced.
- [ ] **CLAUDE.md updated?** If a fact in CLAUDE.md changes, fix it here.
- [ ] **DESIGN_GUIDE.md updated?** This file. Especially recipe sections + line-anchored grep targets if functions moved.
- [ ] **Sprint/audit fix labeled?** Commit body mentions the gap ID or sprint letter so `git log --grep` finds it.
- [ ] **Co-author trailer?** Co-Authored-By line at the end of the commit body.

If you can't tick every box, hold the commit. The cost of pausing is low; the cost of regression is high.

Where to learn more

If you want to learn...	Read
How the operator uses the platform end-to-end	`USER_GUIDE.md`
Day-to-day operations + troubleshooting	`ADMIN_GUIDE.md`
Agent authoring + testing	`AGENTS_GUIDE.md`
Simulator scenarios + Run-in-Simulator	`SIMULATOR_GUIDE.md`
Lab + MCP-driven research workflow	`LAB_MCP_GUIDE.md`
Operator's-eye-view of every page	`CLAUDE.md` (large, but indexed)
Architecture + change recipes	**This document**

Glossary

Curated index of the terms that appear repeatedly in this guide. Alphabetized so you can jump-lookup without scrolling the TOC.

Term	Meaning
Action	Side-effect an agent performs when its condition fires — place order, close position, alert. See §34.
AgentEngine	Declarative rule runner. Reads `agents` table + built-in `BUILTIN_AGENTS`, evaluates conditions each cycle, dispatches actions. §22.5, §34.
Alert	Runtime event produced when an agent condition fires. Persisted to `agent_events`; may or may not have a Notify or Action. Distinct from **Notify**.
Basket order	Multi-account / multi-leg order dispatched atomically. `POST /api/orders/basket` groups by account and fans out via `asyncio.gather`. §6, §22.10.
Broker abstraction	Uniform Python interface (Kite / Dhan / Groww adapters) so route handlers stay broker-agnostic. `backend/brokers/adapters/`. §14.
Chase loop	Adaptive limit-order engine that re-quotes toward the touch until filled or capped. Spread-aware. §7, §12.
CardHeader.svelte	Unified card title bar using CSS custom properties (`--ch-*`) for theming; embeds `CardControls`; wired to 9 card header sites. §18.5.
cap_in_dev	Nested dict of capability flags in `backend_config.yaml`. All True on `main`, per-branch on dev. Gated via `is_enabled('<cap>')`.
ChaseAggPicker.svelte	L/M/H aggression picker with `variant='ticket'|'panel'` prop. Extracted to standalone component. §18.5.
close_settled	Second phase of the snapshot lifecycle — fires 15 min after `<exch>:close`. UPSERTs broker's weighted-avg-last-30-min close price.
conn_service	Standalone Litestar app on `/tmp/ramboq_conn.sock` that owns broker sessions (Kite WebSocket, Dhan/Groww tokens). Restart independent of the main API.
Demo mode	Signed-out + prod branch — read-only + PII-masked. Write paths return 403 or degrade to paper. §22.
Firm NAV	`cash_sod + option_premium + Σ position.unrealised + Σ holdings.cur_val`. Canonical formula (v4) in `backend/api/algo/nav.py:compute_firm_nav`.
GTT	Good-Till-Triggered order — broker-side conditional order. Kite native; Dhan OCO leg; Groww emulated. §9.
@for_all_accounts	Decorator that fans out a broker call across every account and concatenates the results. `pd.concat(..., ignore_index=True)` at call site.
Idempotency guard	Column like `attached_gtts_json IS NULL` that ensures a side-effect fires at most once even under postback retries. §3.2.
KiteTicker	Persistent Kite WebSocket. One per conn_service process. Ticks land in `/dev/shm/ramboq_ticks` mmap; main API reads via `MmapTickReader`.
LP unit	Limited-Partner accounting unit. `units_held × nav_per_unit = slice`. §22.6.
MarketPulse	Two-side ag-Grid page (`/pulse`) — pinned watchlists + movers left, positions + holdings right. Canonical operator surface.
MCP	Model Context Protocol — Claude-Code integration exposing 25 platform tools (17 read-only + 2 persist + 6 write-gated).
Notify	Delivery channel wrapping an Alert — telegram / email / websocket / log. Vocabulary chain: Agent → Alert → Notify → Action.
OHLCV	Open-High-Low-Close-Volume daily bars. Cached in `ohlcv_store` (3-tier: LRU → PostgreSQL → broker). 5-year retention.
Paper mode	Live-quote execution against `PaperTradeEngine` — no broker orders. Mode 2 of the confidence ladder (sim → paper → shadow → live).
PBKDF2	Password hash algorithm (SHA-256, 210k iters). JWT signing separate — HS256.
Preflight	Fat-finger + lot-multiple guards before order placement (G1, G2). Parallelized via `asyncio.gather`. §22.10.
Proxy hedge	Cross-reference between holdings and option roots. β-regressed. `hedge_proxies` table.
Refresh cycle mode	Persistence override — `off` / `soft` / `hard`. Bypasses cache tiers when defect-recovering. Runtime-only, resets on restart.
Shadow mode	Log-only mode — validates payload against real broker but does not execute. Mode 5, prod-only.
snapshot_extras	Payload block in `daily_book.payload_json` carrying open/high/low/close_settled/day_change_val/... for closed-hours reads.
Snapshot lifecycle	Per-exchange event sequence — `open` → `close` (first-cut snapshot) → `close_settled` (broker's settled close).
SSOT	Single Source Of Truth. See `baseDayPnlForPosition` (Day P&L), `compute_firm_nav` (NAV), `resolve_current_price` (LTP resolver).
Template attach	Post-fill automation — attaches GTT exits and take-profit legs to a filled parent. Idempotent via `parent_order_id`. §9.
TemplateBar.svelte	TP/SL/wing override row extracted from SymbolPanel; uses `$bindable()` for all four overrides. §18.5.
Ticker mmap	`/dev/shm/ramboq_ticks` — fixed 4096-slot shared-memory buffer, version-word atomic, lock-free reads. Main API tails it via `MmapTickReader`.
Trail stop	Stop-loss that ratchets toward the touch on favorable moves. Broker-side (Kite trail_gtt) or emulated. §13.
Virtual root	MCX/CDS synthetic symbol (e.g. `CRUDEOIL` = front-month, `CRUDEOIL_NEXT` = back-month). Resolver: `symbol_resolver.py`.

Alphabetical section index

Quick-jump index by first significant word — useful when you remember a name but not the section number.

Section	§
Architecture overview	§1
Audit log — forensic trail	§22.7
Background task topology	§20
Broker abstraction	§14
Broker abstraction — implementation detail	§14.5
Broker gotchas	§16
Chart indicator system	§22.15
Component + module extractions — Phase 1–3	§18.4–18.5
Chase loop invariants	§12
Chase loop lifecycle	§7
CSV export: all ag-Grid cards	§22.19
Concurrency model	§4
Core architectural principles	§3
Cross-cutting checklist before every commit	§42
Data layer — implementation detail	§4.5
Data refresh — PositionStrip + Dashboard	§21
Database schema overview	§4.6
Demo banner + Showcase + Nav + Derivatives Exp Close	§22.25
Demo mode	§22
Deployment notes	§26
Docs folder reorganisation + spec files	§22.20
History — orders / trades / funds	§22.9
Investor portal — token-as-credential	§22.5
Investor portal — units-based NAV math	§22.6
Logging discipline	§25
Market-status probe	§22.14
Metrics + performance tracking	§4.9
Modal consolidation — ActivityLogModal + fullscreen inset	§22.27
Navbar audit — rename + resequence	§22.11
NavStrip — lifetime slot SSOT fix	§22.18
Operator's mental model	§30
Order placement — basket (Chain tab)	§6
Order placement — single ticket	§5
Order placement latency	§22.10
Postback fan-out — book_changed bus	§22.8
Reading order for a new developer	§28
Recipe — add a background task	§33
Recipe — add a broker capability flag	§36
Recipe — add a column to a table	§32
Recipe — add a new agent action	§34
Recipe — add a new page	§37
Recipe — add a new route	§31
Recipe — add a new template field	§35
Recipe — add a setting	§38

Section	§
Recipe — change a default template	§39
Recipe — ship a fix to dev + main	§41
Recipe — wire a notification channel	§40
Retention policies	§4.8
Sprint history + audit fixes	§27
Table relationships	§4.7
Tech stack — at a glance	§2
Template attach pipeline	§9
Template matrix (4 defaults)	§10
Template override merge	§11
Testing philosophy	§24
The order/chase/template tripod	§8
The preview pipeline	§19
Trail-stop subsystem	§13
UX audit slice D	§22.13
When in doubt	§29
#audit workflow + postback scaffold	§22.12